PennOS

Generated by Doxygen 1.9.1

1 Class Index 1
1.1 Class List . . . 1

2 File Index 3
2.1 File List . . . 3

3 Class Documentation 7
3.1 fat_metadata_st Struct Reference . . . 7
3.1.1 Detailed Description . . . 7
3.1.2 Member Data Documentation . . . 7
3.1.2.1 block_size . . . 7
3.1.2.2 tot_blocks . . . 8
3.2 fat_ut Union Reference . . . 8
3.2.1 Detailed Description . . . 8
3.2.2 Member Data Documentation . . . 8
3.2.2.1 metadata . . . 8
3.2.2.2 next . . . 8
3.3 file_st Struct Reference . . . 9
3.3.1 Detailed Description . . . 9
3.3.2 Member Data Documentation . . . 9
3.3.2.1 __reserved . . . 9
3.3.2.2 first_block . . . 9
3.3.2.3 mtime . . . 9
3.3.2.4 name . . . 10
3.3.2.5 perm . . . 10
3.3.2.6 size . . . 10
3.3.2.7 type . . . 10
3.4 job_st Struct Reference . . . 10
3.4.1 Detailed Description . . . 10
3.4.2 Member Data Documentation . . . 11
3.4.2.1 age . . . 11
3.4.2.2 cmd_info . . . 11
3.4.2.3 command . . . 11
3.4.2.4 pid . . . 11
3.5 job_table_st Struct Reference . . . 11
3.5.1 Member Function Documentation . . . 12
3.5.1.1 vector() . . . 12
3.5.2 Member Data Documentation . . . 12
3.5.2.1 curr_age . . . 12
3.5.2.2 latest_job . . . 12
3.6 k_fd_entry_t Struct Reference . . . 12
3.6.1 Detailed Description . . . 13
3.6.2 Member Data Documentation . . . 13

3.6.2.1 curr_block . . . 13
3.6.2.2 hfd . . . 13
3.6.2.3 mode . . . 13
3.6.2.4 open_file_idx . . . 13
3.6.2.5 refcount . . . 13
3.6.2.6 target_offset . . . 13
3.6.2.7 virtual_offset . . . 14
3.7 k_open_file_entry_t Struct Reference . . . 14
3.7.1 Detailed Description . . . 14
3.7.2 Member Data Documentation . . . 14
3.7.2.1 file . . . 14
3.7.2.2 refcount . . . 14
3.8 parsed_command Struct Reference . . . 15
3.8.1 Detailed Description . . . 15
3.9 pcb_t Struct Reference . . . 15
3.9.1 Detailed Description . . . 16
3.9.2 Member Function Documentation . . . 16
3.9.2.1 vector() . . . 16
3.9.3 Member Data Documentation . . . 16
3.9.3.1 alive_child . . . 17
3.9.3.2 ign_int . . . 17
3.9.3.3 ign_stop . . . 17
3.9.3.4 nex_sibling . . . 17
3.9.3.5 parent . . . 17
3.9.3.6 prev_sibling . . . 17
3.9.3.7 process_name . . . 17
3.9.3.8 sched_priority . . . 17
3.9.3.9 state . . . 18
3.9.3.10 terminated_by_signal . . . 18
3.9.3.11 thread . . . 18
3.9.3.12 waitpid_target . . . 18
3.9.3.13 wakeup_time . . . 18
3.9.3.14 wstatus . . . 18
3.9.3.15 zombie_child . . . 18
3.10 pcb_table_t Struct Reference . . . 19
3.10.1 Detailed Description . . . 19
3.10.2 Member Function Documentation . . . 19
3.10.2.1 vector() . . . 19
3.10.3 Member Data Documentation . . . 19
3.10.3.1 avail_process . . . 19
3.11 process_info_t Struct Reference . . . 20
3.11.1 Detailed Description . . . 20

3.11.2 Member Data Documentation . . . 20
3.11.2.1 name . . . 20
3.11.2.2 par_pid . . . 20
3.11.2.3 pid . . . 20
3.11.2.4 priority . . . 21
3.11.2.5 state . . . 21
3.12 spthread_fwd_args_st Struct Reference . . . 21
3.13 spthread_meta_st Struct Reference . . . 21
3.14 spthread_signal_args_st Struct Reference . . . 22
3.15 spthread_st Struct Reference . . . 22
3.16 thread_info_st Struct Reference . . . 22
3.16.1 Detailed Description . . . 23
3.17 vector_info_st Struct Reference . . . 23

4 File Documentation **25**
4.1 src/io/fat/k_fat.c File Reference . . . 25
4.1.1 Detailed Description . . . 26
4.1.2 Function Documentation . . . 26
4.1.2.1 k_get_blk_size() . . . 26
4.1.2.2 k_get_block_size() . . . 26
4.1.2.3 k_get_data_pos() . . . 27
4.1.2.4 k_get_fat() . . . 27
4.1.2.5 k_get_fat_fd() . . . 28
4.1.2.6 k_has_mount() . . . 28
4.1.2.7 k_mkfs() . . . 28
4.1.2.8 k_mount() . . . 29
4.1.2.9 k_release_block() . . . 29
4.1.2.10 k_release_fat_chain() . . . 29
4.1.2.11 k_request_block() . . . 30
4.1.2.12 k_sizeof_fat() . . . 30
4.1.2.13 k_start_block() . . . 30
4.1.2.14 k_unmount() . . . 31
4.2 src/io/fat/k_fat.h File Reference . . . 31
4.2.1 Detailed Description . . . 32
4.2.2 Function Documentation . . . 32
4.2.2.1 k_get_blk_size() . . . 32
4.2.2.2 k_get_block_size() . . . 32
4.2.2.3 k_get_data_pos() . . . 33
4.2.2.4 k_get_fat() . . . 33
4.2.2.5 k_get_fat_fd() . . . 34
4.2.2.6 k_has_mount() . . . 34
4.2.2.7 k_mkfs() . . . 34

4.2.2.8 k_mount() . . . 35
4.2.2.9 k_release_block() . . . 35
4.2.2.10 k_release_fat_chain() . . . 36
4.2.2.11 k_request_block() . . . 36
4.2.2.12 k_sizeof_fat() . . . 36
4.2.2.13 k_start_block() . . . 37
4.2.2.14 k_unmount() . . . 37
4.3 src/io/fat/s_fat.c File Reference . . . 37
4.3.1 Detailed Description . . . 38
4.3.2 Function Documentation . . . 38
4.3.2.1 s_has_mount() . . . 38
4.3.2.2 s_mkfs() . . . 38
4.3.2.3 s_mount() . . . 39
4.3.2.4 s_unmount() . . . 39
4.4 src/io/fat/s_fat.h File Reference . . . 39
4.4.1 Detailed Description . . . 40
4.4.2 Function Documentation . . . 40
4.4.2.1 s_has_mount() . . . 40
4.4.2.2 s_mkfs() . . . 40
4.4.2.3 s_mount() . . . 41
4.4.2.4 s_unmount() . . . 41
4.5 src/io/file/h_file.c File Reference . . . 42
4.5.1 Detailed Description . . . 42
4.5.2 Function Documentation . . . 42
4.5.2.1 h_close() . . . 42
4.5.2.2 h_lseek() . . . 43
4.5.2.3 h_open() . . . 43
4.5.2.4 h_read() . . . 44
4.5.2.5 h_write() . . . 44
4.6 src/io/file/k_file.c File Reference . . . 44
4.6.1 Detailed Description . . . 45
4.6.2 Function Documentation . . . 45
4.6.2.1 k_close() . . . 45
4.6.2.2 k_ext_open() . . . 46
4.6.2.3 k_ext_read() . . . 46
4.6.2.4 k_lseek() . . . 47
4.6.2.5 k_mv() . . . 47
4.6.2.6 k_open() . . . 48
4.6.2.7 k_read() . . . 49
4.6.2.8 k_unlink() . . . 49
4.6.2.9 k_write() . . . 50
4.7 src/io/file/s_file.c File Reference . . . 50

4.7.1 Detailed Description . . . 51
4.7.2 Function Documentation . . . 51
4.7.2.1 s_chmod() . . . 51
4.7.2.2 s_close() . . . 52
4.7.2.3 s_ext_open() . . . 52
4.7.2.4 s_get_open_mode() . . . 53
4.7.2.5 s_lseek() . . . 53
4.7.2.6 s_mv() . . . 53
4.7.2.7 s_open() . . . 54
4.7.2.8 s_read() . . . 54
4.7.2.9 s_same_source() . . . 55
4.7.2.10 s_unlink() . . . 55
4.7.2.11 s_write() . . . 56
4.8 src/io/file/s_file.h File Reference . . . 56
4.8.1 Detailed Description . . . 57
4.8.2 Function Documentation . . . 57
4.8.2.1 s_chmod() . . . 57
4.8.2.2 s_close() . . . 57
4.8.2.3 s_ext_open() . . . 58
4.8.2.4 s_get_open_mode() . . . 58
4.8.2.5 s_lseek() . . . 59
4.8.2.6 s_mv() . . . 59
4.8.2.7 s_open() . . . 60
4.8.2.8 s_read() . . . 60
4.8.2.9 s_same_source() . . . 60
4.8.2.10 s_unlink() . . . 61
4.8.2.11 s_write() . . . 61
4.9 src/io/filetable/filetable.h File Reference . . . 62
4.9.1 Detailed Description . . . 62
4.10 src/io/filetable/k_filetable.c File Reference . . . 62
4.10.1 Detailed Description . . . 64
4.10.2 Function Documentation . . . 64
4.10.2.1 apply_offset() . . . 64
4.10.2.2 fd_alloc() . . . 64
4.10.2.3 file_alloc() . . . 65
4.10.2.4 k_fd_alloc() . . . 65
4.10.2.5 k_fd_alloc_h() . . . 66
4.10.2.6 k_fd_close() . . . 66
4.10.2.7 k_fd_dup() . . . 67
4.10.2.8 k_fd_file_ref() . . . 67
4.10.2.9 k_fd_get_hfd() . . . 68
4.10.2.10 k_fd_get_offset() . . . 68

4.10.2.11 k_fd_get_size() . . . 69
4.10.2.12 k_fd_has_entry() . . . 69
4.10.2.13 k_fd_is_open() . . . 69
4.10.2.14 k_fd_is_write_locked() . . . 70
4.10.2.15 k_fd_mode() . . . 70
4.10.2.16 k_fd_set_offset() . . . 71
4.10.2.17 k_fd_walk() . . . 71
4.10.2.18 k_lfd_get() . . . 72
4.10.2.19 k_lfd_put() . . . 72
4.10.2.20 k_lfd_release() . . . 73
4.11 src/io/filetable/k_filetable.h File Reference . . . 73
4.11.1 Detailed Description . . . 74
4.11.2 Function Documentation . . . 74
4.11.2.1 apply_offset() . . . 74
4.11.2.2 k_fd_alloc() . . . 75
4.11.2.3 k_fd_alloc_h() . . . 75
4.11.2.4 k_fd_close() . . . 76
4.11.2.5 k_fd_dup() . . . 76
4.11.2.6 k_fd_file_ref() . . . 77
4.11.2.7 k_fd_get_hfd() . . . 77
4.11.2.8 k_fd_get_offset() . . . 78
4.11.2.9 k_fd_get_size() . . . 78
4.11.2.10 k_fd_has_entry() . . . 78
4.11.2.11 k_fd_init() . . . 79
4.11.2.12 k_fd_is_open() . . . 79
4.11.2.13 k_fd_is_write_locked() . . . 79
4.11.2.14 k_fd_mode() . . . 80
4.11.2.15 k_fd_set_offset() . . . 80
4.11.2.16 k_fd_walk() . . . 81
4.11.2.17 k_lfd_get() . . . 81
4.11.2.18 k_lfd_put() . . . 82
4.11.2.19 k_lfd_release() . . . 82
4.12 src/io/fstd.h File Reference . . . 83
4.12.1 Detailed Description . . . 83
4.13 src/io/path/k_path.c File Reference . . . 83
4.13.1 Detailed Description . . . 84
4.13.2 Function Documentation . . . 84
4.13.2.1 k_dir_add_entry() . . . 84
4.13.2.2 k_dir_rem_entry() . . . 85
4.13.2.3 k_dir_search() . . . 85
4.13.2.4 k_dir_upd_entry() . . . 86
4.13.2.5 k_get_root_dir() . . . 86

4.13.2.6 k_ls() . . . 86
4.14 src/io/path/k_path.h File Reference . . . 87
4.14.1 Detailed Description . . . 87
4.14.2 Function Documentation . . . 87
4.14.2.1 k_dir_add_entry() . . . 87
4.14.2.2 k_dir_rem_entry() . . . 88
4.14.2.3 k_dir_search() . . . 89
4.14.2.4 k_dir_upd_entry() . . . 89
4.14.2.5 k_get_root_dir() . . . 90
4.14.2.6 k_ls() . . . 90
4.15 src/io/path/s_path.c File Reference . . . 91
4.15.1 Detailed Description . . . 91
4.15.2 Function Documentation . . . 91
4.15.2.1 print_file_entry() . . . 91
4.15.2.2 s_ls() . . . 92
4.16 src/io/path/s_path.h File Reference . . . 92
4.16.1 Detailed Description . . . 92
4.16.2 Function Documentation . . . 92
4.16.2.1 s_ls() . . . 93
4.17 src/io/term/k_term.c File Reference . . . 93
4.17.1 Detailed Description . . . 94
4.17.2 Function Documentation . . . 94
4.17.2.1 k_claim_terminal() . . . 94
4.17.2.2 k_get_terminal_owner() . . . 94
4.17.2.3 k_get_terminal_pass_parent() . . . 94
4.17.2.4 k_is_term() . . . 95
4.17.2.5 k_pass_terminal_to_next_child() . . . 95
4.18 src/io/term/k_term.h File Reference . . . 95
4.18.1 Detailed Description . . . 96
4.18.2 Function Documentation . . . 96
4.18.2.1 k_claim_terminal() . . . 96
4.18.2.2 k_get_terminal_owner() . . . 96
4.18.2.3 k_get_terminal_pass_parent() . . . 97
4.18.2.4 k_is_term() . . . 97
4.18.2.5 k_pass_terminal_to_next_child() . . . 98
4.19 src/io/term/s_term.c File Reference . . . 98
4.19.1 Detailed Description . . . 98
4.19.2 Function Documentation . . . 98
4.19.2.1 s_claim_terminal() . . . 98
4.19.2.2 s_get_terminal_owner() . . . 99
4.19.2.3 s_get_terminal_pass_parent() . . . 99
4.19.2.4 s_pass_terminal_to_next_child() . . . 99

4.20 src/io/term/s_term.h File Reference . . . 99
4.20.1 Detailed Description . . . 100
4.20.2 Macro Definition Documentation . . . 100
4.20.2.1 s_fprintf . . . 101
4.20.2.2 s_perror . . . 101
4.20.2.3 s_printf . . . 102
4.20.3 Function Documentation . . . 102
4.20.3.1 s_claim_terminal() . . . 102
4.20.3.2 s_get_terminal_owner() . . . 102
4.20.3.3 s_get_terminal_pass_parent() . . . 103
4.20.3.4 s_pass_terminal_to_next_child() . . . 103
4.21 src/pennfat.c File Reference . . . 103
4.21.1 Detailed Description . . . 104
4.21.2 Function Documentation . . . 104
4.21.2.1 main() . . . 104
4.22 src/pennfat.h File Reference . . . 105
4.22.1 Detailed Description . . . 105
4.23 src/pennos.c File Reference . . . 105
4.23.1 Detailed Description . . . 106
4.23.2 Function Documentation . . . 106
4.23.2.1 main() . . . 106
4.23.2.2 u_init_main() . . . 107
4.24 src/pennos.h File Reference . . . 107
4.24.1 Detailed Description . . . 108
4.24.2 Function Documentation . . . 108
4.24.2.1 u_init_main() . . . 108
4.25 src/process/lifecycle/k_lifecycle.c File Reference . . . 109
4.25.1 Detailed Description . . . 110
4.25.2 Function Documentation . . . 110
4.25.2.1 k_proc_cleanup() . . . 110
4.25.2.2 k_proc_create() . . . 110
4.25.2.3 k_proc_zombify() . . . 111
4.25.2.4 transfer_parent() . . . 111
4.26 src/process/lifecycle/k_lifecycle.h File Reference . . . 112
4.26.1 Detailed Description . . . 112
4.26.2 Function Documentation . . . 112
4.26.2.1 k_pcb_data() . . . 112
4.26.2.2 k_proc_cleanup() . . . 113
4.26.2.3 k_proc_create() . . . 113
4.26.2.4 k_proc_zombify() . . . 114
4.26.2.5 k_spthread_create() . . . 114
4.27 src/process/lifecycle/s_lifecycle.c File Reference . . . 115

4.27.1 Detailed Description . . . 115
4.27.2 Function Documentation . . . 115
4.27.2.1 process_spawner() . . . 115
4.27.2.2 s_exit() . . . 116
4.27.2.3 s_spawn() . . . 116
4.27.2.4 s_waitpid() . . . 117
4.28 src/process/lifecycle/s_lifecycle.h File Reference . . . 117
4.28.1 Detailed Description . . . 117
4.28.2 Function Documentation . . . 117
4.28.2.1 s_exit() . . . 118
4.28.2.2 s_spawn() . . . 118
4.28.2.3 s_waitpid() . . . 119
4.29 src/process/lifecycle/thread_info_t.h File Reference . . . 119
4.29.1 Detailed Description . . . 120
4.29.2 Typedef Documentation . . . 120
4.29.2.1 thread_info_t . . . 120
4.30 src/process/pcb/k_pcb.c File Reference . . . 120
4.30.1 Detailed Description . . . 121
4.30.2 Function Documentation . . . 121
4.30.2.1 k_next_process() . . . 121
4.30.2.2 k_pcb_claim() . . . 121
4.30.2.3 k_pcb_data() . . . 122
4.30.2.4 k_pcb_init() . . . 122
4.30.2.5 k_pcb_list_add() . . . 122
4.30.2.6 k_pcb_list_insert() . . . 123
4.30.2.7 k_pcb_list_remove() . . . 123
4.30.2.8 k_pcb_table_size() . . . 123
4.30.2.9 k_proc_release() . . . 123
4.31 src/process/pcb/k_pcb.h File Reference . . . 124
4.31.1 Detailed Description . . . 124
4.31.2 Function Documentation . . . 124
4.31.2.1 k_next_process() . . . 124
4.31.2.2 k_pcb_claim() . . . 125
4.31.2.3 k_pcb_data() . . . 125
4.31.2.4 k_pcb_init() . . . 126
4.31.2.5 k_pcb_list_add() . . . 126
4.31.2.6 k_pcb_list_insert() . . . 126
4.31.2.7 k_pcb_list_remove() . . . 127
4.31.2.8 k_pcb_table_size() . . . 127
4.31.2.9 k_proc_release() . . . 127
4.32 src/process/pcb/pcb_state_t.h File Reference . . . 128
4.32.1 Detailed Description . . . 128

4.32.2 Enumeration Type Documentation . . . 128
4.32.2.1 pcb_state_t . . . 128
4.33 src/process/pcb/pcb_t.h File Reference . . . 128
4.33.1 Detailed Description . . . 129
4.34 src/process/pcb/pcb_table_t.h File Reference . . . 129
4.34.1 Detailed Description . . . 129
4.35 src/process/pcb/s_pcb.c File Reference . . . 130
4.35.1 Detailed Description . . . 130
4.35.2 Function Documentation . . . 130
4.35.2.1 s_nice() . . . 130
4.35.2.2 s_proc_state() . . . 131
4.35.2.3 s_ps() . . . 131
4.36 src/process/pcb/s_pcb.h File Reference . . . 131
4.36.1 Detailed Description . . . 132
4.36.2 Function Documentation . . . 132
4.36.2.1 s_nice() . . . 132
4.36.2.2 s_proc_state() . . . 133
4.36.2.3 s_ps() . . . 133
4.37 src/process/pstd.h File Reference . . . 133
4.37.1 Detailed Description . . . 134
4.38 src/process/schedule/k_schedule.c File Reference . . . 134
4.38.1 Detailed Description . . . 135
4.38.2 Function Documentation . . . 136
4.38.2.1 k_can_interrupt() . . . 136
4.38.2.2 k_disable_interrupts() . . . 136
4.38.2.3 k_enable_interrupts() . . . 136
4.38.2.4 k_remove_int() . . . 136
4.38.2.5 k_remove_stop() . . . 137
4.38.2.6 k_sched_add() . . . 137
4.38.2.7 k_sched_block() . . . 137
4.38.2.8 k_sched_clean() . . . 138
4.38.2.9 k_sched_current_pid() . . . 138
4.38.2.10 k_sched_init() . . . 138
4.38.2.11 k_sched_remove() . . . 138
4.38.2.12 k_sched_run() . . . 139
4.38.2.13 k_sched_stop() . . . 139
4.38.2.14 k_sched_unblock() . . . 139
4.39 src/process/schedule/k_schedule.h File Reference . . . 139
4.39.1 Detailed Description . . . 140
4.39.2 Function Documentation . . . 141
4.39.2.1 k_can_interrupt() . . . 141
4.39.2.2 k_disable_interrupts() . . . 141

4.39.2.3 k_enable_interrupts() . . . 141
4.39.2.4 k_remove_int() . . . 141
4.39.2.5 k_remove_stop() . . . 142
4.39.2.6 k_sched_add() . . . 142
4.39.2.7 k_sched_block() . . . 142
4.39.2.8 k_sched_clean() . . . 143
4.39.2.9 k_sched_current_pid() . . . 143
4.39.2.10 k_sched_init() . . . 144
4.39.2.11 k_sched_remove() . . . 144
4.39.2.12 k_sched_run() . . . 144
4.39.2.13 k_sched_stop() . . . 145
4.39.2.14 k_sched_unblock() . . . 145
4.40 src/process/schedule/s_schedule.c File Reference . . . 145
4.40.1 Detailed Description . . . 146
4.40.2 Function Documentation . . . 146
4.40.2.1 s_getpid() . . . 146
4.40.2.2 s_sleep() . . . 146
4.41 src/process/schedule/s_schedule.h File Reference . . . 146
4.41.1 Detailed Description . . . 147
4.41.2 Function Documentation . . . 147
4.41.2.1 s_getpid() . . . 147
4.41.2.2 s_sleep() . . . 147
4.42 src/process/signal/k_signal.c File Reference . . . 148
4.42.1 Detailed Description . . . 148
4.42.2 Function Documentation . . . 148
4.42.2.1 k_notify_parent() . . . 148
4.42.2.2 k_pollpid() . . . 149
4.43 src/process/signal/k_signal.h File Reference . . . 149
4.43.1 Detailed Description . . . 149
4.43.2 Function Documentation . . . 150
4.43.2.1 k_notify_parent() . . . 150
4.43.2.2 k_pollpid() . . . 150
4.44 src/process/signal/s_signal.c File Reference . . . 151
4.44.1 Detailed Description . . . 151
4.44.2 Function Documentation . . . 152
4.44.2.1 s_can_sig_int() . . . 152
4.44.2.2 s_can_sig_stop() . . . 152
4.44.2.3 s_ignore_int() . . . 152
4.44.2.4 s_ignore_stop() . . . 153
4.44.2.5 s_kill() . . . 153
4.44.2.6 s_logout() . . . 154
4.45 src/process/signal/s_signal.h File Reference . . . 154

4.45.1 Detailed Description . . . 154
4.45.2 Function Documentation . . . 154
4.45.2.1 s_can_sig_int() . . . 154
4.45.2.2 s_can_sig_stop() . . . 155
4.45.2.3 s_ignore_int() . . . 156
4.45.2.4 s_ignore_stop() . . . 156
4.45.2.5 s_kill() . . . 157
4.45.2.6 s_logout() . . . 157
4.46 src/shell/commands.c File Reference . . . 158
4.47 src/shell/commands.h File Reference . . . 158
4.47.1 Detailed Description . . . 158
4.47.2 Function Documentation . . . 158
4.47.2.1 run_command() . . . 158
4.47.2.2 u_echo() . . . 159
4.47.2.3 u_man() . . . 160
4.48 src/shell/file/u_file.h File Reference . . . 160
4.48.1 Detailed Description . . . 160
4.48.2 Function Documentation . . . 160
4.48.2.1 u_cat() . . . 160
4.48.2.2 u_chmod() . . . 161
4.48.2.3 u_cp() . . . 161
4.48.2.4 u_ls() . . . 161
4.48.2.5 u_mv() . . . 162
4.48.2.6 u_rm() . . . 162
4.48.2.7 u_touch() . . . 163
4.49 src/shell/job/job_table.c File Reference . . . 163
4.49.1 Detailed Description . . . 163
4.49.2 Function Documentation . . . 164
4.49.2.1 free_job_table() . . . 164
4.49.2.2 init_job_table() . . . 164
4.49.2.3 job_add() . . . 164
4.49.2.4 job_remove() . . . 165
4.49.2.5 job_touch() . . . 165
4.49.2.6 print_job_status() . . . 165
4.50 src/shell/job/job_table_t.h File Reference . . . 166
4.50.1 Detailed Description . . . 166
4.50.2 Function Documentation . . . 167
4.50.2.1 free_job_table() . . . 167
4.50.2.2 init_job_table() . . . 167
4.50.2.3 job_add() . . . 167
4.50.2.4 job_remove() . . . 168
4.50.2.5 job_touch() . . . 168

4.50.2.6 print_job_status() . . . 168
4.51 src/shell/job/u_job.h File Reference . . . 169
4.51.1 Detailed Description . . . 169
4.51.2 Function Documentation . . . 169
4.51.2.1 u_bg() . . . 169
4.51.2.2 u_fg() . . . 170
4.51.2.3 u_jobs() . . . 170
4.52 src/shell/process/u_process.h File Reference . . . 171
4.52.1 Detailed Description . . . 171
4.52.2 Function Documentation . . . 171
4.52.2.1 u_kill() . . . 171
4.52.2.2 u_logout() . . . 172
4.52.2.3 u_nice() . . . 172
4.52.2.4 u_nice_pid() . . . 172
4.52.2.5 u_ps() . . . 173
4.52.2.6 u_sleep() . . . 173
4.53 src/shell/u_shell.h File Reference . . . 173
4.53.1 Detailed Description . . . 174
4.53.2 Function Documentation . . . 174
4.53.2.1 u_shell() . . . 174
4.54 src/util/log.c File Reference . . . 174
4.54.1 Detailed Description . . . 175
4.54.2 Function Documentation . . . 175
4.54.2.1 log_blocked() . . . 175
4.54.2.2 log_close() . . . 176
4.54.2.3 log_continued() . . . 176
4.54.2.4 log_create() . . . 176
4.54.2.5 log_exited() . . . 177
4.54.2.6 log_get_ticks() . . . 177
4.54.2.7 log_init() . . . 177
4.54.2.8 log_nice() . . . 178
4.54.2.9 log_orphan() . . . 178
4.54.2.10 log_schedule() . . . 178
4.54.2.11 log_signaled() . . . 179
4.54.2.12 log_stopped() . . . 179
4.54.2.13 log_tick() . . . 180
4.54.2.14 log_unblocked() . . . 180
4.54.2.15 log_waited() . . . 180
4.54.2.16 log_zombie() . . . 181
4.55 src/util/log.h File Reference . . . 181
4.55.1 Detailed Description . . . 182
4.55.2 Function Documentation . . . 182

4.55.2.1 log_blocked() . . . 182
4.55.2.2 log_close() . . . 183
4.55.2.3 log_continued() . . . 183
4.55.2.4 log_create() . . . 183
4.55.2.5 log_exited() . . . 184
4.55.2.6 log_get_ticks() . . . 184
4.55.2.7 log_init() . . . 185
4.55.2.8 log_nice() . . . 185
4.55.2.9 log_orphan() . . . 186
4.55.2.10 log_schedule() . . . 186
4.55.2.11 log_signaled() . . . 187
4.55.2.12 log_stopped() . . . 188
4.55.2.13 log_tick() . . . 188
4.55.2.14 log_unblocked() . . . 188
4.55.2.15 log_waited() . . . 189
4.55.2.16 log_zombie() . . . 189
4.56 src/util/panic.c File Reference . . . 190
4.56.1 Detailed Description . . . 190
4.56.2 Function Documentation . . . 190
4.56.2.1 print_and_abort() . . . 190
4.57 src/util/panic.h File Reference . . . 191
4.57.1 Detailed Description . . . 191
4.57.2 Macro Definition Documentation . . . 191
4.57.2.1 panic . . . 191
4.57.3 Function Documentation . . . 192
4.57.3.1 print_and_abort() . . . 192
4.58 src/util/parser.c File Reference . . . 192
4.58.1 Detailed Description . . . 193
4.58.2 Macro Definition Documentation . . . 193
4.58.2.1 JUMP_OUT . . . 193
4.58.3 Function Documentation . . . 193
4.58.3.1 parse_command() . . . 194
4.58.3.2 parseId() . . . 194
4.58.3.3 print_parsed_command() . . . 195
4.58.3.4 print_parser_errcode() . . . 195
4.59 src/util/parser.h File Reference . . . 195
4.59.1 Detailed Description . . . 196
4.59.2 Function Documentation . . . 197
4.59.2.1 parse_command() . . . 197
4.59.2.2 parseId() . . . 198
4.59.2.3 print_parsed_command() . . . 198
4.59.2.4 print_parser_errcode() . . . 199

Index 201

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

fat_metadata_st
FAT metadata structure . . . 7
fat_ut
FAT entry union . . . 8
file_st
File Structure for the FAT filesystem . . . 9
job_st
Represents a single job entry in the job table . . . 10
job_table_st . . . 11
k_fd_entry_t
Entry in the kernel file descriptor table . . . 12
k_open_file_entry_t
Entry in the global open file table . . . 14
parsed_command . . . 15
pcb_t
Process Control Block structure . . . 15
pcb_table_t
Global PCB table structure . . . 19
process_info_t
Process information structure for s_ps . . . 20
spthread_fwd_args_st . . . 21
spthread_meta_st . . . 21
spthread_signal_args_st . . . 22
spthread_st . . . 22
thread_info_st
Information passed to a newly spawned process thread . . . 22
vector_info_st . . . 23

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

src/pennfat.c
Standalone PennFAT filesystem utility . . . 103
src/pennfat.h
Header for the PennFAT standalone filesystem utility . . . 105
src/pennos.c
PennOS kernel main entry point and INIT process implementation . . . 105
src/pennos.h
PennOS kernel initialization and INIT process interface . . . 107
src/io/fstd.h
File system standard definitions and constants . . . 83
src/io/fat/**fat_t.h** . . . **??**
src/io/fat/**file_t.h** . . . **??**
src/io/fat/k_fat.c
Kernel-level FAT filesystem management implementation . . . 25
src/io/fat/k_fat.h
Kernel-level FAT filesystem management interface . . . 31
src/io/fat/s_fat.c
System-level FAT filesystem management implementation . . . 37
src/io/fat/s_fat.h
System-level FAT filesystem management interface . . . 39
src/io/file/h_file.c
Implementation of host file I/O wrapper functions . . . 42
src/io/file/**h_file.h** . . . **??**
src/io/file/k_file.c
Implementation of kernel file I/O operations . . . 44
src/io/file/**k_file.h** . . . **??**
src/io/file/s_file.c
Implementation of system call interface for file I/O operations . . . 50
src/io/file/s_file.h
System call interface for file I/O operations . . . 56
src/io/filetable/filetable.h
Data structures for the kernel file descriptor table . . . 62
src/io/filetable/k_filetable.c
Implementation of the kernel file descriptor table . . . 62
src/io/filetable/k_filetable.h
Kernel file descriptor table management API . . . 73

src/io/path/k_path.c
Implementation of kernel-level directory operations . . . 83
src/io/path/k_path.h
Kernel-level directory and path operations for PennFAT . . . 87
src/io/path/s_path.c
Implementation of system-level directory listing . . . 91
src/io/path/s_path.h
System-level path and directory listing interface . . . 92
src/io/term/k_term.c
Implementation of kernel-level terminal management . . . 93
src/io/term/k_term.h
Kernel-level terminal management interface . . . 95
src/io/term/s_term.c
Implementation of system-level terminal operations . . . 98
src/io/term/s_term.h
System-level terminal I/O and ownership interface . . . 99
src/process/pstd.h
Process standard definitions and constants . . . 133
src/process/lifecycle/k_lifecycle.c
Implementation of kernel-level process lifecycle management . . . 109
src/process/lifecycle/k_lifecycle.h
Kernel-level process lifecycle management . . . 112
src/process/lifecycle/s_lifecycle.c
Implementation of system-level process lifecycle operations . . . 115
src/process/lifecycle/s_lifecycle.h
System-level process lifecycle interface . . . 117
src/process/lifecycle/thread_info_t.h
Thread initialization information structure . . . 119
src/process/pcb/k_pcb.c
Implementation of kernel-level PCB table management . . . 120
src/process/pcb/k_pcb.h
Kernel-level PCB table management interface . . . 124
src/process/pcb/pcb_state_t.h
Process state enumeration for PCB management . . . 128
src/process/pcb/pcb_t.h
Process Control Block (PCB) structure definition . . . 128
src/process/pcb/pcb_table_t.h
PCB table structure for managing all processes . . . 129
src/process/pcb/s_pcb.c
Implementation of system-level PCB operations . . . 130
src/process/pcb/s_pcb.h
System-level PCB information interface . . . 131
src/process/schedule/k_schedule.c
Implementation of the PennOS priority scheduler . . . 134
src/process/schedule/k_schedule.h
Kernel-level scheduler interface for PennOS . . . 139
src/process/schedule/s_schedule.c
Implementation of system-level scheduling functions . . . 145
src/process/schedule/s_schedule.h
System-level scheduling interface for user processes . . . 146
src/process/signal/k_signal.c
Implementation of kernel-level signal handling . . . 148
src/process/signal/k_signal.h
Kernel-level signal handling interface for PennOS . . . 149
src/process/signal/s_signal.c
Implementation of system-level signal functions . . . 151
src/process/signal/s_signal.h
System-level signal interface for user processes . . . 154

src/shell/commands.c
Implementation of the PennOS shell command dispatcher 158
src/shell/commands.h
Command dispatch interface for the PennOS shell 158
src/shell/u_shell.h
PennOS shell entry point declaration 173
src/shell/demo/**u_demo.h** . . . . . **??**
src/shell/file/u_file.h
User-level file manipulation commands for the PennOS shell 160
src/shell/job/job_table.c
Implementation of job table management functions 163
src/shell/job/job_table_t.h
Job table data structures and management functions 166
src/shell/job/u_job.h
User-level job control commands for the PennOS shell 169
src/shell/process/u_process.h
User-level process management commands for the PennOS shell 171
src/util/log.c
Implementation of the PennOS scheduler event logger 174
src/util/log.h
Scheduler event logging interface for PennOS 181
src/util/panic.c
Implementation of kernel panic handling 190
src/util/panic.h
Kernel panic handling for PennOS 191
src/util/parser.c
Implementation of the shell command line parser 192
src/util/parser.h
Shell command line parser for PennOS 195
src/util/**spthread.h** . . . . . **??**
src/util/**vector.h** . . . . . **??**

Chapter 3

Class Documentation

3.1 fat_metadata_st Struct Reference

FAT metadata structure.

```
#include <fat_t.h>
```

Public Attributes

- uint8_t block_size
- uint8_t tot_blocks

3.1.1 Detailed Description

FAT metadata structure.

Contains the metadata information for the FAT filesystem.

3.1.2 Member Data Documentation

3.1.2.1 block_size

```
uint8_t fat_metadata_st::block_size
```

Each block in the FAT has (2^{block_size}) $*$ 256 bytes. block_size should be in the range [0, 4].

3.1.2.2 tot_blocks

```
uint8_t fat_metadata_st::tot_blocks
```

Total number of FAT blocks in the filesystem

The documentation for this struct was generated from the following file:

- src/io/fat/fat_t.h

3.2 fat_ut Union Reference

FAT entry union.

```
#include <fat_t.h>
```

Collaboration diagram for fat_ut:

Public Attributes

- uint16_t next
- fat_metadata_t metadata

3.2.1 Detailed Description

FAT entry union.

A union that can represent a pointer to the next block in a FAT file chain or the metadata for the FAT filesystem.

3.2.2 Member Data Documentation

3.2.2.1 metadata

```
fat_metadata_t fat_ut::metadata
```

For FAT[0], this represents metadata of the fat filesystem.

3.2.2.2 next

```
uint16_t fat_ut::next
```

For FAT[i] for i >= 1, this represents the pointer the the next block in the file chain.

The documentation for this union was generated from the following file:

- src/io/fat/fat_t.h

3.3 file_st Struct Reference

File Structure for the FAT filesystem.

```
#include <file_t.h>
```

Collaboration diagram for file_st:

Public Attributes

- char name [32]
- uint32_t size
- fat_t first_block
- uint8_t type
- uint8_t perm
- time_t mtime
- char __reserved [16]

3.3.1 Detailed Description

File Structure for the FAT filesystem.

Represents a file entry in the FAT filesystem, containing file metadata and a pointer to the first block of the given file.

3.3.2 Member Data Documentation

3.3.2.1 __reserved

```
char file_st::__reserved[16]
```

Reserved space for future use

3.3.2.2 first_block

```
fat_t file_st::first_block
```

First block of the file

3.3.2.3 mtime

```
time_t file_st::mtime
```

Last modification time

3.3.2.4 name

```
char file_st::name[32]
```

File name

3.3.2.5 perm

```
uint8_t file_st::perm
```

Standard permissions of the file

3.3.2.6 size

```
uint32_t file_st::size
```

File size in bytes

3.3.2.7 type

```
uint8_t file_st::type
```

The type of file according to specification

The documentation for this struct was generated from the following file:

- src/io/fat/file_t.h

3.4 job_st Struct Reference

Represents a single job entry in the job table.

```
#include <job_table_t.h>
```

Collaboration diagram for job_st:

Public Attributes

- pid_t pid
- int age
- char * command
- struct parsed_command * cmd_info

3.4.1 Detailed Description

Represents a single job entry in the job table.

3.4.2 Member Data Documentation

3.4.2.1 age

```
int job_st::age
```

Age counter for determining most recent job.

3.4.2.2 cmd_info

```
struct parsed_command* job_st::cmd_info
```

Parsed command information.

3.4.2.3 command

```
char* job_st::command
```

Command string that started this job.

3.4.2.4 pid

```
pid_t job_st::pid
```

Process ID of the job, or -1 if slot is empty.

The documentation for this struct was generated from the following file:

- src/shell/job/job_table_t.h

3.5 job_table_st Struct Reference

Public Member Functions

- vector (job_t) jobs

Public Attributes

- int curr_age
- int latest_job

3.5.1 Member Function Documentation

3.5.1.1 vector()

```
job_table_st::vector (
            job_t  )
```

Vector of job entries (index 0 unused).

3.5.2 Member Data Documentation

3.5.2.1 curr_age

```
int job_table_st::curr_age
```

Monotonically increasing age counter.

3.5.2.2 latest_job

```
int job_table_st::latest_job
```

Job ID of the most recently touched job.

The documentation for this struct was generated from the following file:

- src/shell/job/job_table_t.h

3.6 k_fd_entry_t Struct Reference

Entry in the kernel file descriptor table.

```
#include <filetable.h>
```

Public Attributes

- int open_file_idx
- int hfd
- int refcount
- int mode
- int target_offset
- int virtual_offset
- uint16_t curr_block

3.6.1 Detailed Description

Entry in the kernel file descriptor table.

Represents a single file descriptor in the kernel, tracking the open file it refers to, access mode, current position, and reference count for descriptor sharing (e.g., after dup or fork).

3.6.2 Member Data Documentation

3.6.2.1 curr_block

```
uint16_t k_fd_entry_t::curr_block
```

Current FAT block for file traversal.

3.6.2.2 hfd

```
int k_fd_entry_t::hfd
```

Host file descriptor (-1 for PennFAT files).

3.6.2.3 mode

```
int k_fd_entry_t::mode
```

Access mode (F_READ, F_WRITE, F_APPEND, F_EXEC).

3.6.2.4 open_file_idx

```
int k_fd_entry_t::open_file_idx
```

Index into k_active_files (-1 for host files).

3.6.2.5 refcount

```
int k_fd_entry_t::refcount
```

Number of references to this descriptor.

3.6.2.6 target_offset

```
int k_fd_entry_t::target_offset
```

Requested file offset for next I/O operation.

3.6.2.7 virtual_offset

```
int k_fd_entry_t::virtual_offset
```

Actual current offset in file.

The documentation for this struct was generated from the following file:

- src/io/filetable/filetable.h

3.7 k_open_file_entry_t Struct Reference

Entry in the global open file table.

```
#include <filetable.h>
```

Collaboration diagram for k_open_file_entry_t:

Public Attributes

- file_t file
- int refcount

3.7.1 Detailed Description

Entry in the global open file table.

Represents a file that is currently open in the system. Multiple file descriptors may reference the same open file entry.

3.7.2 Member Data Documentation

3.7.2.1 file

```
file_t k_open_file_entry_t::file
```

The underlying FAT file structure.

3.7.2.2 refcount

```
int k_open_file_entry_t::refcount
```

Number of file descriptors referencing this file.

The documentation for this struct was generated from the following file:

- src/io/filetable/filetable.h

3.8 parsed_command Struct Reference

```
#include <parser.h>
```

Public Attributes

- bool **is_background**
- bool **is_file_append**
- const char * **stdin_file**
- const char * **stdout_file**
- size_t **num_commands**
- char ** **commands** [ ]

3.8.1 Detailed Description

struct parsed_command stored all necessary information needed for penn-shell.

The documentation for this struct was generated from the following file:

- src/util/parser.h

3.9 pcb_t Struct Reference

Process Control Block structure.

```
#include <pcb_t.h>
```

Collaboration diagram for pcb_t:

Public Member Functions

- vector (int) fd_table

  Per-process file descriptor table.

Public Attributes

- spthread_t thread
- pid_t parent
- pid_t alive_child
- pid_t zombie_child
- pid_t prev_sibling
- pid_t nex_sibling
- int sched_priority
- pcb_state_t state
- char process_name [MAX_PROCESS_NAME]
- bool **stopped_as_blocked**

Blocked process state

Fields used when process is in BLOCKED state.

- unsigned int wakeup_time
- bool terminated_by_signal
- pid_t waitpid_target
- int * wstatus

Signal handling flags

Flags controlling signal behavior.

- bool ign_int
- bool ign_stop

3.9.1 Detailed Description

Process Control Block structure.

Contains all state information for a single process in PennOS. Parent-child and sibling relationships are maintained as circular doubly-linked lists for efficient traversal.

3.9.2 Member Function Documentation

3.9.2.1 vector()

```
pcb_t::vector (
            int  )
```

Per-process file descriptor table.

Each entry is an index into the kernel-global fd table. Index 0/1/2 typically map to stdin/stdout/stderr.

3.9.3 Member Data Documentation

3.9.3.1 alive_child

```
pid_t pcb_t::alive_child
```

Head of alive children list (-1 if empty).

3.9.3.2 ign_int

```
bool pcb_t::ign_int
```

Ignore S_SIGINT if true.

3.9.3.3 ign_stop

```
bool pcb_t::ign_stop
```

Ignore S_SIGSTOP if true.

3.9.3.4 nex_sibling

```
pid_t pcb_t::nex_sibling
```

Next sibling in circular list.

3.9.3.5 parent

```
pid_t pcb_t::parent
```

Parent process PID (-1 if none).

3.9.3.6 prev_sibling

```
pid_t pcb_t::prev_sibling
```

Previous sibling in circular list.

3.9.3.7 process_name

```
char pcb_t::process_name[MAX_PROCESS_NAME]
```

Process name for display/logging.

3.9.3.8 sched_priority

```
int pcb_t::sched_priority
```

Scheduling priority (0=high, 1=mid, 2=low).

3.9.3.9 state

```
pcb_state_t pcb_t::state
```

Current process state.

3.9.3.10 terminated_by_signal

```
bool pcb_t::terminated_by_signal
```

True if terminated by signal vs exit.

3.9.3.11 thread

```
spthread_t pcb_t::thread
```

Underlying spthread handle.

3.9.3.12 waitpid_target

```
pid_t pcb_t::waitpid_target
```

Target PID for waitpid (-1 = any).

3.9.3.13 wakeup_time

```
unsigned int pcb_t::wakeup_time
```

Tick count to wake up (for s_sleep).

3.9.3.14 wstatus

```
int* pcb_t::wstatus
```

Pointer to store wait status.

3.9.3.15 zombie_child

```
pid_t pcb_t::zombie_child
```

Head of zombie children list (-1 if empty).

The documentation for this struct was generated from the following file:

- src/process/pcb/pcb_t.h

3.10 pcb_table_t Struct Reference

Global PCB table structure.

```
#include <pcb_table_t.h>
```

Public Member Functions

- vector (pcb_t) processes

Public Attributes

- pid_t avail_process

3.10.1 Detailed Description

Global PCB table structure.

Manages all PCBs in the system. PIDs are indices into the processes vector. The avail_process field tracks the lowest potentially free slot for efficient PID allocation.

3.10.2 Member Function Documentation

3.10.2.1 vector()

```
pcb_table_t::vector (
            pcb_t  )
```

Vector of all PCBs indexed by PID.

3.10.3 Member Data Documentation

3.10.3.1 avail_process

```
pid_t pcb_table_t::avail_process
```

Hint for next available PID slot.

The documentation for this struct was generated from the following file:

- src/process/pcb/pcb_table_t.h

3.11 process_info_t Struct Reference

Process information structure for s_ps.

```
#include <s_pcb.h>
```

Public Attributes

- pid_t pid
- pid_t par_pid
- int priority
- int state
- char name [MAX_PROCESS_NAME_PS]

3.11.1 Detailed Description

Process information structure for s_ps.

Contains a snapshot of process information suitable for display by the ps shell command.

3.11.2 Member Data Documentation

3.11.2.1 name

```
char process_info_t::name[MAX_PROCESS_NAME_PS]
```

Process name.

3.11.2.2 par_pid

```
pid_t process_info_t::par_pid
```

Parent process ID.

3.11.2.3 pid

```
pid_t process_info_t::pid
```

Process ID.

3.11.2.4 priority

```
int process_info_t::priority
```

Scheduling priority (0-2).

3.11.2.5 state

```
int process_info_t::state
```

State: 0=ACTIVE, 1=BLOCKED, 2=STOPPED, 3=ZOMBIE, 4=WAITED.

The documentation for this struct was generated from the following file:

- src/process/pcb/s_pcb.h

3.12 spthread_fwd_args_st Struct Reference

Collaboration diagram for spthread_fwd_args_st:

Public Attributes

- pthread_fn **actual_routine**
- void ∗ **actual_arg**
- bool **setup_done**
- pthread_mutex_t **setup_mutex**
- pthread_cond_t **setup_cond**
- spthread_meta_t ∗ **child_meta**

The documentation for this struct was generated from the following file:

- src/util/spthread.c

3.13 spthread_meta_st Struct Reference

Public Attributes

- sigset_t **suspend_set**
- volatile sig_atomic_t **state**
- pthread_mutex_t **meta_mutex**

The documentation for this struct was generated from the following file:

- src/util/spthread.c

3.14 spthread_signal_args_st Struct Reference

Public Attributes

- const int **signal**
- volatile sig_atomic_t **ack**
- pthread_mutex_t **shutup_mutex**

The documentation for this struct was generated from the following file:

- src/util/spthread.c

3.15 spthread_st Struct Reference

Collaboration diagram for spthread_st:

Public Attributes

- pthread_t **thread**
- spthread_meta_t ∗ **meta**

The documentation for this struct was generated from the following file:

- src/util/spthread.h

3.16 thread_info_st Struct Reference

Information passed to a newly spawned process thread.

```
#include <thread_info_t.h>
```

Public Attributes

- pid_t **pid**
- void ∗(∗ **func**)(void ∗)
- char ∗∗ **argv**
- int **fd0**
- int **fd1**
- int **fd2**

3.16.1 Detailed Description

Information passed to a newly spawned process thread.

This structure is allocated by the parent process and passed to the child's thread entry point (process_spawner). The child is responsible for freeing this structure after extracting the data.

The documentation for this struct was generated from the following file:

- src/process/lifecycle/thread_info_t.h

3.17 vector_info_st Struct Reference

Public Attributes

- size_t **len**
- size_t **capacity**
- destroy_fn **ele_dtor**

The documentation for this struct was generated from the following file:

- src/util/vector.h

Chapter 4

File Documentation

4.1 src/io/fat/k_fat.c File Reference

Kernel-level FAT filesystem management implementation.

```
#include "k_fat.h"
#include <fcntl.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <unistd.h>
#include "io/path/k_path.h"
#include "io/term/s_term.h"
#include "process/schedule/k_schedule.h"
```

Include dependency graph for k_fat.c:

Functions

- size_t k_get_block_size (int block_size_config)

  Get the byte size of FAT blocks based on configuration.
- size_t k_sizeof_fat (fat_t fat_blk)

  Given a valid FAT header, calculates the total size of the FAT region in bytes.
- void k_release_fat_chain (uint16_t start)

  Release an entire chain of FAT blocks.
- size_t k_get_blk_size ()

  Get the block size of the currently mounted filesystem.
- int k_get_data_pos (uint16_t blk, int offset)

  Get the actual memory offset of file data in the filesystem.
- fat_t * k_get_fat (uint16_t blk)

  Get a pointer to a FAT entry.
- bool k_has_mount ()

  Check if a filesystem is currently mounted.
- int k_get_fat_fd ()

  Get the file descriptor of the mounted filesystem.
- void k_mkfs (const char *fs_name, size_t blocks_in_fat, size_t block_size_config)

  Create and initialize a new FAT filesystem.
- int k_mount (const char *fs_name)

Mount an existing FAT filesystem.

- int k_unmount ()

  Unmount the currently mounted filesystem.

- uint16_t k_request_block ()

  Allocate a new block from the filesystem.

- void k_release_block (uint16_t blk)

  Release a FAT block back to the free pool.

- void k_start_block (file_t *file)

  Ensure a file has at least one block allocated.

4.1.1 Detailed Description

Kernel-level FAT filesystem management implementation.

Implements a File Allocation Table (FAT) filesystem with block allocation, mounting/unmounting, and filesystem creation capabilities. The FAT is memory-mapped for efficient access.

4.1.2 Function Documentation

4.1.2.1 k_get_blk_size()

```
size_t k_get_blk_size ( )
```

Get the block size of the currently mounted filesystem.

Returns

Block size in bytes of the current filesystem, or -1 if unmounted.

4.1.2.2 k_get_block_size()

```
size_t k_get_block_size (
            int block_size_config )
```

Get the byte size of FAT blocks based on configuration.

Converts a block size configuration index to the actual byte size. Supported sizes: 256, 512, 1024, 2048, 4096 bytes.

Parameters

block_size_config	Configuration index (0-4)

Returns

Block size in bytes, or -1 if configuration is invalid

4.1.2.3 k_get_data_pos()

```
int k_get_data_pos (
            uint16_t blk,
            int offset )
```

Get the actual memory offset of file data in the filesystem.

Given a FAT block index and memory offset within that block, computes the byte offset relative to the start of the filesystem file where this memory is located.

Parameters

blk	FAT block number
offset	Offset within the block

Returns

Absolute file position in bytes

4.1.2.4 k_get_fat()

```
fat_t* k_get_fat (
            uint16_t blk )
```

Get a pointer to a FAT entry.

Returns a pointer to the FAT entry for the specified block number. All reads and writes to a fat block should be done in the same atomic block as obtaining the pointer.

Precondition

Interrupts are disabled

Parameters

blk	Index of FAT block entry to reference

Returns

Pointer to FAT entry

4.1.2.5 k_get_fat_fd()

```
int k_get_fat_fd ( )
```

Get the file descriptor of the mounted filesystem.

Returns

File descriptor, or -1 if no filesystem is mounted

4.1.2.6 k_has_mount()

```
bool k_has_mount ( )
```

Check if a filesystem is currently mounted.

Returns

true if mounted, false otherwise

4.1.2.7 k_mkfs()

```
void k_mkfs (
            const char * fs_name,
            size_t blocks_in_fat,
            size_t block_size_config )
```

Create and initialize a new FAT filesystem.

Creates a new filesystem file with the specified parameters. The file is initialized with:

- FAT header containing metadata
- Root directory entry (block 1) marked as EOF
- All other blocks marked as empty
- Data region sized according to the number of FAT entries

Parameters

fs_name	Path where the filesystem file will be created
blocks_in_fat	Number of blocks in FAT [1-32]
block_size_config	Block size configuration index [0-4]

4.1.2.8 k_mount()

```
int k_mount (
            const char * fs_name )
```

Mount an existing FAT filesystem.

Opens and prepares the specified filesystem file, making it available for file operations. Only one filesystem can be mounted at a time.

Parameters

fs_name	Path to the filesystem file to mount

Returns

0 on success, -1 on error (e.g., file not found, already mounted)

4.1.2.9 k_release_block()

```
void k_release_block (
            uint16_t blk )
```

Release a FAT block back to the free pool.

Marks the specified block as empty (available for reallocation).

Precondition

Interrupts are disabled

Parameters

blk	Block number to release. Must be >= 2.

4.1.2.10 k_release_fat_chain()

```
void k_release_fat_chain (
            uint16_t start )
```

Release an entire chain of FAT blocks.

Traverses the FAT chain starting from the given block, releasing each block back to the free pool. Stops when reaching EOF (0xFFFF).

Parameters

start	Starting block number of the chain

4.1.2.11 k_request_block()

```
uint16_t k_request_block ( )
```

Allocate a new block from the filesystem.

Allocates the lowest-indexed free FAT block from the filesystem.

Precondition

Interrupts are disabled

Returns

Block number of the allocated block, or 0xFFFF if no free blocks

4.1.2.12 k_sizeof_fat()

```
size_t k_sizeof_fat (
            fat_t fat_blk )
```

Given a valid FAT header, calculates the total size of the FAT region in bytes.

Parameters

fat_blk	A valid FAT header containing metadata.

Returns

Total FAT size in bytes

4.1.2.13 k_start_block()

```
void k_start_block (
            file_t * file )
```

Ensure a file has at least one block allocated.

If the file's first block is 0xFFFF (unallocated), requests a new block from the filesystem and updates the directory entry. This is typically called before writing to a newly created or empty file.

Precondition

Interrupts are disabled

Parameters

file	Pointer to the file structure to check/allocate

4.1.2.14 k_unmount()

```
int k_unmount ( )
```

Unmount the currently mounted filesystem.

Unmaps the FAT from memory and closes the filesystem file descriptor. All changes are synchronized to disk before unmounting.

Returns

0 on success, -1 on error (e.g., no filesystem mounted)

Warning

Does not check if files are currently open/in use

4.2 src/io/fat/k_fat.h File Reference

Kernel-level FAT filesystem management interface.

```
#include <stddef.h>
#include "fat_t.h"
#include "file_t.h"
```

Include dependency graph for k_fat.h: This graph shows which files directly or indirectly include this file:

Functions

- size_t k_get_block_size (int block_size_config)

  Get the byte size of FAT blocks based on configuration.
- size_t k_sizeof_fat (fat_t fat_blk)

  Given a valid FAT header, calculates the total size of the FAT region in bytes.
- void k_release_fat_chain (uint16_t start)

  Release an entire chain of FAT blocks.
- size_t k_get_blk_size ()

  Get the block size of the currently mounted filesystem.
- int k_get_data_pos (uint16_t blk, int offset)

  Get the actual memory offset of file data in the filesystem.
- fat_t * k_get_fat (uint16_t blk)

Get a pointer to a FAT entry.

- bool k_has_mount ()

  Check if a filesystem is currently mounted.
- int k_get_fat_fd ()

  Get the file descriptor of the mounted filesystem.
- void k_mkfs (const char *fs_name, size_t blocks_in_fat, size_t block_size_config)

  Create and initialize a new FAT filesystem.
- int k_mount (const char *fs_name)

  Mount an existing FAT filesystem.
- int k_unmount ()

  Unmount the currently mounted filesystem.
- uint16_t k_request_block ()

  Allocate a new block from the filesystem.
- void k_release_block (uint16_t blk)

  Release a FAT block back to the free pool.
- void k_start_block (file_t *file)

  Ensure a file has at least one block allocated.

4.2.1 Detailed Description

Kernel-level FAT filesystem management interface.

Provides functions for creating, mounting, and managing a FAT (File Allocation Table) filesystem. This includes block allocation, FAT chain management, and filesystem lifecycle operations.

4.2.2 Function Documentation

4.2.2.1 k_get_blk_size()

```
size_t k_get_blk_size ( )
```

Get the block size of the currently mounted filesystem.

Returns

Block size in bytes of the current filesystem, or -1 if unmounted.

4.2.2.2 k_get_block_size()

```
size_t k_get_block_size (
            int block_size_config )
```

Get the byte size of FAT blocks based on configuration.

Converts a block size configuration index to the actual byte size. Supported sizes: 256, 512, 1024, 2048, 4096 bytes.

Parameters

block_size_config	Configuration index (0-4)

Returns

Block size in bytes, or -1 if configuration is invalid

4.2.2.3 k_get_data_pos()

```
int k_get_data_pos (
            uint16_t blk,
            int offset )
```

Get the actual memory offset of file data in the filesystem.

Given a FAT block index and memory offset within that block, computes the byte offset relative to the start of the filesystem file where this memory is located.

Parameters

blk	FAT block number
offset	Offset within the block

Returns

Absolute file position in bytes

4.2.2.4 k_get_fat()

```
fat_t* k_get_fat (
            uint16_t blk )
```

Get a pointer to a FAT entry.

Returns a pointer to the FAT entry for the specified block number. All reads and writes to a fat block should be done in the same atomic block as obtaining the pointer.

Precondition

Interrupts are disabled

Parameters

blk	Index of FAT block entry to reference

Returns

Pointer to FAT entry

4.2.2.5 k_get_fat_fd()

```
int k_get_fat_fd ( )
```

Get the file descriptor of the mounted filesystem.

Returns

File descriptor, or -1 if no filesystem is mounted

4.2.2.6 k_has_mount()

```
bool k_has_mount ( )
```

Check if a filesystem is currently mounted.

Returns

true if mounted, false otherwise

4.2.2.7 k_mkfs()

```
void k_mkfs (
            const char * fs_name,
            size_t blocks_in_fat,
            size_t block_size_config )
```

Create and initialize a new FAT filesystem.

Creates a new filesystem file with the specified parameters. The file is initialized with:

- FAT header containing metadata
- Root directory entry (block 1) marked as EOF
- All other blocks marked as empty
- Data region sized according to the number of FAT entries

Parameters

fs_name	Path where the filesystem file will be created
blocks_in_fat	Number of blocks in FAT [1-32]
block_size_config	Block size configuration index [0-4]

4.2.2.8 k_mount()

```
int k_mount (
            const char * fs_name )
```

Mount an existing FAT filesystem.

Opens and prepares the specified filesystem file, making it available for file operations. Only one filesystem can be mounted at a time.

Parameters

fs_name	Path to the filesystem file to mount

Returns

0 on success, -1 on error (e.g., file not found, already mounted)

4.2.2.9 k_release_block()

```
void k_release_block (
            uint16_t blk )
```

Release a FAT block back to the free pool.

Marks the specified block as empty (available for reallocation).

Precondition

Interrupts are disabled

Parameters

blk	Block number to release. Must be >= 2.

4.2.2.10 k_release_fat_chain()

```
void k_release_fat_chain (
            uint16_t start )
```

Release an entire chain of FAT blocks.

Traverses the FAT chain starting from the given block, releasing each block back to the free pool. Stops when reaching EOF (0xFFFF).

Parameters

start	Starting block number of the chain

4.2.2.11 k_request_block()

```
uint16_t k_request_block ( )
```

Allocate a new block from the filesystem.

Allocates the lowest-indexed free FAT block from the filesystem.

Precondition

Interrupts are disabled

Returns

Block number of the allocated block, or 0xFFFF if no free blocks

4.2.2.12 k_sizeof_fat()

```
size_t k_sizeof_fat (
            fat_t fat_blk )
```

Given a valid FAT header, calculates the total size of the FAT region in bytes.

Parameters

fat_blk	A valid FAT header containing metadata.

Returns

Total FAT size in bytes

4.2.2.13 k_start_block()

```
void k_start_block (
            file_t * file )
```

Ensure a file has at least one block allocated.

If the file's first block is 0xFFFF (unallocated), requests a new block from the filesystem and updates the directory entry. This is typically called before writing to a newly created or empty file.

Precondition

Interrupts are disabled

Parameters

file	Pointer to the file structure to check/allocate

4.2.2.14 k_unmount()

```
int k_unmount ( )
```

Unmount the currently mounted filesystem.

Unmaps the FAT from memory and closes the filesystem file descriptor. All changes are synchronized to disk before unmounting.

Returns

0 on success, -1 on error (e.g., no filesystem mounted)

Warning

Does not check if files are currently open/in use

4.3 src/io/fat/s_fat.c File Reference

System-level FAT filesystem management implementation.

```
#include "s_fat.h"
#include "k_fat.h"
```

Include dependency graph for s_fat.c:

Functions

- void s_mkfs (const char *fs_name, size_t blocks_in_fat, size_t block_size_config)

  Create and initialize a new FAT filesystem.
- int s_mount (const char *filename)

  Mount an existing FAT filesystem.
- void s_unmount ()

  Unmount the currently mounted filesystem.
- bool s_has_mount ()

  Check if a filesystem is currently mounted.

4.3.1 Detailed Description

System-level FAT filesystem management implementation.

Implements system-level wrappers for FAT filesystem operations. These functions provide a clean interface for user-space applications while delegating actual work to the kernel-level implementations.

4.3.2 Function Documentation

4.3.2.1 s_has_mount()

```
bool s_has_mount ( )
```

Check if a filesystem is currently mounted.

Returns

true if mounted, false otherwise

4.3.2.2 s_mkfs()

```
void s_mkfs (
            const char * fs_name,
            size_t blocks_in_fat,
            size_t block_size_config )
```

Create and initialize a new FAT filesystem.

Creates a new filesystem file with the specified parameters. The file is initialized with:

- FAT header containing metadata
- Root directory entry (block 1) marked as EOF
- All other blocks marked as empty
- Data region sized according to the number of FAT entries

Parameters

fs_name	Path where the filesystem file will be created
blocks_in_fat	Number of blocks in FAT [1-32]
block_size_config	Block size configuration index [0-4]

4.3.2.3 s_mount()

```
int s_mount (
            const char * filename )
```

Mount an existing FAT filesystem.

Opens and prepares the specified filesystem file, making it available for file operations. Only one filesystem can be mounted at a time.

Parameters

fs_name	Path to the filesystem file to mount

Returns

0 on success, -1 on error (e.g., file not found, already mounted)

4.3.2.4 s_unmount()

```
void s_unmount ( )
```

Unmount the currently mounted filesystem.

Unmaps the FAT from memory and closes the filesystem file descriptor. All changes are synchronized to disk before unmounting.

Returns

0 on success, -1 on error (e.g., no filesystem mounted)

Warning

Does not check if files are currently open/in use

4.4 src/io/fat/s_fat.h File Reference

System-level FAT filesystem management interface.

`#include "io/fstd.h"`
Include dependency graph for s_fat.h: This graph shows which files directly or indirectly include this file:

Functions

- void s_mkfs (const char *fs_name, size_t blocks_in_fat, size_t block_size_config)

  Create and initialize a new FAT filesystem.
- int s_mount (const char *filename)

  Mount an existing FAT filesystem.
- void s_unmount ()

  Unmount the currently mounted filesystem.
- bool s_has_mount ()

  Check if a filesystem is currently mounted.

4.4.1 Detailed Description

System-level FAT filesystem management interface.

Provides user/system-level wrappers for FAT filesystem operations. These functions serve as the interface between user applications and the kernel-level FAT implementation.

4.4.2 Function Documentation

4.4.2.1 s_has_mount()

```
bool s_has_mount ( )
```

Check if a filesystem is currently mounted.

Returns

true if mounted, false otherwise

4.4.2.2 s_mkfs()

```
void s_mkfs (
            const char * fs_name,
            size_t blocks_in_fat,
            size_t block_size_config )
```

Create and initialize a new FAT filesystem.

Creates a new filesystem file with the specified parameters. The file is initialized with:

- FAT header containing metadata
- Root directory entry (block 1) marked as EOF
- All other blocks marked as empty
- Data region sized according to the number of FAT entries

Parameters

fs_name	Path where the filesystem file will be created
blocks_in_fat	Number of blocks in FAT [1-32]
block_size_config	Block size configuration index [0-4]

4.4.2.3 s_mount()

```
int s_mount (
            const char * filename )
```

Mount an existing FAT filesystem.

Opens and prepares the specified filesystem file, making it available for file operations. Only one filesystem can be mounted at a time.

Parameters

fs_name	Path to the filesystem file to mount

Returns

0 on success, -1 on error (e.g., file not found, already mounted)

4.4.2.4 s_unmount()

```
void s_unmount ( )
```

Unmount the currently mounted filesystem.

Unmaps the FAT from memory and closes the filesystem file descriptor. All changes are synchronized to disk before unmounting.

Returns

0 on success, -1 on error (e.g., no filesystem mounted)

Warning

Does not check if files are currently open/in use

4.5 src/io/file/h_file.c File Reference

Implementation of host file I/O wrapper functions.

```
#include "h_file.h"
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
```

Include dependency graph for h_file.c:

Functions

- int h_open (const char *fname, int mode)

  Opens a file with the specified mode on the host filesystem.
- int h_read (int fd, void *buf, size_t nbytes)

  Reads data from a host file descriptor.
- int h_write (int fd, const void *buf, size_t nbytes)

  Writes data to a host file descriptor.
- int h_close (int fd)

  Closes a host file descriptor.
- void h_lseek (int fd, int offset, int whence)

  Sets the file offset for the given host file descriptor.

4.5.1 Detailed Description

Implementation of host file I/O wrapper functions.

Wraps standard Unix file operations with custom mode flags defined in fstd.h (F_READ, F_WRITE, F_APPEND, F_EXEC, etc.).

4.5.2 Function Documentation

4.5.2.1 h_close()

```
int h_close (
            int fd )
```

Closes a host file descriptor.

Parameters

fd	File descriptor to close

Returns

0 on success, -1 on error

4.5.2.2 h_lseek()

```
void h_lseek (
            int fd,
            int offset,
            int whence )
```

Sets the file offset for the given host file descriptor.

Parameters

fd	Host file descriptor
offset	Offset value
whence	Reference position (F_SEEK_SET, F_SEEK_CUR, or F_SEEK_END)

4.5.2.3 h_open()

```
int h_open (
            const char * fname,
            int mode )
```

Opens a file with the specified mode on the host filesystem.

Translates custom mode flags to Unix open flags:

- F_READ/F_EXEC: Opens read-only
- F_WRITE: Creates/truncates file for writing
- F_APPEND: Creates/appends to file

Parameters

fname	Path to the file to open
mode	File access mode (F_READ, F_WRITE, F_APPEND, or F_EXEC)

Returns

File descriptor on success, -1 on error

4.5.2.4 h_read()

```
int h_read (
            int fd,
            void * buf,
            size_t nbytes )
```

Reads data from a host file descriptor.

Parameters

fd	Host file descriptor to read from
buf	Buffer to store read data
nbytes	Number of bytes to read

Returns

Number of bytes read on success, -1 on error

See also

the standard read() function for additional details

4.5.2.5 h_write()

```
int h_write (
             int fd,
             const void * buf,
             size_t nbytes )
```

Writes data to a host file descriptor.

Automatically flushes if writing to stdout, stderr to ensure immediate output visibility.

Parameters

fd	Host file descriptor to write to
buf	Buffer containing data to write
nbytes	Number of bytes to write

Returns

Number of bytes written on success, -1 on error

4.6 src/io/file/k_file.c File Reference

Implementation of kernel file I/O operations.

```
#include "k_file.h"
#include <string.h>
#include <unistd.h>
#include "io/fat/k_fat.h"
#include "io/file/h_file.h"
#include "io/filetable/k_filetable.h"
#include "io/path/k_path.h"
#include "process/schedule/k_schedule.h"
```

Include dependency graph for k_file.c:

Functions

- int k_open (const char ∗fname, int mode)

  Opens a file in the FAT filesystem in the provided mode.
- int k_ext_open (const char ∗fname, int mode)

  Opens an external host file.
- int k_read (int fd, void ∗buf, size_t nbytes)

  Reads data from a kernel file descriptor for a local fs file.
- int k_ext_read (int fd, void ∗buf, size_t nbytes)

  Reads data from a host file descriptor.
- int k_write (int fd, const void ∗buf, size_t nbytes)

  Writes data to a kernel file descriptor.
- int k_close (int fd)

  Close a kernel file descriptor.
- int k_unlink (const char ∗fname)

  Deletes a file from the filesystem.
- void k_lseek (int fd, int offset, int whence)

  Sets the file offset for the given file descriptor.
- int k_mv (const char ∗source, const char ∗dest)

  Renames a file in the filesystem.

4.6.1 Detailed Description

Implementation of kernel file I/O operations.

Implements file operations for a custom FAT-based filesystem including file creation, deletion, reading, writing, and metadata management. Handles both internal filesystem files and external host files.

4.6.2 Function Documentation

4.6.2.1 k_close()

```
int k_close (
            int fd )
```

Close a kernel file descriptor.

Precondition

Interrupts are disabled

Parameters

fd	The kernel file descriptor to close.

Returns

0 on success.

4.6.2.2 k_ext_open()

```
int k_ext_open (
            const char * fname,
            int mode )
```

Opens an external host file.

Opens a file from the host filesystem and registers it in the kernel file descriptor table for unified access.

Precondition

Interrupts are disabled

Parameters

fname	Path to the external file
mode	File access mode (F_READ, F_WRITE, F_APPEND, or F_EXEC)

Returns

Kernel file descriptor on success, -1 on error

See also

h_open for how modes are handled

4.6.2.3 k_ext_read()

```
int k_ext_read (
            int fd,
            void * buf,
            size_t nbytes )
```

Reads data from a host file descriptor.

Parameters

fd	Host file descriptor to read from
buf	Buffer to store read data
nbytes	Number of bytes to read

Returns

Number of bytes read on success, -1 on error

See also

the standard read() function for additional details

4.6.2.4 k_lseek()

```
void k_lseek (
            int fd,
            int offset,
            int whence )
```

Sets the file offset for the given file descriptor.

Sets the file offset for the given file descriptor. The whence operator acts the same as whence for standard lseek on the subset of options F_SEEK_SET, F_SEEK_CUR, and F_SEEK_END.

Precondition

Interrupts are disabled

Parameters

fd	Kernel file descriptor
offset	Offset value
whence	Reference position (F_SEEK_SET, F_SEEK_CUR, or F_SEEK_END)

4.6.2.5 k_mv()

```
int k_mv (
            const char * source,
            const char * dest )
```

Renames a file in the filesystem.

Renames a file to the new name, replacing the destination file if it exists. Will error if the source file doesn't exist, if either file is open, or if the source and destination are the same.

Precondition

Interrupts are disabled

Parameters

source	Current file name
dest	New file name

Returns

0 on success, -1 on error (file open, not found, or cannot add)

4.6.2.6 k_open()

```
int k_open (
            const char * fname,
            int mode )
```

Opens a file in the FAT filesystem in the provided mode.

Opens a file using the provided mode.

The modes work as follows:

- F_READ / F_EXEC = Readonly access to file, errors if doesn't exist.
- F_WRITE = Write access to file, truncates the file if exists.
- F_APPEND = Write access to file, subsequent writes extend the file.

Will error if the file exists and has the wrong permission. Only one open can have write capability to a file at a time, subsequent opens will error.

Precondition

Interrupts are disabled

Parameters

fname	Path to the file in the filesystem
mode	File access mode (F_READ, F_WRITE, F_APPEND, or F_EXEC)

Returns

Kernel file descriptor on success, -1 on error

Note

Emits perror messages upon error with the relevant information

4.6.2.7 k_read()

```
int k_read (
            int fd,
            void * buf,
            size_t nbytes )
```

Reads data from a kernel file descriptor for a local fs file.

Precondition

fd references a local file

Interrupts are disabled

Parameters

fd	File descriptor to read from
buf	Buffer to store read data
nbytes	Number of bytes to read

Returns

Number of bytes read on success, -1 on error

See also

standard read() function for details

k_ext_read for reading from host file descriptors

4.6.2.8 k_unlink()

```
int k_unlink (
            const char * fname )
```

Deletes a file from the filesystem.

Deletes a file from the filesystem. Fails if the file is currently open by any process.

Precondition

Interrupts are disabled

Parameters

fname	Path to the file to delete

Returns

0 on success, -1 on error (file open or not found)

4.6.2.9 k_write()

```
int k_write (
            int fd,
            const void * buf,
            size_t nbytes )
```

Writes data to a kernel file descriptor.

Precondition

Interrupts are disabled

Parameters

fd	Kernel file descriptor
buf	Buffer containing data to write
nbytes	Number of bytes to write

Returns

Number of bytes written on success, -1 on error

4.7 src/io/file/s_file.c File Reference

Implementation of system call interface for file I/O operations.

```
#include "s_file.h"
#include "io/filetable/k_filetable.h"
#include "io/path/k_path.h"
#include "io/term/k_term.h"
#include "k_file.h"
#include "process/schedule/k_schedule.h"
#include "io/term/s_term.h"
#include "process/schedule/s_schedule.h"
#include "process/signal/s_signal.h"
```

Include dependency graph for s_file.c:

Functions

- int s_open (const char ∗fname, int mode)

  Opens a file in the fat filesystem.
- int s_ext_open (const char ∗fname, int mode)

  Opens an external host file.
- int s_read (int fd, void ∗buf, size_t nbytes)

  Reads data from a file.
- int s_write (int fd, const void ∗buf, size_t nbytes)

  Writes data to a file.
- int s_close (int fd)

  Closes a file descriptor (system call interface)
- int s_unlink (const char ∗fname)

  Deletes a file from the filesystem.
- void s_lseek (int fd, int offset, int whence)

  Sets the file offset.
- int s_mv (const char ∗source, const char ∗dest)

  Renames a file in the filesystem.
- int s_chmod (const char ∗filename, unsigned char flag, int add)

  Changes the file permissions of the provided file.
- int s_get_open_mode (int fd)

  Gets the open mode of a file descriptor s.
- bool s_same_source (int fd1, int fd2)

  Checks if two file descriptors refer to the same file.

4.7.1 Detailed Description

Implementation of system call interface for file I/O operations.

Implements user-level system call wrappers that translate process-local file descriptors to kernel file descriptors and manage interrupt states. Handles terminal control for stdin reads and delegates to kernel file operations (k_file.h) for actual filesystem access.

4.7.2 Function Documentation

4.7.2.1 s_chmod()

```
int s_chmod (
            const char * filename,
            unsigned char flag,
            int add )
```

Changes the file permissions of the provided file.

Changes the file permissions of the provided file (1=execute, 2=write, 4=read).

Parameters

filename	Path to the file
flag	Permission bit to modify (1=execute, 2=write, 4=read)
add	1 to add permission, 0 to remove permission

Returns

0 on success, -1 on error (file not found)

4.7.2.2 s_close()

```
int s_close (
            int fd )
```

Closes a file descriptor (system call interface)

If called from a process context, releases the process-local file descriptor which decrements the reference count on the kernel file descriptor. If called without a process context (pid == -1), directly closes the kernel file descriptor.

Parameters

fd	Process-local file descriptor to close

Returns

0 on success, -1 on error

4.7.2.3 s_ext_open()

```
int s_ext_open (
            const char * fname,
            int mode )
```

Opens an external host file.

Parameters

fname	Path to the external file
mode	File access mode (F_READ, F_WRITE, F_APPEND, or F_EXEC)

Returns

Process-local file descriptor on success, -1 on error

See also

h_open for documentation about mode behavior

4.7.2.4 s_get_open_mode()

```
int s_get_open_mode (
            int fd )
```

Gets the open mode of a file descriptor s.

Parameters

fd	Process-local file descriptor

Returns

Open mode on success, -1 on error

4.7.2.5 s_lseek()

```
void s_lseek (
            int fd,
            int offset,
            int whence )
```

Sets the file offset.

Parameters

fd	Process-local file descriptor
offset	Offset value
whence	Reference position (F_SEEK_SET, F_SEEK_CUR, or F_SEEK_END)

See also

k_lseek for more details

4.7.2.6 s_mv()

```
int s_mv (
            const char * source,
            const char * dest )
```

Renames a file in the filesystem.

Parameters

source	Current file name
dest	New file name

Returns

0 on success, -1 on error

See also

k_mv for more details

4.7.2.7 s_open()

```
int s_open (
            const char * fname,
            int mode )
```

Opens a file in the fat filesystem.

Parameters

fname	Path to the file in the filesystem
mode	File access mode (F_READ, F_WRITE, F_APPEND, or F_EXEC)

Returns

Process-local file descriptor on success, -1 on error

See also

k_open for documentation about mode behavior

4.7.2.8 s_read()

```
int s_read (
            int fd,
            void * buf,
            size_t nbytes )
```

Reads data from a file.

Parameters

fd	Process-local file descriptor
buf	Buffer to store read data
nbytes	Number of bytes to read

Returns

Number of bytes read on success, -1 on error, 0 on EOF

4.7.2.9 s_same_source()

```
bool s_same_source (
            int fd1,
            int fd2 )
```

Checks if two file descriptors refer to the same file.

Parameters

fd1	First process-local file descriptor
fd2	Second process-local file descriptor

Returns

true if both refer to the same file, false otherwise

4.7.2.10 s_unlink()

```
int s_unlink (
            const char * fname )
```

Deletes a file from the filesystem.

Parameters

fname	Path to the file to delete

Returns

0 on success, -1 on error

See also

k_unlink for more details

4.7.2.11 s_write()

```
int s_write (
            int fd,
            const void * buf,
            size_t nbytes )
```

Writes data to a file.

Parameters

fd	Process-local file descriptor
buf	Buffer containing data to write
nbytes	Number of bytes to write

Returns

Number of bytes written on success, -1 on error

4.8 src/io/file/s_file.h File Reference

System call interface for file I/O operations.

`#include "io/fstd.h"`
Include dependency graph for s_file.h: This graph shows which files directly or indirectly include this file:

Functions

- int s_open (const char ∗fname, int mode)

  Opens a file in the fat filesystem.
- int s_ext_open (const char ∗fname, int mode)

  Opens an external host file.
- int s_read (int fd, void ∗buf, size_t nbytes)

  Reads data from a file.
- int s_write (int fd, const void ∗buf, size_t nbytes)

  Writes data to a file.
- int s_close (int fd)

  Closes a file descriptor (system call interface)
- int s_unlink (const char ∗fname)

  Deletes a file from the filesystem.
- void s_lseek (int fd, int offset, int whence)

  Sets the file offset.

- int s_mv (const char *source, const char *dest)

  Renames a file in the filesystem.
- int s_chmod (const char *filename, unsigned char flag, int add)

  Changes the file permissions of the provided file.
- int s_get_open_mode (int fd)

  Gets the open mode of a file descriptor s.
- bool s_same_source (int fd1, int fd2)

  Checks if two file descriptors refer to the same file.

4.8.1 Detailed Description

System call interface for file I/O operations.

Provides user-level system call wrappers for file operations that interface with the kernel file layer. These functions perform process-specific file descriptor management and delegate to kernel functions (k_file.h) for actual filesystem operations.

4.8.2 Function Documentation

4.8.2.1 s_chmod()

```
int s_chmod (
            const char * filename,
            unsigned char flag,
            int add )
```

Changes the file permissions of the provided file.

Changes the file permissions of the provided file (1=execute, 2=write, 4=read).

Parameters

filename	Path to the file
flag	Permission bit to modify (1=execute, 2=write, 4=read)
add	1 to add permission, 0 to remove permission

Returns

0 on success, -1 on error (file not found)

4.8.2.2 s_close()

```
int s_close (
            int fd )
```

Closes a file descriptor (system call interface)

If called from a process context, releases the process-local file descriptor which decrements the reference count on the kernel file descriptor. If called without a process context (pid == -1), directly closes the kernel file descriptor.

Parameters

fd	Process-local file descriptor to close

Returns

0 on success, -1 on error

4.8.2.3 s_ext_open()

```
int s_ext_open (
            const char * fname,
            int mode )
```

Opens an external host file.

Parameters

fname	Path to the external file
mode	File access mode (F_READ, F_WRITE, F_APPEND, or F_EXEC)

Returns

Process-local file descriptor on success, -1 on error

See also

h_open for documentation about mode behavior

4.8.2.4 s_get_open_mode()

```
int s_get_open_mode (
            int fd )
```

Gets the open mode of a file descriptor s.

Parameters

fd	Process-local file descriptor

Returns

Open mode on success, -1 on error

4.8.2.5 s_lseek()

```
void s_lseek (
            int fd,
            int offset,
            int whence )
```

Sets the file offset.

Parameters

fd	Process-local file descriptor
offset	Offset value
whence	Reference position (F_SEEK_SET, F_SEEK_CUR, or F_SEEK_END)

See also

k_lseek for more details

4.8.2.6 s_mv()

```
int s_mv (
            const char * source,
            const char * dest )
```

Renames a file in the filesystem.

Parameters

source	Current file name
dest	New file name

Returns

0 on success, -1 on error

See also

k_mv for more details

4.8.2.7 s_open()

```
int s_open (
            const char * fname,
            int mode )
```

Opens a file in the fat filesystem.

Parameters

fname	Path to the file in the filesystem
mode	File access mode (F_READ, F_WRITE, F_APPEND, or F_EXEC)

Returns

Process-local file descriptor on success, -1 on error

See also

k_open for documentation about mode behavior

4.8.2.8 s_read()

```
int s_read (
            int fd,
            void * buf,
            size_t nbytes )
```

Reads data from a file.

Parameters

fd	Process-local file descriptor
buf	Buffer to store read data
nbytes	Number of bytes to read

Returns

Number of bytes read on success, -1 on error, 0 on EOF

4.8.2.9 s_same_source()

```
bool s_same_source (
            int fd1,
            int fd2 )
```

Checks if two file descriptors refer to the same file.

Parameters

fd1	First process-local file descriptor
fd2	Second process-local file descriptor

Returns

true if both refer to the same file, false otherwise

4.8.2.10 s_unlink()

```
int s_unlink (
            const char * fname )
```

Deletes a file from the filesystem.

Parameters

fname	Path to the file to delete

Returns

0 on success, -1 on error

See also

k_unlink for more details

4.8.2.11 s_write()

```
int s_write (
            int fd,
            const void * buf,
            size_t nbytes )
```

Writes data to a file.

Parameters

fd	Process-local file descriptor
buf	Buffer containing data to write
nbytes	Number of bytes to write

Returns

Number of bytes written on success, -1 on error

4.9 src/io/filetable/filetable.h File Reference

Data structures for the kernel file descriptor table.

```
#include "io/fat/file_t.h"
#include "io/fstd.h"
```

Include dependency graph for filetable.h: This graph shows which files directly or indirectly include this file:

Classes

- struct k_open_file_entry_t

  Entry in the global open file table.
- struct k_fd_entry_t

  Entry in the kernel file descriptor table.

4.9.1 Detailed Description

Data structures for the kernel file descriptor table.

This header defines the core data structures used by the kernel to manage open files and file descriptors in the PennOS operating system. It provides two key structures: one for tracking globally open files and another for individual file descriptor entries.

4.10 src/io/filetable/k_filetable.c File Reference

Implementation of the kernel file descriptor table.

```
#include "k_filetable.h"
#include "filetable.h"
#include "io/file/h_file.h"
#include "io/fat/k_fat.h"
#include "process/pcb/k_pcb.h"
#include "process/schedule/k_schedule.h"
#include "io/term/s_term.h"
#include <string.h>
#include "util/vector.h"
```

Include dependency graph for k_filetable.c:

Macros

- #define **ASSERT_HAS_FD**(fd, func_name) (void)0

Functions

- void k_fd_clean ()

  Free the kernel file descriptor table.
- int file_alloc (file_t file)

  Allocate an entry in the open file table.
- int fd_alloc (k_fd_entry_t entry)

  Allocate an entry in the file descriptor table.
- int k_fd_alloc (file_t file, int mode)

  Allocate a kernel file descriptor for a file.
- int k_fd_alloc_h (int hfd, int mode)

  Allocate a kernel file descriptor for a host fd.
- bool k_fd_has_entry (int fd)

  Check if a file descriptor entry exists and is valid.
- int k_fd_get_hfd (int fd)

  Get the host file descriptor if this is a host file.
- int k_fd_dup (int fd)

  Duplicate a kernel file descriptor.
- int k_fd_close (int fd)

  Close a kernel file descriptor.
- int apply_offset (int fd)

  Apply pending offset changes to a file descriptor.
- int k_fd_walk (int fd, size_t *target_size, int *pos, size_t *size)

  Walk through file blocks for I/O operations.
- uint32_t k_fd_get_offset (int fd)

  Get the current file offset.
- void k_fd_set_offset (int fd, int off)

  Set the file offset.
- int k_fd_get_size (int fd)

  Get the size of the file.
- int k_fd_mode (int fd)

  Get the access mode of a file descriptor.
- file_t * k_fd_file_ref (int fd)

  Get a reference to the file referenced by a fd.
- int k_lfd_put (int pid, int hfd)

  Add a kernel file descriptor to a process's local table.
- int k_lfd_release (int pid, int fd)

  Release a local file descriptor from a process.
- int k_lfd_get (int pid, int fd)

  Get the kernel file descriptor from a local descriptor.
- int k_fd_is_open (const char *fname)

  Check if a file is currently open.
- int k_fd_is_write_locked (const char *fname)

  Check if a file is open with write or append mode.

4.10.1 Detailed Description

Implementation of the kernel file descriptor table.

This file implements the kernel-level file descriptor management for PennOS. It maintains two core data structures:

- **k_active_files**: A table of open files shared across all descriptors.
- **k_fd_table**: A table of file descriptors with individual state.

The implementation supports:

- Reference-counted file descriptors for sharing via dup/fork.
- Both PennFAT files and host OS files.
- Lazy block allocation for writes in the FAT filesystem.
- Process-local to kernel-global descriptor mapping.

Note

All functions that modify state assert they are not called from interrupt context using ASSERT_NO_↩ INTERRUPT.

4.10.2 Function Documentation

4.10.2.1 apply_offset()

```
int apply_offset (
            int fd )
```

Apply pending offset changes to a file descriptor.

Parameters

fd	The file descriptor to update.

Returns

0 on success, -1 on error (EOF in read mode or allocation failure).

4.10.2.2 fd_alloc()

```
int fd_alloc (
            k_fd_entry_t entry )
```

Allocate an entry in the file descriptor table.

Allocate an entry in the open file table. Will always allocate the lowest available file descriptor

Parameters

entry	The file descriptor entry to insert.

Returns

Index into k_fd_table (the new fd number).

4.10.2.3 file_alloc()

```
int file_alloc (
            file_t file )
```

Allocate an entry in the open file table.

Allocate an entry in the open file table. If the file is already open, returns the existing index.

Parameters

file	The file to add to the open file table.

Returns

Index into k_active_files.

4.10.2.4 k_fd_alloc()

```
int k_fd_alloc (
            file_t file,
            int mode )
```

Allocate a kernel file descriptor for a file.

Creates a new file descriptor entry for the given file with the specified access mode.

Precondition

Interrupts are disabled

Parameters

file	The file to open.
mode	Access mode (F_READ, F_WRITE, F_APPEND, or F_EXEC).

Returns

The allocated kernel file descriptor index.

4.10.2.5 k_fd_alloc_h()

```
int k_fd_alloc_h (
            int hfd,
            int mode )
```

Allocate a kernel file descriptor for a host fd.

Creates a new file descriptor entry that wraps a host fd.

Precondition

Interrupts are disabled

Parameters

hfd	The host OS file descriptor.
mode	Access mode (F_READ, F_WRITE, etc.).

Returns

The allocated kernel file descriptor index.

4.10.2.6 k_fd_close()

```
int k_fd_close (
            int fd )
```

Close a kernel file descriptor.

Precondition

Interrupts are disabled

Parameters

fd	The kernel file descriptor to close.

Returns

0 on success.

4.10.2.7 k_fd_dup()

```
int k_fd_dup (
            int fd )
```

Duplicate a kernel file descriptor.

Precondition

Interrupts are disabled

Parameters

fd	The kernel file descriptor to duplicate.

Returns

The same file descriptor on success.

4.10.2.8 k_fd_file_ref()

```
file_t* k_fd_file_ref (
            int fd )
```

Get a reference to the file referenced by a fd.

Precondition

Interrupts are disabled

Parameters

fd	The kernel file descriptor.

Returns

Pointer to the file_t structure.

4.10.2.9 k_fd_get_hfd()

```
int k_fd_get_hfd (
            int fd )
```

Get the host file descriptor if this is a host file.

Precondition

Interrupts are disabled

Parameters

fd	The kernel file descriptor.

Returns

The host file descriptor if it is a host file, else -1

4.10.2.10 k_fd_get_offset()

```
uint32_t k_fd_get_offset (
            int fd )
```

Get the current file offset.

Precondition

Interrupts are disabled

Parameters

fd	The kernel file descriptor.

Returns

The current target offset in bytes.

4.10.2.11 k_fd_get_size()

```
int k_fd_get_size (
            int fd )
```

Get the size of the file.

Precondition

Interrupts are disabled

Parameters

fd	The kernel file descriptor.

Returns

The file size in bytes.

4.10.2.12 k_fd_has_entry()

```
bool k_fd_has_entry (
            int fd )
```

Check if a file descriptor entry exists and is valid.

Parameters

fd	The kernel file descriptor.

Returns

true if the descriptor is valid and has a positive refcount.

4.10.2.13 k_fd_is_open()

```
int k_fd_is_open (
            const char * fname )
```

Check if a file is currently open.

Precondition

Interrupts are disabled

Parameters

fname	The filename to check.

Returns

1 if the file is open, 0 otherwise.

4.10.2.14 k_fd_is_write_locked()

```
int k_fd_is_write_locked (
            const char * fname )
```

Check if a file is open with write or append mode.

Used to enforce write locking - only one writer allowed at a time.

Precondition

Interrupts are disabled

Parameters

fname	The filename to check.

Returns

1 if the file is write-locked, 0 otherwise.

4.10.2.15 k_fd_mode()

```
int k_fd_mode (
            int fd )
```

Get the access mode of a file descriptor.

Parameters

fd	The kernel file descriptor.

Returns

The access mode (F_READ, F_WRITE, F_APPEND, or F_EXEC).

4.10.2.16 k_fd_set_offset()

```
void k_fd_set_offset (
            int fd,
            int off )
```

Set the file offset.

Precondition

Interrupts are disabled

Parameters

fd	The kernel file descriptor.
off	The new offset in bytes.

4.10.2.17 k_fd_walk()

```
int k_fd_walk (
            int fd,
            size_t * target_size,
            int * pos,
            size_t * size )
```

Walk through file blocks for I/O operations.

Advances through the file's FAT chain, returning the position and size of the next chunk to read/write.

Precondition

Interrupts are disabled

Parameters

	fd	The kernel file descriptor.
in,out	*target_size*	Remaining bytes to process; updated after call.
out	*pos*	Output: position in the data region for this chunk.
out	*size*	Output: number of bytes available in this chunk.

Returns

0 on success, -1 on error.

4.10.2.18 k_lfd_get()

```
int k_lfd_get (
            int pid,
            int fd )
```

Get the kernel file descriptor from a local descriptor.

Translates a process-local fd to the global kernel fd.

Precondition

Interrupts are disabled

Parameters

pid	The process ID.
fd	The local file descriptor.

Returns

The kernel file descriptor, or -1 if invalid.

4.10.2.19 k_lfd_put()

```
int k_lfd_put (
            int pid,
            int hfd )
```

Add a kernel file descriptor to a process's local table.

Maps a global kernel fd to a process-local fd number.

Precondition

Interrupts are disabled

Parameters

pid	The process ID.
hfd	The kernel file descriptor to add.

Returns

The local file descriptor number assigned.

4.10.2.20 k_lfd_release()

```
int k_lfd_release (
            int pid,
            int fd )
```

Release a local file descriptor from a process.

Removes the mapping and closes the underlying kernel fd.

Precondition

Interrupts are disabled

Parameters

pid	The process ID.
fd	The local file descriptor to release.

Returns

0 on success, -1 if invalid.

4.11 src/io/filetable/k_filetable.h File Reference

Kernel file descriptor table management API.

`#include "io/fat/file_t.h"`
Include dependency graph for k_filetable.h: This graph shows which files directly or indirectly include this file:

Functions

- int k_fd_init ()

  Initialize the kernel file descriptor table.
- void k_fd_clean ()

  Free the kernel file descriptor table.
- int k_fd_alloc (file_t file, int mode)

  Allocate a kernel file descriptor for a file.
- int k_fd_alloc_h (int hfd, int mode)

  Allocate a kernel file descriptor for a host fd.
- bool k_fd_has_entry (int fd)

  Check if a file descriptor entry exists and is valid.
- int k_fd_get_hfd (int fd)

  Get the host file descriptor if this is a host file.
- int k_fd_dup (int fd)

  Duplicate a kernel file descriptor.
- int k_fd_close (int fd)

  Close a kernel file descriptor.
- int apply_offset (int fd)

Apply pending offset changes to a file descriptor.

- int k_fd_walk (int fd, size_t *target_size, int *pos, size_t *size)

  Walk through file blocks for I/O operations.
- uint32_t k_fd_get_offset (int fd)

  Get the current file offset.
- void k_fd_set_offset (int fd, int off)

  Set the file offset.
- int k_fd_get_size (int fd)

  Get the size of the file.
- int k_fd_mode (int fd)

  Get the access mode of a file descriptor.
- file_t * k_fd_file_ref (int fd)

  Get a reference to the file referenced by a fd.
- int k_lfd_put (int pid, int hfd)

  Add a kernel file descriptor to a process's local table.
- int k_lfd_release (int pid, int fd)

  Release a local file descriptor from a process.
- int k_lfd_get (int pid, int fd)

  Get the kernel file descriptor from a local descriptor.
- int k_fd_is_open (const char *fname)

  Check if a file is currently open.
- int k_fd_is_write_locked (const char *fname)

  Check if a file is open with write or append mode.

4.11.1 Detailed Description

Kernel file descriptor table management API.

This header provides the kernel-level interface for managing file descriptors in PennOS. It supports both Penn↩FAT filesystem files and host OS files, allowing the kernel to handle I/O operations, file seeking, and descriptor duplication/sharing between processes.

Note

Functions prefixed with `k_fd_` operate on the global kernel file descriptor table. Functions prefixed with `k_lfd_` operate on process-local file descriptor mappings.

4.11.2 Function Documentation

4.11.2.1 apply_offset()

```
int apply_offset (
            int fd )
```

Apply pending offset changes to a file descriptor.

Parameters

fd	The file descriptor to update.

Returns

0 on success, -1 on error (EOF in read mode or allocation failure).

4.11.2.2 k_fd_alloc()

```
int k_fd_alloc (
            file_t file,
            int mode )
```

Allocate a kernel file descriptor for a file.

Creates a new file descriptor entry for the given file with the specified access mode.

Precondition

Interrupts are disabled

Parameters

file	The file to open.
mode	Access mode (F_READ, F_WRITE, F_APPEND, or F_EXEC).

Returns

The allocated kernel file descriptor index.

4.11.2.3 k_fd_alloc_h()

```
int k_fd_alloc_h (
            int hfd,
            int mode )
```

Allocate a kernel file descriptor for a host fd.

Creates a new file descriptor entry that wraps a host fd.

Precondition

Interrupts are disabled

Parameters

hfd	The host OS file descriptor.
mode	Access mode (F_READ, F_WRITE, etc.).

Returns

The allocated kernel file descriptor index.

4.11.2.4 k_fd_close()

```
int k_fd_close (
            int fd )
```

Close a kernel file descriptor.

Precondition

Interrupts are disabled

Parameters

fd	The kernel file descriptor to close.

Returns

0 on success.

4.11.2.5 k_fd_dup()

```
int k_fd_dup (
            int fd )
```

Duplicate a kernel file descriptor.

Precondition

Interrupts are disabled

Parameters

fd	The kernel file descriptor to duplicate.

Returns

The same file descriptor on success.

4.11.2.6 k_fd_file_ref()

```
file_t* k_fd_file_ref (
            int fd )
```

Get a reference to the file referenced by a fd.

Precondition

Interrupts are disabled

Parameters

fd	The kernel file descriptor.

Returns

Pointer to the file_t structure.

4.11.2.7 k_fd_get_hfd()

```
int k_fd_get_hfd (
            int fd )
```

Get the host file descriptor if this is a host file.

Precondition

Interrupts are disabled

Parameters

fd	The kernel file descriptor.

Returns

The host file descriptor if it is a host file, else -1

4.11.2.8 k_fd_get_offset()

```
uint32_t k_fd_get_offset (
            int fd )
```

Get the current file offset.

Precondition

Interrupts are disabled

Parameters

fd	The kernel file descriptor.

Returns

The current target offset in bytes.

4.11.2.9 k_fd_get_size()

```
int k_fd_get_size (
            int fd )
```

Get the size of the file.

Precondition

Interrupts are disabled

Parameters

fd	The kernel file descriptor.

Returns

The file size in bytes.

4.11.2.10 k_fd_has_entry()

```
bool k_fd_has_entry (
            int fd )
```

Check if a file descriptor entry exists and is valid.

Parameters

fd	The kernel file descriptor.

Returns

true if the descriptor is valid and has a positive refcount.

4.11.2.11 k_fd_init()

```
int k_fd_init ( )
```

Initialize the kernel file descriptor table.

Returns

0 on success.

4.11.2.12 k_fd_is_open()

```
int k_fd_is_open (
            const char * fname )
```

Check if a file is currently open.

Precondition

Interrupts are disabled

Parameters

fname	The filename to check.

Returns

1 if the file is open, 0 otherwise.

4.11.2.13 k_fd_is_write_locked()

```
int k_fd_is_write_locked (
            const char * fname )
```

Check if a file is open with write or append mode.

Used to enforce write locking - only one writer allowed at a time.

Precondition

Interrupts are disabled

Parameters

fname	The filename to check.

Returns

1 if the file is write-locked, 0 otherwise.

4.11.2.14 k_fd_mode()

```
int k_fd_mode (
            int fd )
```

Get the access mode of a file descriptor.

Parameters

fd	The kernel file descriptor.

Returns

The access mode (F_READ, F_WRITE, F_APPEND, or F_EXEC).

4.11.2.15 k_fd_set_offset()

```
void k_fd_set_offset (
            int fd,
            int off )
```

Set the file offset.

Precondition

Interrupts are disabled

Parameters

fd	The kernel file descriptor.
off	The new offset in bytes.

4.11.2.16 k_fd_walk()

```
int k_fd_walk (
            int fd,
            size_t * target_size,
            int * pos,
            size_t * size )
```

Walk through file blocks for I/O operations.

Advances through the file's FAT chain, returning the position and size of the next chunk to read/write.

Precondition

Interrupts are disabled

Parameters

	fd	The kernel file descriptor.
in,out	*target_size*	Remaining bytes to process; updated after call.
out	*pos*	Output: position in the data region for this chunk.
out	*size*	Output: number of bytes available in this chunk.

Returns

0 on success, -1 on error.

4.11.2.17 k_lfd_get()

```
int k_lfd_get (
            int pid,
            int fd )
```

Get the kernel file descriptor from a local descriptor.

Translates a process-local fd to the global kernel fd.

Precondition

Interrupts are disabled

Parameters

pid	The process ID.
fd	The local file descriptor.

Returns

The kernel file descriptor, or -1 if invalid.

4.11.2.18 k_lfd_put()

```
int k_lfd_put (
            int pid,
            int hfd )
```

Add a kernel file descriptor to a process's local table.

Maps a global kernel fd to a process-local fd number.

Precondition

Interrupts are disabled

Parameters

pid	The process ID.
hfd	The kernel file descriptor to add.

Returns

The local file descriptor number assigned.

4.11.2.19 k_lfd_release()

```
int k_lfd_release (
            int pid,
            int fd )
```

Release a local file descriptor from a process.

Removes the mapping and closes the underlying kernel fd.

Precondition

Interrupts are disabled

Parameters

pid	The process ID.
fd	The local file descriptor to release.

Returns

0 on success, -1 if invalid.

4.12 src/io/fstd.h File Reference

File system standard definitions and constants.

```
#include <stddef.h>
```

Include dependency graph for fstd.h: This graph shows which files directly or indirectly include this file:

Macros

File Access Modes

Constants for specifying how a file is opened.

- #define **F_WRITE** 0
- #define **F_READ** 1
- #define **F_APPEND** 2
- #define **F_EXEC** 3

Seek Whence Constants

Constants for lseek() position reference.

- #define **F_SEEK_SET** 0
- #define **F_SEEK_CUR** 1
- #define **F_SEEK_END** 2

Standard File Descriptors

Process-local file descriptor numbers for standard I/O streams.

- #define **STDOUT_FILENO** 1
- #define **STDIN_FILENO** 0
- #define **STDERR_FILENO** 2

4.12.1 Detailed Description

File system standard definitions and constants.

This header defines constants used throughout PennOS for file operations, including file access modes, seek positions, and standard file descriptors.

4.13 src/io/path/k_path.c File Reference

Implementation of kernel-level directory operations.

```
#include "k_path.h"
#include "s_path.h"
#include "io/fat/k_fat.h"
#include "io/file/k_file.h"
#include "io/filetable/k_filetable.h"
#include "io/fstd.h"
#include "io/term/s_term.h"
#include "process/schedule/k_schedule.h"
#include <stdio.h>
#include <string.h>
```

Include dependency graph for k_path.c:

Functions

- file_t * k_get_root_dir ()

  Get a pointer to the root directory.
- int k_dir_search (const char *fname, file_t *dir, file_t *file)

  Search for a file in a directory.
- void k_dir_upd_entry (file_t *dir, int idx, file_t file)

  Update a directory entry at a given index.
- void k_dir_rem_entry (file_t *dir, int idx)

  Remove a directory entry at a given index.
- int k_dir_add_entry (file_t *dir, file_t file)

  Add a new entry to a directory.
- int k_ls (const char *filename)

  List directory contents (alias for s_ls).

Variables

- file_t root_dir = (file_t){.name = "", .type = 2, .first_block = {1}}

  The global root directory of the PennFAT filesystem.

4.13.1 Detailed Description

Implementation of kernel-level directory operations.

This file implements directory management for the PennFAT filesystem. It provides functions to search, add, update, and remove directory entries. Directory entries are stored as sequential file_t structures within the directory's data blocks.

Note

The root directory is a special file_t with type=2 and first_block=1. All file operations on directories use the same k_read/k_write infrastructure as regular files.

4.13.2 Function Documentation

4.13.2.1 k_dir_add_entry()

```
int k_dir_add_entry (
            file_t * dir,
            file_t file )
```

Add a new entry to a directory.

Scans through the directory to find the end (first null entry), then writes the new file entry. Updates directory size if needed. Ensures proper null termination for block boundary alignment.

Parameters

dir	The directory to add the entry to.
file	The file entry to add.

Returns

0 on success, -1 on error.

4.13.2.2 k_dir_rem_entry()

```
void k_dir_rem_entry (
            file_t * dir,
            int idx )
```

Remove a directory entry at a given index.

Uses two file descriptors: one for reading subsequent entries and one for writing them back shifted by one position. Terminates with a zeroed entry.

Parameters

dir	The directory to modify.
idx	The index of the entry to remove.

4.13.2.3 k_dir_search()

```
int k_dir_search (
            const char * fname,
            file_t * dir,
            file_t * file )
```

Search for a file in a directory.

Opens the directory for reading and iterates through all entries until a match is found or the end is reached (null-terminated name).

Parameters

	fname	The filename to search for.
	dir	The directory to search in.
out	*file*	Output: the file entry if found (can be NULL).

Returns

The index of the file entry if found, -1 otherwise.

4.13.2.4 k_dir_upd_entry()

```
void k_dir_upd_entry (
            file_t * dir,
            int idx,
            file_t file )
```

Update a directory entry at a given index.

Opens the directory for writing, seeks to the entry position, and overwrites it with the new file data.

Parameters

dir	The directory to update.
idx	The index of the entry to update.
file	The new file entry data.

4.13.2.5 k_get_root_dir()

```
file_t* k_get_root_dir ( )
```

Get a pointer to the root directory.

Returns

Pointer to the root directory file_t structure.

4.13.2.6 k_ls()

```
int k_ls (
            const char * filename )
```

List directory contents (alias for s_ls).

List directory contents.

Required by PennOS specification. Delegates to s_ls() for implementation.

Parameters

filename	The specific file to list, or NULL for all files.

Returns

0 on success, -1 on error.

4.14 src/io/path/k_path.h File Reference

Kernel-level directory and path operations for PennFAT.

`#include "io/fat/file_t.h"`
Include dependency graph for k_path.h: This graph shows which files directly or indirectly include this file:

Functions

- file_t ∗ k_get_root_dir ()
  Get a pointer to the root directory.
- int k_dir_search (const char ∗fname, file_t ∗dir, file_t ∗file)
  Search for a file in a directory.
- void k_dir_upd_entry (file_t ∗dir, int idx, file_t file)
  Update a directory entry at a given index.
- void k_dir_rem_entry (file_t ∗dir, int idx)
  Remove a directory entry at a given index.
- int k_dir_add_entry (file_t ∗dir, file_t file)
  Add a new entry to a directory.
- int k_ls (const char ∗filename)
  List directory contents.

4.14.1 Detailed Description

Kernel-level directory and path operations for PennFAT.

This header provides the kernel-level interface for directory operations in the PennFAT filesystem. It supports searching, adding, updating, and removing directory entries, as well as listing directory contents.

Note

PennFAT uses a flat directory structure with a single root directory. All files reside directly under the root.

4.14.2 Function Documentation

4.14.2.1 k_dir_add_entry()

```
int k_dir_add_entry (
            file_t * dir,
            file_t file )
```

Add a new entry to a directory.

Appends a new file entry to the directory. Searches for the first empty slot (null-terminated name) or appends at the end.

Parameters

dir	The directory to add the entry to.
file	The file entry to add.

Returns

0 on success, -1 on error.

Scans through the directory to find the end (first null entry), then writes the new file entry. Updates directory size if needed. Ensures proper null termination for block boundary alignment.

Parameters

dir	The directory to add the entry to.
file	The file entry to add.

Returns

0 on success, -1 on error.

4.14.2.2 k_dir_rem_entry()

```
void k_dir_rem_entry (
            file_t * dir,
            int idx )
```

Remove a directory entry at a given index.

Removes the entry at the specified index and shifts all subsequent entries up to fill the gap. Writes a null terminator after the last entry.

Parameters

dir	The directory to modify.
idx	The index of the entry to remove.

Uses two file descriptors: one for reading subsequent entries and one for writing them back shifted by one position. Terminates with a zeroed entry.

Parameters

dir	The directory to modify.
idx	The index of the entry to remove.

4.14.2.3 k_dir_search()

```
int k_dir_search (
            const char * fname,
            file_t * dir,
            file_t * file )
```

Search for a file in a directory.

Searches the specified directory for a file with the given name. If found, optionally copies the file entry to the output parameter.

Parameters

	fname	The filename to search for.
	dir	The directory to search in.
out	*file*	Output: the file entry if found (can be NULL).

Returns

The index of the file entry if found, -1 otherwise.

Opens the directory for reading and iterates through all entries until a match is found or the end is reached (null-terminated name).

Parameters

	fname	The filename to search for.
	dir	The directory to search in.
out	*file*	Output: the file entry if found (can be NULL).

Returns

The index of the file entry if found, -1 otherwise.

4.14.2.4 k_dir_upd_entry()

```
void k_dir_upd_entry (
            file_t * dir,
            int idx,
            file_t file )
```

Update a directory entry at a given index.

Overwrites the directory entry at the specified index with the provided file structure.

Parameters

dir	The directory to update.
idx	The index of the entry to update.
file	The new file entry data.

Opens the directory for writing, seeks to the entry position, and overwrites it with the new file data.

Parameters

dir	The directory to update.
idx	The index of the entry to update.
file	The new file entry data.

4.14.2.5 k_get_root_dir()

```
file_t* k_get_root_dir ( )
```

Get a pointer to the root directory.

Returns a reference to the global root directory structure. The root directory is the only directory in PennFAT.

Returns

Pointer to the root directory file_t structure.

Pointer to the root directory file_t structure.

4.14.2.6 k_ls()

```
int k_ls (
            const char * filename )
```

List directory contents.

Lists files in the root directory. If a filename is provided, lists only that file. Otherwise, lists all files. This is an alias for s_ls().

Parameters

filename	The specific file to list, or NULL for all files.

Returns

0 on success, -1 on error.

List directory contents.

Required by PennOS specification. Delegates to s_ls() for implementation.

Parameters

filename	The specific file to list, or NULL for all files.

Returns

0 on success, -1 on error.

4.15 src/io/path/s_path.c File Reference

Implementation of system-level directory listing.

```
#include "s_path.h"
#include "io/fat/file_t.h"
#include "io/file/k_file.h"
#include "io/filetable/k_filetable.h"
#include "io/fstd.h"
#include "io/term/s_term.h"
#include "k_path.h"
#include "process/schedule/k_schedule.h"
#include <string.h>
```

Include dependency graph for s_path.c:

Functions

- void print_file_entry (file_t file)

  Print a formatted file entry to the terminal.
- int s_ls (const char *filename)

  List files in the root directory.

4.15.1 Detailed Description

Implementation of system-level directory listing.

This file implements the s_ls() system call for listing directory contents in PennOS. It formats and prints file entries with their metadata including permissions, size, modification time, and name.

4.15.2 Function Documentation

4.15.2.1 print_file_entry()

```
void print_file_entry (
            file_t file )
```

Print a formatted file entry to the terminal.

Displays file information in the format: `[block] - [rwx] [size] [mtime] [name]`

Where block is the first data block number (or "-" if none allocated), rwx are the permission flags, size is in bytes, mtime is the modification timestamp, and name is the filename.

Parameters

file	The file entry to print.

4.15.2.2 s_ls()

```
int s_ls (
            const char * filename )
```

List files in the root directory.

Disables interrupts during execution for consistency. If a filename is provided, searches for and displays only that file. Otherwise, iterates through all directory entries and prints each one.

Parameters

filename	The specific file to list, or NULL to list all files.

Returns

0 on success, -1 on error (e.g., cannot open directory).

4.16 src/io/path/s_path.h File Reference

System-level path and directory listing interface.

This graph shows which files directly or indirectly include this file:

Functions

- int s_ls (const char ∗filename)
  List files in the root directory.

4.16.1 Detailed Description

System-level path and directory listing interface.

This header provides the system call interface for directory listing in PennOS. It is used by user programs and the shell to display directory contents.

4.16.2 Function Documentation

4.16.2.1 s_ls()

```
int s_ls (
            const char * filename )
```

List files in the root directory.

If a filename is provided, displays information for that specific file. Otherwise, lists all files in the root directory with their permissions, size, modification time, and name.

Parameters

filename	The specific file to list, or NULL to list all files.

Returns

0 on success, -1 on error.

Disables interrupts during execution for consistency. If a filename is provided, searches for and displays only that file. Otherwise, iterates through all directory entries and prints each one.

Parameters

filename	The specific file to list, or NULL to list all files.

Returns

0 on success, -1 on error (e.g., cannot open directory).

4.17 src/io/term/k_term.c File Reference

Implementation of kernel-level terminal management.

```
#include "k_term.h"
#include "io/filetable/k_filetable.h"
#include "io/fstd.h"
#include "process/schedule/k_schedule.h"
```

Include dependency graph for k_term.c:

Functions

- bool k_is_term (int fd)

  Check if a file descriptor refers to the terminal.
- void k_claim_terminal (pid_t pid)

  Set the terminal owner to the specified process.
- pid_t k_get_terminal_pass_parent ()

  Get the parent process that should pass terminal to its child.
- void k_pass_terminal_to_next_child (pid_t parent)

  Mark a parent to pass terminal ownership to its next child.
- pid_t k_get_terminal_owner ()

  Get the current terminal owner.

Variables

- pid_t **terminal_owner** = -1
- pid_t **terminal_pass_parent** = -1

4.17.1 Detailed Description

Implementation of kernel-level terminal management.

This file implements terminal ownership tracking for PennOS job control. It maintains state about which process currently owns the terminal (foreground process) and supports terminal inheritance during fork/exec.

4.17.2 Function Documentation

4.17.2.1 k_claim_terminal()

```
void k_claim_terminal (
            pid_t pid )
```

Set the terminal owner to the specified process.

Parameters

pid	The process ID to grant terminal ownership.

4.17.2.2 k_get_terminal_owner()

```
pid_t k_get_terminal_owner ( )
```

Get the current terminal owner.

Returns

The terminal owner's PID, or -1 if no owner is set.

4.17.2.3 k_get_terminal_pass_parent()

```
pid_t k_get_terminal_pass_parent ( )
```

Get the parent process that should pass terminal to its child.

Returns

The parent PID, or -1 if none is set.

4.17.2.4 k_is_term()

```
bool k_is_term (
            int fd )
```

Check if a file descriptor refers to the terminal.

A file descriptor refers to the terminal if its underlying host file descriptor is STDIN_FILENO.

Precondition

Interrupts are disabled

Parameters

fd	The file descriptor to check.

Returns

true if fd refers to the terminal, false otherwise.

4.17.2.5 k_pass_terminal_to_next_child()

```
void k_pass_terminal_to_next_child (
            pid_t parent )
```

Mark a parent to pass terminal ownership to its next child.

Parameters

parent	The parent process ID that will pass terminal ownership.

4.18 src/io/term/k_term.h File Reference

Kernel-level terminal management interface.

`#include "process/pstd.h"`
Include dependency graph for k_term.h: This graph shows which files directly or indirectly include this file:

Functions

- bool k_is_term (int fd)

  Check if a file descriptor refers to the terminal.
- void k_claim_terminal (pid_t pid)

Set the terminal owner to the specified process.

- pid_t k_get_terminal_pass_parent ()

  Get the parent process that should pass terminal to its child.

- void k_pass_terminal_to_next_child (pid_t parent)

  Mark a parent to pass terminal ownership to its next child.

- pid_t k_get_terminal_owner ()

  Get the current terminal owner.

4.18.1 Detailed Description

Kernel-level terminal management interface.

This header provides the kernel-level interface for terminal ownership and control in PennOS. It manages which process currently owns the terminal (foreground process) for job control purposes.

4.18.2 Function Documentation

4.18.2.1 k_claim_terminal()

```
void k_claim_terminal (
            pid_t pid )
```

Set the terminal owner to the specified process.

Grants terminal ownership to the given process, making it the foreground process for terminal I/O.

Parameters

pid	The process ID to grant terminal ownership.
pid	The process ID to grant terminal ownership.

4.18.2.2 k_get_terminal_owner()

```
pid_t k_get_terminal_owner ( )
```

Get the current terminal owner.

Returns the PID of the process that currently owns the terminal (the foreground process).

Returns

The terminal owner's PID, or -1 if no owner is set.

The terminal owner's PID, or -1 if no owner is set.

4.18.2.3 k_get_terminal_pass_parent()

```
pid_t k_get_terminal_pass_parent ( )
```

Get the parent process that should pass terminal to its child.

Returns the PID of the parent process that has been marked to pass terminal ownership to its next child process.

Returns

The parent PID, or -1 if none is set.

The parent PID, or -1 if none is set.

4.18.2.4 k_is_term()

```
bool k_is_term (
            int fd )
```

Check if a file descriptor refers to the terminal.

Determines whether the given file descriptor is connected to standard input (the terminal).

Parameters

fd	The file descriptor to check.

Returns

true if fd refers to the terminal, false otherwise.

A file descriptor refers to the terminal if its underlying host file descriptor is STDIN_FILENO.

Precondition

Interrupts are disabled

Parameters

fd	The file descriptor to check.

Returns

true if fd refers to the terminal, false otherwise.

4.18.2.5 k_pass_terminal_to_next_child()

```
void k_pass_terminal_to_next_child (
            pid_t parent )
```

Mark a parent to pass terminal ownership to its next child.

Used during process creation to ensure the child process inherits terminal ownership from its parent (for foreground job execution).

Parameters

parent	The parent process ID that will pass terminal ownership.
parent	The parent process ID that will pass terminal ownership.

4.19 src/io/term/s_term.c File Reference

Implementation of system-level terminal operations.

```
#include "s_term.h"
#include "k_term.h"
#include "process/schedule/k_schedule.h"
```

Include dependency graph for s_term.c:

Functions

- void s_claim_terminal (pid_t pid)

  Set the terminal owner to the specified process.
- pid_t s_get_terminal_owner ()

  Get the current terminal owner.
- void s_pass_terminal_to_next_child (pid_t parent)

  Mark a parent to pass terminal ownership to its next child.
- pid_t s_get_terminal_pass_parent ()

  Get the parent process that should pass terminal to its child.

4.19.1 Detailed Description

Implementation of system-level terminal operations.

This file implements the system call wrappers for terminal ownership management in PennOS. These functions provide safe access to kernel terminal state by disabling interrupts during critical sections.

4.19.2 Function Documentation

4.19.2.1 s_claim_terminal()

```
void s_claim_terminal (
            pid_t pid )
```

Set the terminal owner to the specified process.

Disables interrupts before calling the kernel function to ensure atomic modification of terminal ownership state.

Parameters

pid	The process ID to grant terminal ownership.

4.19.2.2 s_get_terminal_owner()

```
pid_t s_get_terminal_owner ( )
```

Get the current terminal owner.

Returns

The terminal owner's PID, or -1 if no owner is set.

4.19.2.3 s_get_terminal_pass_parent()

```
pid_t s_get_terminal_pass_parent ( )
```

Get the parent process that should pass terminal to its child.

Returns

The parent PID, or -1 if none is set.

4.19.2.4 s_pass_terminal_to_next_child()

```
void s_pass_terminal_to_next_child (
            pid_t parent )
```

Mark a parent to pass terminal ownership to its next child.

Disables interrupts before calling the kernel function to ensure atomic modification of terminal pass state.

Parameters

parent	The parent process ID that will pass terminal ownership.

4.20 src/io/term/s_term.h File Reference

System-level terminal I/O and ownership interface.

```
#include <stdio.h>
#include <stdlib.h>
#include "io/file/s_file.h"
#include "process/pstd.h"
```

Include dependency graph for s_term.h: This graph shows which files directly or indirectly include this file:

Macros

- #define s_fprintf(fd, fmt, ...)

  Formatted print to a file descriptor.
- #define s_printf(...) s_fprintf(STDOUT_FILENO, __VA_ARGS__)

  Formatted print to standard output.
- #define s_perror(...) s_fprintf(STDERR_FILENO, __VA_ARGS__)

  Formatted print to standard error.

Functions

- void s_claim_terminal (pid_t pid)

  Set the terminal owner to the specified process.
- pid_t s_get_terminal_pass_parent ()

  Get the parent process that should pass terminal to its child.
- void s_pass_terminal_to_next_child (pid_t parent)

  Mark a parent to pass terminal ownership to its next child.
- pid_t s_get_terminal_owner ()

  Get the current terminal owner.

4.20.1 Detailed Description

System-level terminal I/O and ownership interface.

This header provides the system call interface for terminal operations in PennOS. It includes functions for terminal ownership management (job control) and formatted output macros for printing to the terminal.

Note

The s_fprintf/s_printf/s_perror macros provide printf-style formatted output using the PennOS file system calls.

4.20.2 Macro Definition Documentation

4.20.2.1 s_fprintf

```
#define s_fprintf(
            fd,
            fmt,
            ...  )
```

Value:

```
({                                                                  \
  int __sp_nbytes = snprintf(NULL, 0, fmt __VA_OPT__(, ) __VA_ARGS__);      \
  char* __sp_buf = malloc(__sp_nbytes + 2);                         \
  __sp_nbytes =                                                     \
      snprintf(__sp_buf, __sp_nbytes + 2, fmt __VA_OPT__(, ) __VA_ARGS__); \
  int res = s_write(fd, __sp_buf, __sp_nbytes);                     \
  free(__sp_buf);                                                   \
  res;                                                              \
})
```

Formatted print to a file descriptor.

Printf-style formatted output that writes to the specified file descriptor using s_write(). Allocates a temporary buffer for formatting.

Parameters

fd	The file descriptor to write to.
fmt	The printf-style format string.
...	Variable arguments for format string.

Returns

Number of bytes written, or -1 on error.

4.20.2.2 s_perror

```
#define s_perror(
            ...  ) s_fprintf(STDERR_FILENO, __VA_ARGS__)
```

Formatted print to standard error.

Convenience macro that calls s_fprintf with STDERR_FILENO. Used for error messages.

Parameters

...	Printf-style format string and arguments.

Returns

Number of bytes written, or -1 on error.

4.20.2.3 s_printf

```
#define s_printf(
              ...  ) s_fprintf(STDOUT_FILENO, __VA_ARGS__)
```

Formatted print to standard output.

Convenience macro that calls s_fprintf with STDOUT_FILENO.

Parameters

...	Printf-style format string and arguments.

Returns

Number of bytes written, or -1 on error.

4.20.3 Function Documentation

4.20.3.1 s_claim_terminal()

```
void s_claim_terminal (
            pid_t pid )
```

Set the terminal owner to the specified process.

System call wrapper that safely claims terminal ownership with interrupts disabled.

Parameters

pid	The process ID to grant terminal ownership.

Disables interrupts before calling the kernel function to ensure atomic modification of terminal ownership state.

Parameters

pid	The process ID to grant terminal ownership.

4.20.3.2 s_get_terminal_owner()

```
pid_t s_get_terminal_owner ( )
```

Get the current terminal owner.

Returns

The terminal owner's PID, or -1 if no owner is set.

4.20.3.3 s_get_terminal_pass_parent()

```
pid_t s_get_terminal_pass_parent ( )
```

Get the parent process that should pass terminal to its child.

Returns

The parent PID, or -1 if none is set.

4.20.3.4 s_pass_terminal_to_next_child()

```
void s_pass_terminal_to_next_child (
            pid_t parent )
```

Mark a parent to pass terminal ownership to its next child.

System call wrapper that safely sets the terminal pass parent with interrupts disabled.

Parameters

parent	The parent process ID that will pass terminal ownership.

Disables interrupts before calling the kernel function to ensure atomic modification of terminal pass state.

Parameters

parent	The parent process ID that will pass terminal ownership.

4.21 src/pennfat.c File Reference

Standalone PennFAT filesystem utility.

```
#include "pennfat.h"
#include <errno.h>
#include <fcntl.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
```

```
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <unistd.h>
#include "io/fat/s_fat.h"
#include "io/filetable/k_filetable.h"
#include "io/term/s_term.h"
#include "shell/file/u_file.h"
#include "util/parser.h"
#include "util/vector.h"
```

Include dependency graph for pennfat.c:

Functions

- int main (int argc, char ∗argv[ ])

  Main entry point for the PennFAT utility.

4.21.1 Detailed Description

Standalone PennFAT filesystem utility.

This file implements a standalone command-line interface for interacting with PennFAT filesystems. It provides commands for creating, mounting, and manipulating FAT filesystems outside of PennOS.

Supported commands:

- mkfs: Create a new filesystem
- mount/unmount: Mount/unmount a filesystem
- ls, touch, cat, chmod, rm, mv, cp: File operations

4.21.2 Function Documentation

4.21.2.1 main()

```
int main (
            int argc,
            char * argv[] )
```

Main entry point for the PennFAT utility.

Runs an interactive REPL loop that accepts filesystem commands. Ignores SIGINT and SIGTSTP to prevent accidental termination. Initializes stdin/stdout/stderr in the kernel file descriptor table.

Parameters

argc	Argument count (unused).
argv	Argument vector (unused).

Returns

0 on successful exit.

4.22 src/pennfat.h File Reference

Header for the PennFAT standalone filesystem utility.

This graph shows which files directly or indirectly include this file:

4.22.1 Detailed Description

Header for the PennFAT standalone filesystem utility.

This header provides declarations for the PennFAT filesystem interface. The actual filesystem operations are implemented in the io/fat/ modules.

Note

Most FAT operations have been modularized into separate files:

- io/fat/k_fat.h: Kernel-level FAT operations
- io/fat/s_fat.h: System-level FAT wrappers
- io/file/k_file.h: File I/O operations
- shell/file/u_file.h: User-level file commands

4.23 src/pennos.c File Reference

PennOS kernel main entry point and INIT process implementation.

```
#include "pennos.h"
#include "io/fat/s_fat.h"
#include "io/filetable/k_filetable.h"
#include "io/term/s_term.h"
#include "process/lifecycle/k_lifecycle.h"
#include "process/lifecycle/s_lifecycle.h"
#include "process/pcb/k_pcb.h"
#include "process/pcb/s_pcb.h"
#include "process/schedule/k_schedule.h"
#include "process/signal/s_signal.h"
#include "shell/u_shell.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "util/log.h"
#include "util/spthread.h"
```

Include dependency graph for pennos.c:

Macros

- #define DEFAULT_LOG_FILE "log"

  Default log file path if not specified on command line.

Functions

- int main (int argc, char ∗argv[ ])

  Main entry point for PennOS.
- void ∗ u_init_main (void ∗arg)

  INIT process - reaps orphaned and zombie children.

4.23.1 Detailed Description

PennOS kernel main entry point and INIT process implementation.

This file contains the main() function that boots PennOS and the INIT process that serves as the parent of all user processes. PennOS is a Unix-like operating system with:

- Priority-based process scheduling (3 levels)
- FAT filesystem support
- Job control and signal handling
- Interactive shell

4.23.2 Function Documentation

4.23.2.1 main()

```
int main (
            int argc,
            char * argv[] )
```

Main entry point for PennOS.

Initializes all kernel subsystems and starts the scheduler. Boot sequence:

1. Initialize standard file descriptors (stdin, stdout, stderr)
2. Parse command line arguments (fatfs file, optional log file)
3. Initialize logging subsystem
4. Mount the FAT filesystem
5. Initialize PCB table
6. Initialize scheduler
7. Spawn INIT process (PID 1)
8. Start scheduler main loop
9. Cleanup and shutdown

Parameters

argc	Argument count.
argv	Argument vector: argv[1] = FAT filesystem file, argv[2] = log file (optional, defaults to "log").

Returns

0 on successful shutdown, 1 on initialization error.

4.23.2.2 u_init_main()

```
void* u_init_main (
            void * arg )
```

INIT process - reaps orphaned and zombie children.

INIT process main function.

INIT is the first user process (PID 1) and serves as the ancestor of all other processes. It performs two main functions:

1. **Shell spawning**: Creates the interactive shell process and configures it to ignore SIGINT/SIGSTOP (so Ctrl-C/Ctrl-Z affect foreground jobs, not the shell itself).
2. **Zombie reaping**: Continuously calls s_waitpid(-1) to reap any child that terminates. This includes:
   - Direct children (the shell)
   - Orphaned processes whose parents exited

INIT exits only when the shell terminates (via logout or Ctrl-D), at which point it reaps any remaining zombies and returns.

Parameters

arg	Unused parameter (required by spthread function signature).

Returns

NULL on exit.

4.24 src/pennos.h File Reference

PennOS kernel initialization and INIT process interface.

This graph shows which files directly or indirectly include this file:

Functions

- void ∗ u_init_main (void ∗arg)

  INIT process main function.

4.24.1 Detailed Description

PennOS kernel initialization and INIT process interface.

This header declares the INIT process entry point. INIT is PID 1 and serves as the ancestor of all user processes in PennOS. It is responsible for spawning the shell and reaping orphaned/zombie children.

4.24.2 Function Documentation

4.24.2.1 u_init_main()

```
void* u_init_main (
            void * arg )
```

INIT process main function.

INIT is the first user-space process started by PennOS (PID 1). Its responsibilities include:

- Spawning the interactive shell process
- Adopting orphaned processes when their parents exit
- Reaping zombie children to prevent resource leaks
- Coordinating graceful shutdown when the shell exits

INIT runs in an infinite loop calling s_waitpid(-1) to reap any children that become zombies. It exits only when the shell terminates.

Parameters

arg	Unused parameter (required by spthread signature).

Returns

NULL (never returns during normal operation).

INIT process main function.

INIT is the first user process (PID 1) and serves as the ancestor of all other processes. It performs two main functions:

1. **Shell spawning**: Creates the interactive shell process and configures it to ignore SIGINT/SIGSTOP (so Ctrl-C/Ctrl-Z affect foreground jobs, not the shell itself).
2. **Zombie reaping**: Continuously calls s_waitpid(-1) to reap any child that terminates. This includes:
   - Direct children (the shell)
   - Orphaned processes whose parents exited

INIT exits only when the shell terminates (via logout or Ctrl-D), at which point it reaps any remaining zombies and returns.

Parameters

arg	Unused parameter (required by spthread function signature).

Returns

NULL on exit.

4.25 src/process/lifecycle/k_lifecycle.c File Reference

Implementation of kernel-level process lifecycle management.

```
#include "k_lifecycle.h"
#include "io/filetable/k_filetable.h"
#include "process/pcb/k_pcb.h"
#include "process/schedule/k_schedule.h"
#include "process/signal/k_signal.h"
#include "util/log.h"
#include <string.h>
```

Include dependency graph for k_lifecycle.c:

Functions

- void transfer_parent (pid_t *head, pid_t to_move, pid_t new_parent)

  Transfer a list of children to a new parent.
- pid_t k_proc_create (pid_t parent, pid_t pid, spthread_t thread)

  Create a new child process with initialized PCB.
- void k_proc_zombify (pid_t pid, bool signaled)

  Convert an active process to zombie state.
- void k_proc_cleanup (pid_t pid)

  Reap a zombie process and release its resources.

Variables

- size_t ORPHAN_FALLBACK = 1

  PID of the process that adopts orphaned children (init process).

4.25.1 Detailed Description

Implementation of kernel-level process lifecycle management.

This file implements process creation, termination, and cleanup for PennOS. It manages the parent-child relationships, orphan reparenting to init, and proper cleanup of process resources upon termination.

Note

All functions that modify PCB state assert they are not called from interrupt context to prevent race conditions.

4.25.2 Function Documentation

4.25.2.1 k_proc_cleanup()

```
void k_proc_cleanup (
            pid_t pid )
```

Reap a zombie process and release its resources.

Clean up a terminated/finished thread's resources. This may include freeing the PCB, handling children, etc.

Parameters

pid	The zombie process to clean up.

Precondition

Interrupts are disabled

4.25.2.2 k_proc_create()

```
pid_t k_proc_create (
            pid_t parent,
            pid_t pid,
            spthread_t thread )
```

Create a new child process with initialized PCB.

Create a new child process, inheriting applicable properties from the parent.

Parameters

parent	The parent process ID.
pid	The PID claimed for this process.
thread	The spthread for this process.

Precondition

Interrupts are disabled

Returns

The process PID.

4.25.2.3 k_proc_zombify()

```
void k_proc_zombify (
            pid_t pid,
            bool signaled )
```

Convert an active process to zombie state.

Convert an active process to a zombified process.

Parameters

pid	The process to zombify.
signaled	true if killed by signal, false for normal exit.

Precondition

Interrupts are disabled

4.25.2.4 transfer_parent()

```
void transfer_parent (
            pid_t * head,
            pid_t to_move,
            pid_t new_parent )
```

Transfer a list of children to a new parent.

Parameters

in,out	*head*	Pointer to the new parent's child list head.
	to_move	Head of the circular sibling list to transfer.
	new_parent	PID of the new parent process.

Precondition

Interrupts are disabled

4.26 src/process/lifecycle/k_lifecycle.h File Reference

Kernel-level process lifecycle management.

```
#include "process/pcb/pcb_t.h"
#include "process/pstd.h"
#include "util/spthread.h"
```

Include dependency graph for k_lifecycle.h: This graph shows which files directly or indirectly include this file:

Functions

- spthread_t k_spthread_create (void ∗(∗func)(void ∗), char ∗argv[ ], int fd0, int fd1)

  Create an spthread for a new process.
- pid_t k_proc_create (pid_t parent, pid_t child, spthread_t thread)

  Create a new child process, inheriting applicable properties from the parent.
- void k_proc_cleanup (pid_t proc)

  Clean up a terminated/finished thread's resources. This may include freeing the PCB, handling children, etc.
- void k_proc_zombify (pid_t proc, bool signaled)

  Convert an active process to a zombified process.
- pcb_t ∗ k_pcb_data (pid_t pid)

  Obtain the PCB data associated with a given pid. There should be no interrupts while holding this pcb data else it may become stale or cause a race condition.

4.26.1 Detailed Description

Kernel-level process lifecycle management.

This header provides the kernel-level interface for managing process lifecycles in PennOS, including process creation, termination (zombification), and cleanup (reaping). These functions manipulate PCB state directly and must be called with interrupts disabled.

4.26.2 Function Documentation

4.26.2.1 k_pcb_data()

```
pcb_t* k_pcb_data (
            pid_t pid )
```

Obtain the PCB data associated with a given pid. There should be no interrupts while holding this pcb data else it may become stale or cause a race condition.

Obtain the PCB data associated with a given pid. There should be no interrupts while holding this pcb data else it may become stale or cause a race condition.

Returns a pointer to the PCB for direct manipulation.

Parameters

pid	The process ID to look up.

Precondition

Interrupts are disabled

Returns

Pointer to the PCB structure.

4.26.2.2 k_proc_cleanup()

```
void k_proc_cleanup (
            pid_t pid )
```

Clean up a terminated/finished thread's resources. This may include freeing the PCB, handling children, etc.

Clean up a terminated/finished thread's resources. This may include freeing the PCB, handling children, etc.

Parameters

pid	The zombie process to clean up.

Precondition

Interrupts are disabled

4.26.2.3 k_proc_create()

```
pid_t k_proc_create (
            pid_t parent,
            pid_t pid,
            spthread_t thread )
```

Create a new child process, inheriting applicable properties from the parent.

Returns

Reference to the child PCB.

Create a new child process, inheriting applicable properties from the parent.

Parameters

parent	The parent process ID.
pid	The PID claimed for this process.
thread	The spthread for this process.

Precondition

Interrupts are disabled

Returns

The process PID.

4.26.2.4 k_proc_zombify()

```
void k_proc_zombify (
            pid_t pid,
            bool signaled )
```

Convert an active process to a zombified process.

Convert an active process to a zombified process.

Parameters

pid	The process to zombify.
signaled	true if killed by signal, false for normal exit.

Precondition

Interrupts are disabled

4.26.2.5 k_spthread_create()

```
spthread_t k_spthread_create (
            void *(*)(void *) func,
            char * argv[],
            int fd0,
            int fd1 )
```

Create an spthread for a new process.

Creates the underlying spthread that will execute the process's code. This is a low-level function used internally by process creation.

Parameters

func	The entry point function for the thread.
argv	Null-terminated argument array for the process.
fd0	File descriptor for stdin.
fd1	File descriptor for stdout.

Returns

The created spthread structure.

4.27 src/process/lifecycle/s_lifecycle.c File Reference

Implementation of system-level process lifecycle operations.

```
#include "s_lifecycle.h"
#include "k_lifecycle.h"
#include "io/filetable/k_filetable.h"
#include "process/pcb/k_pcb.h"
#include "process/pcb/pcb_t.h"
#include "process/schedule/k_schedule.h"
#include "process/signal/k_signal.h"
#include "thread_info_t.h"
#include <string.h>
#include "util/log.h"
```

Include dependency graph for s_lifecycle.c:

Functions

- void ∗ process_spawner (void ∗args)

  Entry point for newly spawned process threads.
- pid_t s_spawn (void ∗(∗func)(void ∗), const char ∗name, char ∗argv[ ], int fd0, int fd1, int fd2)

  Spawn a new child process.
- pid_t s_waitpid (pid_t target, int ∗wstatus, bool nohang)

  Wait for a child process to change state.
- void s_exit ()

  Exit the current process.

4.27.1 Detailed Description

Implementation of system-level process lifecycle operations.

This file implements the user-facing system calls for process management: spawning new processes, waiting for children, and exiting. It handles the translation between user-space and kernel-space operations.

4.27.2 Function Documentation

4.27.2.1 process_spawner()

```
void* process_spawner (
            void * args )
```

Entry point for newly spawned process threads.

Parameters

args	Pointer to thread_info_t structure.

Returns

The return value from the process's main function.

4.27.2.2 s_exit()

```
void s_exit (
            void  )
```

Exit the current process.

Unconditionally exit the calling process.

4.27.2.3 s_spawn()

```
pid_t s_spawn (
            void *(*)(void *) func,
            const char * name,
            char * argv[],
            int fd0,
            int fd1,
            int fd2 )
```

Spawn a new child process.

Create a child process that executes the function `func`. The child will retain some attributes of the parent.

Parameters

func	Entry point function for the process.
name	Process name for logging.
argv	Argument vector.
fd0	Local stdin fd.
fd1	Local stdout fd.
fd2	Local stderr fd.

Returns

The new process's PID, or -1 on error.

4.27.2.4 s_waitpid()

```
pid_t s_waitpid (
            pid_t target,
            int * wstatus,
            bool nohang )
```

Wait for a child process to change state.

Wait on a child of the calling process, until it changes state. If `nohang` is true, this will not block the calling process and return immediately.

Parameters

	target	PID to wait for (-1 = any child).
`out`	*wstatus*	Status output.
	nohang	Return immediately if no child ready.

Returns

PID of child, 0 if nohang with no child ready, -1 on error.

4.28 src/process/lifecycle/s_lifecycle.h File Reference

System-level process lifecycle interface.

`#include "process/pstd.h"`
Include dependency graph for s_lifecycle.h: This graph shows which files directly or indirectly include this file:

Functions

- pid_t s_spawn (void ∗(∗func)(void ∗), const char ∗name, char ∗argv[ ], int fd0, int fd1, int fd2)

  Create a child process that executes the function `func`. The child will retain some attributes of the parent.
- void s_exit (void)

  Unconditionally exit the calling process.
- pid_t s_waitpid (pid_t pid, int ∗wstatus, bool nohang)

  Wait on a child of the calling process, until it changes state. If `nohang` is true, this will not block the calling process and return immediately.

4.28.1 Detailed Description

System-level process lifecycle interface.

This header provides the system call interface for process lifecycle operations in PennOS, including spawning new processes, waiting for child termination, and voluntary process exit.

4.28.2 Function Documentation

4.28.2.1 s_exit()

```
void s_exit (
            void  )
```

Unconditionally exit the calling process.

Unconditionally exit the calling process.

4.28.2.2 s_spawn()

```
pid_t s_spawn (
            void *(*)(void *) func,
            const char * name,
            char * argv[],
            int fd0,
            int fd1,
            int fd2 )
```

Create a child process that executes the function `func`. The child will retain some attributes of the parent.

Parameters

func	Function to be executed by the child process.
argv	Null-terminated array of args, including the command name as argv[0].
fd0	Input file descriptor.
fd1	Output file descriptor.
fd2	Error file descriptor.

Returns

pid_t The process ID of the created child process.

Create a child process that executes the function `func`. The child will retain some attributes of the parent.

Parameters

func	Entry point function for the process.
name	Process name for logging.
argv	Argument vector.
fd0	Local stdin fd.
fd1	Local stdout fd.
fd2	Local stderr fd.

Returns

The new process's PID, or -1 on error.

4.28.2.3 s_waitpid()

```
pid_t s_waitpid (
            pid_t target,
            int * wstatus,
            bool nohang )
```

Wait on a child of the calling process, until it changes state. If `nohang` is true, this will not block the calling process and return immediately.

Parameters

pid	Process ID of the child to wait for.
wstatus	Pointer to an integer variable where the status will be stored.
nohang	If true, return immediately if no child has exited.

Returns

pid_t The process ID of the child which has changed state on success, -1 on error.

Wait on a child of the calling process, until it changes state. If `nohang` is true, this will not block the calling process and return immediately.

Parameters

	target	PID to wait for (-1 = any child).
`out`	*wstatus*	Status output.
	nohang	Return immediately if no child ready.

Returns

PID of child, 0 if nohang with no child ready, -1 on error.

4.29 src/process/lifecycle/thread_info_t.h File Reference

Thread initialization information structure.

`#include "util/spthread.h"`
Include dependency graph for thread_info_t.h: This graph shows which files directly or indirectly include this file:

Classes

- struct thread_info_st

  Information passed to a newly spawned process thread.

Typedefs

- typedef struct thread_info_st thread_info_t

  Information passed to a newly spawned process thread.

4.29.1 Detailed Description

Thread initialization information structure.

This header defines the structure used to pass initialization data to newly created process threads. It contains all information needed to set up a new process's execution context.

4.29.2 Typedef Documentation

4.29.2.1 thread_info_t

```
typedef struct thread_info_st thread_info_t
```

Information passed to a newly spawned process thread.

This structure is allocated by the parent process and passed to the child's thread entry point (process_spawner). The child is responsible for freeing this structure after extracting the data.

4.30 src/process/pcb/k_pcb.c File Reference

Implementation of kernel-level PCB table management.

```
#include "k_pcb.h"
#include "process/pcb/pcb_table_t.h"
#include "process/schedule/k_schedule.h"
```

Include dependency graph for k_pcb.c:

Functions

- void k_pcb_list_add (pid_t *head, pid_t to_add)

  Initialize the PCB table.
- void k_pcb_list_remove (pid_t *head, pid_t to_remove)

  Remove a PID from a circular sibling list.
- void k_pcb_list_insert (pid_t *head, pid_t to_insert)

  Insert an entire circular list into another.
- void k_proc_release (pid_t pid)

  Release a PID for reuse.
- pid_t k_pcb_claim ()

  Claim a new PID from the PCB table.
- int k_pcb_init ()

  Initialize the PCB table.
- void k_pcb_clean ()

  Free the PCB table when exiting.
- pcb_t * k_pcb_data (pid_t pid)

  Get direct access to a process's PCB.
- int k_pcb_table_size ()

  Get the current size of the PCB table.
- pid_t k_next_process (pid_t pid)

  Get the next active process PID after a given PID.

Variables

- pcb_table_t k_pcb_table

  Global PCB table instance.

4.30.1 Detailed Description

Implementation of kernel-level PCB table management.

This file implements the PCB table operations for PennOS, including PID allocation, release, and circular doubly-linked list operations for managing parent-child sibling relationships.

4.30.2 Function Documentation

4.30.2.1 k_next_process()

```
pid_t k_next_process (
            pid_t pid )
```

Get the next active process PID after a given PID.

Iterates through the PCB table to find the next non-WAITED process. Used for implementing ps command.

Parameters

pid	The starting PID (exclusive).

Returns

Next active PID, or 0 if none found.

4.30.2.2 k_pcb_claim()

```
pid_t k_pcb_claim ( )
```

Claim a new PID from the PCB table.

Finds the first available PCB slot or expands the vector if none available.

Returns

The newly claimed PID.

4.30.2.3 k_pcb_data()

```
pcb_t* k_pcb_data (
            pid_t pid )
```

Get direct access to a process's PCB.

Obtain the PCB data associated with a given pid. There should be no interrupts while holding this pcb data else it may become stale or cause a race condition.

Returns a pointer to the PCB for direct manipulation.

Parameters

pid	The process ID to look up.

Precondition

Interrupts are disabled

Returns

Pointer to the PCB structure.

4.30.2.4 k_pcb_init()

```
int k_pcb_init ( )
```

Initialize the PCB table.

Creates the vector with DEFAULT_CAPACITY and reserves PID 0 for the main process. Sets avail_process to 1.

Returns

0 on success.

4.30.2.5 k_pcb_list_add()

```
void k_pcb_list_add (
            pid_t * head,
            pid_t to_add )
```

Initialize the PCB table.

Allocates the PCB vector with default capacity and reserves PID 0 for the main/init process.

Returns

0 on success, -1 on failure.

4.30.2.6 k_pcb_list_insert()

```
void k_pcb_list_insert (
            pid_t * head,
            pid_t to_insert )
```

Insert an entire circular list into another.

Merges two circular lists by splicing.

Parameters

in,out	*head*	Pointer to destination list head.
	to_insert	Head of the source list to merge in.

4.30.2.7 k_pcb_list_remove()

```
void k_pcb_list_remove (
            pid_t * head,
            pid_t to_remove )
```

Remove a PID from a circular sibling list.

Removes the process from its circular doubly-linked list and resets its sibling pointers to self.

Parameters

in,out	*head*	Pointer to the list head (updated if removing head).
	to_remove	The PID to remove from the list.

4.30.2.8 k_pcb_table_size()

```
int k_pcb_table_size ( )
```

Get the current size of the PCB table.

Returns

Number of entries (including WAITED) in the PCB table.

4.30.2.9 k_proc_release()

```
void k_proc_release (
            pid_t pid )
```

Release a PID for reuse.

Updates avail_process hint if this PID is lower than current hint.

Parameters

pid	The PID to release.

4.31 src/process/pcb/k_pcb.h File Reference

Kernel-level PCB table management interface.

`#include "pcb_t.h"`

Include dependency graph for k_pcb.h: This graph shows which files directly or indirectly include this file:

Functions

- void k_pcb_list_add (pid_t *head, pid_t to_add)

  Initialize the PCB table.
- void k_pcb_list_remove (pid_t *head, pid_t to_remove)

  Remove a PID from a circular sibling list.
- void k_pcb_list_insert (pid_t *head, pid_t to_insert)

  Insert an entire circular list into another.
- void k_proc_release (pid_t pid)

  Release a PID for reuse.
- pid_t k_pcb_claim ()

  Claim a new PID from the PCB table.
- int k_pcb_init ()

  Initialize the PCB table.
- void k_pcb_clean ()

  Free the PCB table when exiting.
- pcb_t * k_pcb_data (pid_t pid)

  Get direct access to a process's PCB.
- int k_pcb_table_size ()

  Get the current size of the PCB table.
- pid_t k_next_process (pid_t pid)

  Get the next active process PID after a given PID.

4.31.1 Detailed Description

Kernel-level PCB table management interface.

This header provides the kernel-level interface for managing the PCB table, including initialization, PID allocation/release, and circular sibling list operations for parent-child relationships.

4.31.2 Function Documentation

4.31.2.1 k_next_process()

```
pid_t k_next_process (
            pid_t pid )
```

Get the next active process PID after a given PID.

Iterates through the PCB table to find the next non-WAITED process. Used for implementing ps command.

Parameters

pid	The starting PID (exclusive).

Returns

Next active PID, or 0 if none found.

4.31.2.2 k_pcb_claim()

```
pid_t k_pcb_claim ( )
```

Claim a new PID from the PCB table.

Finds the first available PCB slot or expands the vector if none available.

Returns

The newly claimed PID.

4.31.2.3 k_pcb_data()

```
pcb_t* k_pcb_data (
            pid_t pid )
```

Get direct access to a process's PCB.

Returns a pointer to the PCB for direct manipulation.

Parameters

pid	The process ID to look up.

Precondition

Interrupts are disabled

Returns

Pointer to the PCB structure.

Obtain the PCB data associated with a given pid. There should be no interrupts while holding this pcb data else it may become stale or cause a race condition.

Returns a pointer to the PCB for direct manipulation.

Parameters

pid	The process ID to look up.

Precondition

Interrupts are disabled

Returns

Pointer to the PCB structure.

4.31.2.4 k_pcb_init()

```
int k_pcb_init ( )
```

Initialize the PCB table.

Creates the vector with DEFAULT_CAPACITY and reserves PID 0 for the main process. Sets avail_process to 1.

Returns

0 on success.

4.31.2.5 k_pcb_list_add()

```
void k_pcb_list_add (
            pid_t * head,
            pid_t to_add )
```

Initialize the PCB table.

Allocates the PCB vector with default capacity and reserves PID 0 for the main/init process.

Returns

0 on success, -1 on failure.

4.31.2.6 k_pcb_list_insert()

```
void k_pcb_list_insert (
            pid_t * head,
            pid_t to_insert )
```

Insert an entire circular list into another.

Merges two circular lists by splicing.

Parameters

in,out	*head*	Pointer to destination list head.
	to_insert	Head of the source list to merge in.

4.31.2.7 k_pcb_list_remove()

```
void k_pcb_list_remove (
            pid_t * head,
            pid_t to_remove )
```

Remove a PID from a circular sibling list.

Removes the process from its circular doubly-linked list and resets its sibling pointers to self.

Parameters

in,out	*head*	Pointer to the list head (updated if removing head).
	to_remove	The PID to remove from the list.

4.31.2.8 k_pcb_table_size()

```
int k_pcb_table_size ( )
```

Get the current size of the PCB table.

Returns

Number of entries (including WAITED) in the PCB table.

4.31.2.9 k_proc_release()

```
void k_proc_release (
            pid_t pid )
```

Release a PID for reuse.

Updates avail_process hint if this PID is lower than current hint.

Parameters

pid	The PID to release.

4.32 src/process/pcb/pcb_state_t.h File Reference

Process state enumeration for PCB management.

This graph shows which files directly or indirectly include this file:

Enumerations

- enum pcb_state_t {
  ACTIVE , **BLOCKED** , **STOPPED** , **ZOMBIE** ,
  WAITED }

  Enumeration of process states.

4.32.1 Detailed Description

Process state enumeration for PCB management.

This header defines the possible states a process can be in during its lifecycle in PennOS.

4.32.2 Enumeration Type Documentation

4.32.2.1 pcb_state_t

`enum pcb_state_t`

Enumeration of process states.

- **ACTIVE**: Process is ready to run or currently running.
- **BLOCKED**: Process is waiting for an event (sleep, waitpid, I/O).
- **STOPPED**: Process has been stopped by a signal (S_SIGSTOP).
- **ZOMBIE**: Process has terminated but not yet reaped by parent.
- **WAITED**: Process has been reaped; PCB slot is available for reuse.

4.33 src/process/pcb/pcb_t.h File Reference

Process Control Block (PCB) structure definition.

```
#include "pcb_state_t.h"
#include "util/spthread.h"
#include "util/vector.h"
```

Include dependency graph for pcb_t.h: This graph shows which files directly or indirectly include this file:

Classes

- struct pcb_t

  Process Control Block structure.

Macros

- #define DEFAULT_SCHEDULE_PRIORITY 1

  Default scheduling priority for new processes.
- #define MAX_PROCESS_NAME 256

  Maximum length of process name string.

4.33.1 Detailed Description

Process Control Block (PCB) structure definition.

This header defines the PCB structure that contains all per-process state information in PennOS, including thread handle, parent/child relationships, file descriptors, scheduling info, and signal state.

4.34 src/process/pcb/pcb_table_t.h File Reference

PCB table structure for managing all processes.

```
#include "pcb_t.h"
#include "util/vector.h"
```

Include dependency graph for pcb_table_t.h: This graph shows which files directly or indirectly include this file:

Classes

- struct pcb_table_t

  Global PCB table structure.

Macros

- #define DEFAULT_CAPACITY 100

  Initial capacity for the PCB table vector.
- #define MAIN_PROC 0

  Reserved PID for the main/init process.

4.34.1 Detailed Description

PCB table structure for managing all processes.

This header defines the global PCB table structure that holds all process control blocks in the system. The table uses a vector for dynamic growth and tracks the next available PID slot.

4.35 src/process/pcb/s_pcb.c File Reference

Implementation of system-level PCB operations.

```
#include "s_pcb.h"
#include "process/pcb/k_pcb.h"
#include "process/pstd.h"
#include "process/schedule/k_schedule.h"
#include <string.h>
#include "util/log.h"
```

Include dependency graph for s_pcb.c:

Functions

- int s_nice (pid_t pid, int priority)

  Set the priority of the specified thread.
- int s_ps (process_info_t *info, int max_processes)

  Get information about all active processes.
- pcb_state_t s_proc_state (pid_t pid)

  Get the current state of a process.

4.35.1 Detailed Description

Implementation of system-level PCB operations.

This file implements the system calls for process information and priority management: s_nice, s_ps, and s_proc↩_state.

4.35.2 Function Documentation

4.35.2.1 s_nice()

```
int s_nice (
            pid_t pid,
            int priority )
```

Set the priority of the specified thread.

Parameters

pid	Process ID of the target thread.
priority	The new priorty value of the thread (0, 1, or 2)

Returns

0 on success, -1 on failure.

4.35.2.2 s_proc_state()

```
pcb_state_t s_proc_state (
            pid_t pid )
```

Get the current state of a process.

Parameters

pid	Process ID to query.

Returns

The process state (ACTIVE, BLOCKED, STOPPED, ZOMBIE, or WAITED).

4.35.2.3 s_ps()

```
int s_ps (
            process_info_t * info,
            int max_processes )
```

Get information about all active processes.

Fills the provided array with process information for all non-WAITED processes in the system.

Parameters

out	*info*	Array to fill with process information.
	max_processes	Maximum number of entries to fill.

Returns

Number of processes written to info array.

4.36 src/process/pcb/s_pcb.h File Reference

System-level PCB information interface.

```
#include "process/pcb/pcb_state_t.h"
#include "process/pstd.h"
```

Include dependency graph for s_pcb.h: This graph shows which files directly or indirectly include this file:

Classes

- struct process_info_t

  Process information structure for s_ps.

Macros

- #define MAX_PROCESS_NAME_PS 256

  Maximum process name length for s_ps output.

Functions

- int s_nice (pid_t pid, int priority)

  Set the priority of the specified thread.
- int s_ps (process_info_t *info, int max_processes)

  Get information about all active processes.
- pcb_state_t s_proc_state (pid_t pid)

  Get the current state of a process.

4.36.1 Detailed Description

System-level PCB information interface.

This header provides the system call interface for querying and modifying process information, including process listing (ps) and priority adjustment (nice).

4.36.2 Function Documentation

4.36.2.1 s_nice()

```
int s_nice (
            pid_t pid,
            int priority )
```

Set the priority of the specified thread.

Parameters

pid	Process ID of the target thread.
priority	The new priorty value of the thread (0, 1, or 2)

Returns

0 on success, -1 on failure.

4.36.2.2 s_proc_state()

```
pcb_state_t s_proc_state (
            pid_t pid )
```

Get the current state of a process.

Parameters

pid	Process ID to query.

Returns

The process state (ACTIVE, BLOCKED, STOPPED, ZOMBIE, or WAITED).

4.36.2.3 s_ps()

```
int s_ps (
            process_info_t * info,
            int max_processes )
```

Get information about all active processes.

Fills the provided array with process information for all non-WAITED processes in the system.

Parameters

out	*info*	Array to fill with process information.
	max_processes	Maximum number of entries to fill.

Returns

Number of processes written to info array.

4.37 src/process/pstd.h File Reference

Process standard definitions and constants.

This graph shows which files directly or indirectly include this file:

Macros

- #define **SIGINT** 2
- #define **SIGTERM** 15
- #define **SIGCONT** 18
- #define **SIGSTOP** 19
- #define **W_EXITED** 1 /∗ Child exited normally (returned from function) ∗/
- #define **W_SIGNALED** 2 /∗ Child was terminated by a signal (e.g., S_SIGTERM) ∗/
- #define **W_STOPPED** 4 /∗ Child was stopped by a signal (e.g., S_SIGSTOP) ∗/
- #define **W_CONTINUED** 8 /∗ Child was continued after being stopped ∗/
- #define **W_WIFEXITED**(status) ((status) == W_EXITED)
- #define **W_WIFSIGNALED**(status) ((status) == W_SIGNALED)
- #define **W_WIFSTOPPED**(status) ((status) == W_STOPPED)
- #define **W_WIFCONTINUED**(status) ((status) == W_CONTINUED)

Typedefs

- typedef int **pid_t**

4.37.1 Detailed Description

Process standard definitions and constants.

This header defines constants used throughout PennOS for process management related to signaling and waitpid.

4.38 src/process/schedule/k_schedule.c File Reference

Implementation of the PennOS priority scheduler.

```
#include "k_schedule.h"
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/time.h>
#include "process/pcb/k_pcb.h"
#include "process/signal/s_signal.h"
#include "util/log.h"
```

Include dependency graph for k_schedule.c:

Functions

- int k_sched_init (void)

  Initialize the scheduler.
- int k_sched_add (pid_t pid)

  Add a process to the appropriate queue.
- void k_sched_remove (pid_t pid)

  Remove a PID from all scheduler queues.
- void k_remove_int (pid_t pid)

  Remove PID from pending SIGINT queue.
- void k_remove_stop (pid_t pid)

  Remove PID from pending SIGSTOP queue.
- void k_sched_block (pid_t pid)

  Block a process.
- void k_sched_unblock (pid_t pid)

  Unblock a process.
- void k_sched_run (void)

  Main scheduler loop.
- void k_sched_stop (void)

  Stop the scheduler loop.
- pid_t k_sched_current_pid (void)

  Get current process PID.
- int k_sched_clean (void)

  Free all scheduler queue resources.
- void k_disable_interrupts ()

  Disable interrupts (nested). Increments counter; disables spthread interrupts on 0->1 transition.
- void k_enable_interrupts ()

  Enable interrupts (nested). Decrements counter; enables spthread interrupts on 1->0 transition.
- bool k_can_interrupt ()

  Check if interrupts are enabled.

4.38.1 Detailed Description

Implementation of the PennOS priority scheduler.

This file implements a multi-level priority scheduler with three queues (high, medium, low priority). Processes are scheduled using weighted round-robin with approximately 1.5x time ratio between priority levels.

Key features:

- Three priority queues (pq0=high, pq1=medium, pq2=low)
- Weighted scheduling: pq0 gets 9 credits, pq1 gets 6, pq2 gets 4
- 100ms clock tick using SIGALRM
- Blocked queue for sleeping/waiting processes
- Interrupt disable counting for nested critical sections
- SIGINT/SIGTSTP handling for foreground process signaling

4.38.2 Function Documentation

4.38.2.1 k_can_interrupt()

```
bool k_can_interrupt ( )
```

Check if interrupts are enabled.

Returns

true if process running and counter is 0.

4.38.2.2 k_disable_interrupts()

```
void k_disable_interrupts ( )
```

Disable interrupts (nested). Increments counter; disables spthread interrupts on 0->1 transition.

Disable interrupts (nested).

4.38.2.3 k_enable_interrupts()

```
void k_enable_interrupts ( )
```

Enable interrupts (nested). Decrements counter; enables spthread interrupts on 1->0 transition.

Enable interrupts (nested).

4.38.2.4 k_remove_int()

```
void k_remove_int (
            pid_t pid )
```

Remove PID from pending SIGINT queue.

Parameters

pid	Process ID to remove.

4.38.2.5 k_remove_stop()

```
void k_remove_stop (
            pid_t pid )
```

Remove PID from pending SIGSTOP queue.

Parameters

pid	Process ID to remove.

4.38.2.6 k_sched_add()

```
int k_sched_add (
            pid_t pid )
```

Add a process to the appropriate queue.

Add a process to the scheduler. This can happen regardless of whether the scheduler is running.

Routes to blocked_queue if state is BLOCKED, otherwise to priority queue based on sched_priority (0, 1, or 2).

Parameters

pid	Process ID to add.

Returns

0 on success.

4.38.2.7 k_sched_block()

```
void k_sched_block (
            pid_t pid )
```

Block a process.

Block a process (remove from active queues, add to blocked queue).

Removes from active queues, sets state to BLOCKED, adds to blocked_queue.

Parameters

pid	Process ID to block.

4.38.2.8 k_sched_clean()

```
int k_sched_clean (
            void  )
```

Free all scheduler queue resources.

Clean up the scheduler, for when the program is shutting down.

Returns

0 on success.

4.38.2.9 k_sched_current_pid()

```
pid_t k_sched_current_pid (
            void  )
```

Get current process PID.

Get the currently running process PID.

Returns

Current PID or -1.

4.38.2.10 k_sched_init()

```
int k_sched_init (
            void  )
```

Initialize the scheduler.

Perform necessary initializations for a scheduler.

Allocates all queue vectors, sets up signal handlers for SIGALRM, SIGINT, and SIGTSTP, and unblocks these signals.

Returns

0 on success, -1 if allocation fails.

4.38.2.11 k_sched_remove()

```
void k_sched_remove (
            pid_t pid )
```

Remove a PID from all scheduler queues.

Remove a process from all scheduler queues.

Removes from pq0, pq1, pq2, and blocked_queue.

Parameters

pid	Process ID to remove.

4.38.2.12 k_sched_run()

```
void k_sched_run (
            void  )
```

Main scheduler loop.

Start the scheduler.

4.38.2.13 k_sched_stop()

```
void k_sched_stop (
            void  )
```

Stop the scheduler loop.

Stop the scheduler.

4.38.2.14 k_sched_unblock()

```
void k_sched_unblock (
            pid_t pid )
```

Unblock a process.

Unblock a process (remove from blocked queue, add back to active queue).

Removes from blocked_queue, sets state to ACTIVE, re-adds to priority queue.

Parameters

pid	Process ID to unblock.

4.39 src/process/schedule/k_schedule.h File Reference

Kernel-level scheduler interface for PennOS.

```
#include "io/term/s_term.h"
#include "process/pcb/pcb_t.h"
```

Include dependency graph for k_schedule.h: This graph shows which files directly or indirectly include this file:

Macros

- #define ASSERT_NO_INTERRUPT(func_name) (void)0

  Enable debug mode for scheduler assertions.

Functions

- int k_sched_init (void)

  Perform necessary initializations for a scheduler.
- int k_sched_add (pid_t pid)

  Add a process to the scheduler. This can happen regardless of whether the scheduler is running.
- void k_sched_remove (pid_t pid)

  Remove a process from all scheduler queues.
- void k_remove_int (pid_t pid)

  Remove PID from pending SIGINT queue.
- void k_remove_stop (pid_t pid)

  Remove PID from pending SIGSTOP queue.
- void k_sched_block (pid_t pid)

  Block a process (remove from active queues, add to blocked queue).
- void k_sched_unblock (pid_t pid)

  Unblock a process (remove from blocked queue, add back to active queue).
- void k_sched_run (void)

  Start the scheduler.
- void k_sched_stop (void)

  Stop the scheduler.
- pid_t k_sched_current_pid (void)

  Get the currently running process PID.
- int k_sched_clean (void)

  Clean up the scheduler, for when the program is shutting down.
- void k_disable_interrupts ()

  Disable interrupts (nested).
- void k_enable_interrupts ()

  Enable interrupts (nested).
- bool k_can_interrupt ()

  Check if interrupts are enabled.

4.39.1 Detailed Description

Kernel-level scheduler interface for PennOS.

This header provides the kernel-level interface for the PennOS priority scheduler. It supports three priority levels with weighted round-robin scheduling, process blocking/unblocking, and interrupt management for critical sections.

Note

The scheduler uses SIGALRM for 100ms clock ticks and handles SIGINT/SIGTSTP for terminal foreground process signaling.

4.39.2 Function Documentation

4.39.2.1 k_can_interrupt()

```
bool k_can_interrupt ( )
```

Check if interrupts are enabled.

Returns true if a process is running and interrupt counter is 0.

Returns

true if interrupts enabled, false otherwise.

true if process running and counter is 0.

4.39.2.2 k_disable_interrupts()

```
void k_disable_interrupts ( )
```

Disable interrupts (nested).

Increments counter; disables spthread interrupts on 0->1 transition. Supports nested calls for critical sections.

Disable interrupts (nested).

4.39.2.3 k_enable_interrupts()

```
void k_enable_interrupts ( )
```

Enable interrupts (nested).

Decrements counter; enables spthread interrupts on 1->0 transition. Must match number of k_disable_interrupts() calls.

Enable interrupts (nested).

4.39.2.4 k_remove_int()

```
void k_remove_int (
            pid_t pid )
```

Remove PID from pending SIGINT queue.

Parameters

pid	Process ID to remove.

4.39.2.5 k_remove_stop()

```
void k_remove_stop (
            pid_t pid )
```

Remove PID from pending SIGSTOP queue.

Parameters

pid	Process ID to remove.

4.39.2.6 k_sched_add()

```
int k_sched_add (
            pid_t pid )
```

Add a process to the scheduler. This can happen regardless of whether the scheduler is running.

Returns

0 on success, -1 on error.

Add a process to the scheduler. This can happen regardless of whether the scheduler is running.

Routes to blocked_queue if state is BLOCKED, otherwise to priority queue based on sched_priority (0, 1, or 2).

Parameters

pid	Process ID to add.

Returns

0 on success.

4.39.2.7 k_sched_block()

```
void k_sched_block (
            pid_t pid )
```

Block a process (remove from active queues, add to blocked queue).

Parameters

pid	Process ID to block

Block a process (remove from active queues, add to blocked queue).

Removes from active queues, sets state to BLOCKED, adds to blocked_queue.

Parameters

pid	Process ID to block.

4.39.2.8 k_sched_clean()

```
int k_sched_clean (
            void  )
```

Clean up the scheduler, for when the program is shutting down.

Returns

0 on success, -1 on error.

Clean up the scheduler, for when the program is shutting down.

Returns

0 on success.

4.39.2.9 k_sched_current_pid()

```
pid_t k_sched_current_pid (
            void  )
```

Get the currently running process PID.

Returns

Current PID, or -1 if none

Get the currently running process PID.

Returns

Current PID or -1.

4.39.2.10 k_sched_init()

```
int k_sched_init (
            void  )
```

Perform necessary initializations for a scheduler.

Returns

0 on success, -1 on error.

Perform necessary initializations for a scheduler.

Allocates all queue vectors, sets up signal handlers for SIGALRM, SIGINT, and SIGTSTP, and unblocks these signals.

Returns

0 on success, -1 if allocation fails.

4.39.2.11 k_sched_remove()

```
void k_sched_remove (
            pid_t pid )
```

Remove a process from all scheduler queues.

Parameters

pid	Process ID to remove

Remove a process from all scheduler queues.

Removes from pq0, pq1, pq2, and blocked_queue.

Parameters

pid	Process ID to remove.

4.39.2.12 k_sched_run()

```
void k_sched_run (
            void  )
```

Start the scheduler.

Begins the main scheduling loop with 100ms timer ticks. Implements weighted priority scheduling (9:6:4 ratio) and handles signal delivery.

Start the scheduler.

4.39.2.13 k_sched_stop()

```
void k_sched_stop (
            void  )
```

Stop the scheduler.

Stop the scheduler.

4.39.2.14 k_sched_unblock()

```
void k_sched_unblock (
            pid_t pid )
```

Unblock a process (remove from blocked queue, add back to active queue).

Parameters

pid	Process ID to unblock

Unblock a process (remove from blocked queue, add back to active queue).

Removes from blocked_queue, sets state to ACTIVE, re-adds to priority queue.

Parameters

pid	Process ID to unblock.

4.40 src/process/schedule/s_schedule.c File Reference

Implementation of system-level scheduling functions.

```
#include "s_schedule.h"
#include "process/pcb/k_pcb.h"
#include "process/pcb/pcb_t.h"
#include "process/schedule/k_schedule.h"
#include "util/log.h"
```

Include dependency graph for s_schedule.c:

Functions

- void s_sleep (unsigned int ticks)

  Suspends execution of the calling proces for a specified number of clock ticks.
- pid_t s_getpid (void)

  Get the current process ID.

4.40.1 Detailed Description

Implementation of system-level scheduling functions.

This file implements the user-facing system calls for scheduling: s_getpid for obtaining the current process ID and s_sleep for suspending execution for a specified duration.

4.40.2 Function Documentation

4.40.2.1 s_getpid()

```
pid_t s_getpid (
            void  )
```

Get the current process ID.

Returns the current process ID.

Returns

The current process ID.

4.40.2.2 s_sleep()

```
void s_sleep (
            unsigned int ticks )
```

Suspends execution of the calling proces for a specified number of clock ticks.

This function is analogous to `sleep(3)` in Linux, with the behavior that the system clock continues to tick even if the call is interrupted.

Parameters

ticks	Duration of the sleep in system clock ticks. Must be greater than 0.

4.41 src/process/schedule/s_schedule.h File Reference

System-level scheduling interface for user processes.

`#include "process/pstd.h"`
Include dependency graph for s_schedule.h: This graph shows which files directly or indirectly include this file:

Functions

- void s_sleep (unsigned int ticks)

  Suspends execution of the calling proces for a specified number of clock ticks.
- pid_t s_getpid (void)

  Returns the current process ID.

4.41.1 Detailed Description

System-level scheduling interface for user processes.

This header provides the system call interface for scheduling-related operations available to user processes, including getting the current PID and sleeping for a specified duration.

4.41.2 Function Documentation

4.41.2.1 s_getpid()

```
pid_t s_getpid (
            void  )
```

Returns the current process ID.

Returns the current process ID.

Returns

The current process ID.

4.41.2.2 s_sleep()

```
void s_sleep (
            unsigned int ticks )
```

Suspends execution of the calling proces for a specified number of clock ticks.

This function is analogous to `sleep(3)` in Linux, with the behavior that the system clock continues to tick even if the call is interrupted.

Parameters

ticks	Duration of the sleep in system clock ticks. Must be greater than 0.

4.42 src/process/signal/k_signal.c File Reference

Implementation of kernel-level signal handling.

```
#include "k_signal.h"
#include "process/lifecycle/k_lifecycle.h"
#include "process/pcb/k_pcb.h"
#include "process/schedule/k_schedule.h"
```

Include dependency graph for k_signal.c:

Functions

- void k_notify_parent (pid_t child, int wstatus)

  Notify a parent process of a child's state change.
- pid_t k_pollpid (pid_t parent, pid_t target, int *wstatus)

  Poll for a zombie child process.

4.42.1 Detailed Description

Implementation of kernel-level signal handling.

This file implements the kernel-level signal infrastructure for PennOS, including the mechanism for parent-child notification on state changes and polling for zombie children during waitpid operations.

4.42.2 Function Documentation

4.42.2.1 k_notify_parent()

```
void k_notify_parent (
            pid_t child,
            int wstatus )
```

Notify a parent process of a child's state change.

Parameters

child	PID of the child that changed state.
wstatus	Status value (W_EXITED, W_SIGNALED, W_STOPPED, W_CONTINUED).

Precondition

Interrupts are disabled

4.42.2.2 k_pollpid()

```
pid_t k_pollpid (
            pid_t parent,
            pid_t target,
            int * wstatus )
```

Poll for a zombie child process.

Poll for a child process that has changed state.

Parameters

parent	PID of the parent process.
target	PID of specific child or -1 for any.
wstatus	Pointer to store exit status, can be NULL.

Precondition

Interrupts are disabled

Returns

Child PID if reaped, 0 if no zombie, -1 on error.

4.43 src/process/signal/k_signal.h File Reference

Kernel-level signal handling interface for PennOS.

`#include "process/pstd.h"`
Include dependency graph for k_signal.h: This graph shows which files directly or indirectly include this file:

Functions

- pid_t k_pollpid (pid_t parent, pid_t target, int ∗wstatus)

  Poll for a child process that has changed state.
- void k_notify_parent (pid_t child, int wstatus)

  Notify a parent process of a child's state change.

4.43.1 Detailed Description

Kernel-level signal handling interface for PennOS.

This header provides kernel-level functions for signal-related operations, including polling for zombie children and notifying parent processes when a child's state changes.

4.43.2 Function Documentation

4.43.2.1 k_notify_parent()

```
void k_notify_parent (
            pid_t child,
            int wstatus )
```

Notify a parent process of a child's state change.

Called when a child process changes state (zombified, stopped, continued). If the parent is blocked waiting for this child (or any child), unblocks the parent and sets the waitpid return values.

Parameters

child	PID of the child that changed state.
wstatus	Status value to report (W_EXITED, W_SIGNALED, W_STOPPED, W_CONTINUED).
child	PID of the child that changed state.
wstatus	Status value (W_EXITED, W_SIGNALED, W_STOPPED, W_CONTINUED).

Precondition

Interrupts are disabled

4.43.2.2 k_pollpid()

```
pid_t k_pollpid (
            pid_t parent,
            pid_t target,
            int * wstatus )
```

Poll for a child process that has changed state.

Checks if the specified target child (or any child if target is -1) has become a zombie. If found, reaps the child and returns its PID.

Parameters

parent	PID of the parent process doing the poll.
target	PID of specific child to poll, or -1 for any child. A value of 0 is treated as -1.
wstatus	Pointer to store the exit status (W_EXITED or W_SIGNALED). Can be NULL if status is not needed.

Returns

Child PID if a zombie was reaped, 0 if no zombie found, -1 on error (invalid PID or not a child of parent).

Poll for a child process that has changed state.

Parameters

parent	PID of the parent process.
target	PID of specific child or -1 for any.
wstatus	Pointer to store exit status, can be NULL.

Precondition

Interrupts are disabled

Returns

Child PID if reaped, 0 if no zombie, -1 on error.

4.44 src/process/signal/s_signal.c File Reference

Implementation of system-level signal functions.

```
#include "s_signal.h"
#include "k_signal.h"
#include "process/lifecycle/k_lifecycle.h"
#include "process/lifecycle/s_lifecycle.h"
#include "process/pcb/k_pcb.h"
#include "process/pcb/pcb_t.h"
#include "process/pstd.h"
#include "process/schedule/k_schedule.h"
#include "util/log.h"
```

Include dependency graph for s_signal.c:

Functions

- int s_kill (pid_t pid, int signal)

  Send a signal to a process.
- void s_logout (void)

  Perform system logout and shutdown.
- bool s_can_sig_stop (pid_t pid)

  Check if a process accepts SIGSTOP.
- bool s_can_sig_int (pid_t pid)

  Check if a process accepts SIGINT.
- void s_ignore_int (pid_t pid, bool ignore)

  Set SIGINT ignore flag for a process.
- void s_ignore_stop (pid_t pid, bool ignore)

  Set SIGSTOP ignore flag for a process.

4.44.1 Detailed Description

Implementation of system-level signal functions.

This file implements the user-facing signal system calls for PennOS, including s_kill for sending signals, s_logout for system shutdown, and functions to manage signal ignore masks for SIGINT and SIGSTOP.

4.44.2 Function Documentation

4.44.2.1 s_can_sig_int()

```
bool s_can_sig_int (
            pid_t pid )
```

Check if a process accepts SIGINT.

Check if a process can receive SIGINT.

Thread-safe check of the process's ign_int flag.

Parameters

pid	Process ID to check.

Returns

true if SIGINT will be handled, false if ignored.

4.44.2.2 s_can_sig_stop()

```
bool s_can_sig_stop (
            pid_t pid )
```

Check if a process accepts SIGSTOP.

Check if a process can receive SIGSTOP.

Thread-safe check of the process's ign_stop flag.

Parameters

pid	Process ID to check.

Returns

true if SIGSTOP will be handled, false if ignored.

4.44.2.3 s_ignore_int()

```
void s_ignore_int (
            pid_t pid,
            bool ignore )
```

Set SIGINT ignore flag for a process.

Set whether a process ignores SIGINT signals.

Thread-safe modification of the process's ign_int flag.

Parameters

pid	Process ID to modify.
ignore	true to ignore SIGINT, false to handle.

4.44.2.4 s_ignore_stop()

```
void s_ignore_stop (
            pid_t pid,
            bool ignore )
```

Set SIGSTOP ignore flag for a process.

Set whether a process ignores SIGSTOP signals.

Thread-safe modification of the process's ign_stop flag.

Parameters

pid	Process ID to modify.
ignore	true to ignore SIGSTOP, false to handle.

4.44.2.5 s_kill()

```
int s_kill (
            pid_t pid,
            int signal )
```

Send a signal to a process.

Send a signal to a particular process.

Parameters

pid	Target process ID.
signal	Signal number (SIGSTOP, SIGCONT, SIGINT, SIGTERM).

Returns

0 on success, -1 on error.

4.44.2.6 s_logout()

```
void s_logout (
            void  )
```

Perform system logout and shutdown.

Logout and shutdown PennOS.

4.45 src/process/signal/s_signal.h File Reference

System-level signal interface for user processes.

`#include "process/pstd.h"`
Include dependency graph for s_signal.h: This graph shows which files directly or indirectly include this file:

Functions

- int s_kill (pid_t pid, int signal)

  Send a signal to a particular process.
- void s_logout (void)

  Logout and shutdown PennOS.
- bool s_can_sig_stop (pid_t pid)

  Check if a process can receive SIGSTOP.
- bool s_can_sig_int (pid_t pid)

  Check if a process can receive SIGINT.
- void s_ignore_stop (pid_t pid, bool ignore)

  Set whether a process ignores SIGSTOP signals.
- void s_ignore_int (pid_t pid, bool ignore)

  Set whether a process ignores SIGINT signals.

4.45.1 Detailed Description

System-level signal interface for user processes.

This header provides the system call interface for signal operations in PennOS, including sending signals to processes, system logout, and signal ignore/unignore functionality.

4.45.2 Function Documentation

4.45.2.1 s_can_sig_int()

```
bool s_can_sig_int (
            pid_t pid )
```

Check if a process can receive SIGINT.

Parameters

pid	Process ID to check.

Returns

true if the process will handle SIGINT, false if ignoring.

Check if a process can receive SIGINT.

Thread-safe check of the process's ign_int flag.

Parameters

pid	Process ID to check.

Returns

true if SIGINT will be handled, false if ignored.

4.45.2.2 s_can_sig_stop()

```
bool s_can_sig_stop (
            pid_t pid )
```

Check if a process can receive SIGSTOP.

Parameters

pid	Process ID to check.

Returns

true if the process will handle SIGSTOP, false if ignoring.

Check if a process can receive SIGSTOP.

Thread-safe check of the process's ign_stop flag.

Parameters

pid	Process ID to check.

Returns

true if SIGSTOP will be handled, false if ignored.

4.45.2.3 s_ignore_int()

```
void s_ignore_int (
            pid_t pid,
            bool ignore )
```

Set whether a process ignores SIGINT signals.

Used by background processes to prevent Ctrl-C from interrupting them.

Parameters

pid	Process ID to modify.
ignore	true to ignore SIGINT, false to handle it.

Set whether a process ignores SIGINT signals.

Thread-safe modification of the process's ign_int flag.

Parameters

pid	Process ID to modify.
ignore	true to ignore SIGINT, false to handle.

4.45.2.4 s_ignore_stop()

```
void s_ignore_stop (
            pid_t pid,
            bool ignore )
```

Set whether a process ignores SIGSTOP signals.

Used by background processes to prevent Ctrl-Z from stopping them.

Parameters

pid	Process ID to modify.
ignore	true to ignore SIGSTOP, false to handle it.

Set whether a process ignores SIGSTOP signals.

Thread-safe modification of the process's ign_stop flag.

Parameters

pid	Process ID to modify.
ignore	true to ignore SIGSTOP, false to handle.

4.45.2.5 s_kill()

```
int s_kill (
            pid_t pid,
            int signal )
```

Send a signal to a particular process.

Parameters

pid	Process ID of the target proces.
signal	Signal number to be sent.

Returns

0 on success, -1 on error.

Send a signal to a particular process.

Parameters

pid	Target process ID.
signal	Signal number (SIGSTOP, SIGCONT, SIGINT, SIGTERM).

Returns

0 on success, -1 on error.

4.45.2.6 s_logout()

```
void s_logout (
            void )
```

Logout and shutdown PennOS.

This function kills all processes except the current one (using SIGTERM), waits for them to be reaped, then exits the current process.

Logout and shutdown PennOS.

4.46 src/shell/commands.c File Reference

Implementation of the PennOS shell command dispatcher.

```
#include "commands.h"
#include <string.h>
#include "io/term/s_term.h"
#include "process/lifecycle/s_lifecycle.h"
#include "process/pcb/s_pcb.h"
#include "process/schedule/s_schedule.h"
#include "demo/u_demo.h"
#include "file/u_file.h"
#include "job/u_job.h"
#include "process/u_process.h"
#include "u_shell.h"
```

Include dependency graph for commands.c:

4.47 src/shell/commands.h File Reference

Command dispatch interface for the PennOS shell.

```
#include "job/job_table_t.h"
#include "util/parser.h"
```

Include dependency graph for commands.h: This graph shows which files directly or indirectly include this file:

Functions

- void run_command (char **argv, struct parsed_command *cmd, int nice, job_table_t *job_table)

  Execute a shell command.
- void u_man ()

  Display the manual/help page.
- void * u_echo (void *arg)

  Echo arguments to standard output.

4.47.1 Detailed Description

Command dispatch interface for the PennOS shell.

This header provides the main interface for executing shell commands, including the run_command dispatcher, manual page display, and echo.

4.47.2 Function Documentation

4.47.2.1 run_command()

```
void run_command (
            char ** argv,
            struct parsed_command * cmd,
            int nice,
            job_table_t * job_table )
```

Execute a shell command.

Main command dispatcher that handles built-in commands directly and spawns external commands as separate processes. Handles I/O redirection, background execution, and job table management.

Parameters

argv	NULL-terminated argument array (argv[0] = command name).
cmd	Parsed command structure with redirection and background flags.
nice	Priority level for the command (0-2), or -1 for default.
job_table	Pointer to the shell's job table for tracking background jobs.

Execute a shell command.

Parameters

argv	NULL-terminated argument array.
cmd	Parsed command with redirection/background flags.
nice	Priority (0-2) or -1 for default.
job_table	Pointer to shell's job table.

4.47.2.2 u_echo()

```
void* u_echo (
            void * arg )
```

Echo arguments to standard output.

Usage: echo [ARGS...] Prints all arguments separated by spaces, followed by a newline.

Parameters

arg	Pointer to NULL-terminated argv array.

Returns

Always NULL.

Echo arguments to standard output.

Prints argv[1..n] separated by spaces, ending with a newline.

Parameters

arg	Pointer to NULL-terminated argv array.

Returns

Always NULL.

4.47.2.3 u_man()

```
void u_man ( )
```

Display the manual/help page.

Usage: man Lists all available built-in and external commands.

Display the manual/help page.

Prints a categorized list of all shell commands to stdout.

4.48 src/shell/file/u_file.h File Reference

User-level file manipulation commands for the PennOS shell.

This graph shows which files directly or indirectly include this file:

Functions

- void ∗ u_cat (void ∗arg)

  Concatenate and display file contents.
- void ∗ u_touch (void ∗arg)

  Create files or update timestamps.
- void ∗ u_mv (void ∗arg)

  Move or rename a file.
- void ∗ u_cp (void ∗arg)

  Copy a file with optional host filesystem support.
- void ∗ u_rm (void ∗arg)

  Remove files from the filesystem.
- void ∗ u_ls (void ∗arg)

  List directory contents.
- void ∗ u_chmod (void ∗arg)

  Change file permissions.

4.48.1 Detailed Description

User-level file manipulation commands for the PennOS shell.

This header declares the built-in shell commands for file operations. Each function follows the spthread entry point signature (void∗ -> void∗) and expects argv-style arguments passed via the arg parameter.

4.48.2 Function Documentation

4.48.2.1 u_cat()

```
void* u_cat (
            void * arg )
```

Concatenate and display file contents.

Parameters

arg	Pointer to NULL-terminated argv array.

Returns

Always NULL.

4.48.2.2 u_chmod()

```
void* u_chmod (
            void * arg )
```

Change file permissions.

Parameters

arg	Pointer to NULL-terminated argv array.

Returns

Always NULL.

4.48.2.3 u_cp()

```
void* u_cp (
            void * arg )
```

Copy a file with optional host filesystem support.

Parameters

arg	Pointer to NULL-terminated argv array.

Returns

Always NULL.

4.48.2.4 u_ls()

```
void* u_ls (
            void * arg )
```

List directory contents.

Calls s_ls to display file information.

Parameters

arg	Pointer to NULL-terminated argv array.

Returns

Always NULL.

4.48.2.5 u_mv()

```
void* u_mv (
            void * arg )
```

Move or rename a file.

Parameters

arg	Pointer to NULL-terminated argv array.

Returns

Always NULL.

4.48.2.6 u_rm()

```
void* u_rm (
            void * arg )
```

Remove files from the filesystem.

Parameters

arg	Pointer to NULL-terminated argv array.

Returns

Always NULL.

4.48.2.7 u_touch()

```
void* u_touch (
            void * arg )
```

Create files or update timestamps.

Parameters

arg	Pointer to NULL-terminated argv array.

Returns

Always NULL.

4.49 src/shell/job/job_table.c File Reference

Implementation of job table management functions.

```
#include "io/term/s_term.h"
#include "job_table_t.h"
#include "util/vector.h"
```

Include dependency graph for job_table.c:

Functions

- job_table_t init_job_table ()

  Initialize a new job table.
- void free_job_table (job_table_t *job_table)

  Free a job table.
- int job_add (job_table_t *job_table, job_t job)

  Add a new job to the table.
- void job_remove (job_table_t *job_table, int job_id)

  Remove a job from the table.
- void job_touch (job_table_t *job_table, int job_id)

  Mark a job as most recently accessed.
- void print_job_status (job_table_t *job_table, int job_id, const char *status)

  Print a job's status line.

4.49.1 Detailed Description

Implementation of job table management functions.

This file implements the data structure operations for the shell's job table, including initialization, adding/removing jobs, and status display.

4.49.2 Function Documentation

4.49.2.1 free_job_table()

```
void free_job_table (
            job_table_t * job_table )
```

Free a job table.

Returns

Pointer to the job table.

4.49.2.2 init_job_table()

```
job_table_t init_job_table ( )
```

Initialize a new job table.

Creates a job table with initial capacity for 10 jobs.

Returns

Initialized job_table_t structure.

4.49.2.3 job_add()

```
int job_add (
            job_table_t * job_table,
            job_t job )
```

Add a new job to the table.

Assigns the current age to the job and finds the first empty slot (pid == -1) to reuse, or appends to the end.

Parameters

job_table	Pointer to the job table.
job	Job entry with pid and command set.

Returns

Job ID of the inserted job.

4.49.2.4 job_remove()

```
void job_remove (
            job_table_t * job_table,
            int job_id )
```

Remove a job from the table.

Sets pid to -1 to mark the slot as empty and frees the dynamically allocated command string.

Parameters

job_table	Pointer to the job table.
job_id	Job ID to remove.

4.49.2.5 job_touch()

```
void job_touch (
            job_table_t * job_table,
            int job_id )
```

Mark a job as most recently accessed.

Updates the job's age to the current counter value and sets latest_job to this job ID.

Parameters

job_table	Pointer to the job table.
job_id	Job ID to touch.

4.49.2.6 print_job_status()

```
void print_job_status (
            job_table_t * job_table,
            int job_id,
            const char * status )
```

Print a job's status line.

Parameters

job_table	Pointer to the job table.
job_id	Job ID to display.
status	Human-readable status string.

4.50 src/shell/job/job_table_t.h File Reference

Job table data structures and management functions.

`#include "util/vector.h"`
Include dependency graph for job_table_t.h: This graph shows which files directly or indirectly include this file:

Classes

- struct job_st

  Represents a single job entry in the job table.
- struct job_table_st

Typedefs

- typedef struct job_st job_t

  Represents a single job entry in the job table.
- typedef struct job_table_st **job_table_t**

Functions

- job_table_t init_job_table ()

  Initialize a new job table.
- void free_job_table (job_table_t *job_table)

  Free a job table.
- int job_add (job_table_t *job_table, job_t job)

  Add a new job to the table.
- void job_remove (job_table_t *job_table, int job_id)

  Remove a job from the table.
- void job_touch (job_table_t *job_table, int job_id)

  Mark a job as most recently accessed.
- void print_job_status (job_table_t *job_table, int job_id, const char *status)

  Print a job's status line.

4.50.1 Detailed Description

Job table data structures and management functions.

This header defines the data structures for tracking background jobs in the PennOS shell, along with functions for adding, removing, and displaying job entries.

4.50.2 Function Documentation

4.50.2.1 free_job_table()

```
void free_job_table (
            job_table_t * job_table )
```

Free a job table.

Returns

Pointer to the job table.

4.50.2.2 init_job_table()

```
job_table_t init_job_table ( )
```

Initialize a new job table.

Creates an empty job table with a placeholder at index 0 (job IDs start at 1).

Returns

Initialized job_table_t structure.

Creates a job table with initial capacity for 10 jobs.

Returns

Initialized job_table_t structure.

4.50.2.3 job_add()

```
int job_add (
            job_table_t * job_table,
            job_t job )
```

Add a new job to the table.

Assigns the current age to the job and finds the first empty slot (pid == -1) to reuse, or appends to the end.

Parameters

job_table	Pointer to the job table.
job	Job entry with pid and command set.

Returns

Job ID of the inserted job.

4.50.2.4 job_remove()

```
void job_remove (
            job_table_t * job_table,
            int job_id )
```

Remove a job from the table.

Sets pid to -1 to mark the slot as empty and frees the dynamically allocated command string.

Parameters

job_table	Pointer to the job table.
job_id	Job ID to remove.

4.50.2.5 job_touch()

```
void job_touch (
            job_table_t * job_table,
            int job_id )
```

Mark a job as most recently accessed.

Updates the job's age to the current counter value and sets latest_job to this job ID.

Parameters

job_table	Pointer to the job table.
job_id	Job ID to touch.

4.50.2.6 print_job_status()

```
void print_job_status (
            job_table_t * job_table,
```

```
int job_id,
const char * status )
```

Print a job's status line.

Parameters

job_table	Pointer to the job table.
job_id	Job ID to display.
status	Human-readable status string.

4.51 src/shell/job/u_job.h File Reference

User-level job control commands for the PennOS shell.

`#include "job_table_t.h"`
Include dependency graph for u_job.h: This graph shows which files directly or indirectly include this file:

Functions

- void u_bg (job_table_t ∗shell, char ∗∗argv)

  Resume a stopped job in the background.
- void u_fg (job_table_t ∗shell, char ∗∗argv)

  Bring a job to the foreground.
- void u_jobs (job_table_t ∗shell)

  List all current jobs.

4.51.1 Detailed Description

User-level job control commands for the PennOS shell.

This header declares the built-in shell commands for job control, allowing users to manage background and foreground processes.

4.51.2 Function Documentation

4.51.2.1 u_bg()

```
void u_bg (
            job_table_t * job_table,
            char ** argv )
```

Resume a stopped job in the background.

Usage: bg [JOB_ID] If JOB_ID is provided, resumes that specific job. Otherwise, resumes the most recently stopped job. Sends SIGCONT to the target process.

Parameters

job_table	Pointer to the shell's job table.
argv	NULL-terminated argument array (argv[1] = optional job ID).

If a job ID is provided, validates and uses that job. Otherwise, searches for the most recently stopped job.

Parameters

job_table	Pointer to the shell's job table.
argv	NULL-terminated argument array.

4.51.2.2 u_fg()

```
void u_fg (
            job_table_t * job_table,
            char ** argv )
```

Bring a job to the foreground.

Usage: fg [JOB_ID] If JOB_ID is provided, foregrounds that specific job. Otherwise, foregrounds the most recent job. Claims the terminal, sends SIGCONT, and waits for the job to complete or stop.

Parameters

job_table	Pointer to the shell's job table.
argv	NULL-terminated argument array (argv[1] = optional job ID).

If a job ID is provided, validates and uses that job. Otherwise, selects the most recent job (highest age, doesn't need to be stopped).

Parameters

job_table	Pointer to the shell's job table.
argv	NULL-terminated argument array.

4.51.2.3 u_jobs()

```
void u_jobs (
            job_table_t * job_table )
```

List all current jobs.

Usage: jobs Displays all jobs in the job table with their status (Running, Blocked, or Stopped), job ID, and command string.

Parameters

job_table	Pointer to the shell's job table.

List all current jobs.

Parameters

job_table	Pointer to the shell's job table.

4.52 src/shell/process/u_process.h File Reference

User-level process management commands for the PennOS shell.

```
#include "shell/job/job_table_t.h"
#include "util/parser.h"
```

Include dependency graph for u_process.h: This graph shows which files directly or indirectly include this file:

Functions

- void u_nice (job_table_t ∗shell, char ∗∗argv, struct parsed_command ∗cmd)

  Spawn a command with a specified priority.
- void u_nice_pid (char ∗∗argv)

  Change the priority of an existing process.
- void ∗ u_ps (void ∗arg)

  List all processes in the system.
- void ∗ u_kill (void ∗arg)

  Send signals to processes.
- void u_logout (void)

  Logout and shutdown PennOS.
- void ∗ u_sleep (void ∗arg)

  Sleep for a specified duration.

4.52.1 Detailed Description

User-level process management commands for the PennOS shell.

This header declares built-in shell commands for process management, including listing processes, sending signals, changing priorities, sleeping, and system logout.

4.52.2 Function Documentation

4.52.2.1 u_kill()

```
void* u_kill (
            void * arg )
```

Send signals to processes.

Sends signals (SIGTERM/SIGSTOP/SIGCONT) to one or more processes specified by their PIDs.

Parameters

arg	Pointer to NULL-terminated argv array.

Returns

Always NULL.

4.52.2.2 u_logout()

```
void u_logout (
            void  )
```

Logout and shutdown PennOS.

Initiates system shutdown by killing all processes and exiting cleanly.

4.52.2.3 u_nice()

```
void u_nice (
            job_table_t * shell,
            char ** argv,
            struct parsed_command * cmd )
```

Spawn a command with a specified priority.

Spawns a new process with specified priority level (0=high, 1=med, 2=low).

Parameters

job_table	Pointer to the shell's job table for background jobs.
argv	NULL-terminated argument array.
cmd	Parsed command structure for redirection/background handling.

4.52.2.4 u_nice_pid()

```
void u_nice_pid (
            char ** argv )
```

Change the priority of an existing process.

Changes priority of already-running process by PID.

Parameters

argv	NULL-terminated argument array.

4.52.2.5 u_ps()

```
void* u_ps (
            void * arg )
```

List all processes in the system.

Displays all active processes with their PID, parent PID, priority, state, and name in a formatted table.

Parameters

arg	Unused parameter (for spthread compatibility).

Returns

Always NULL.

4.52.2.6 u_sleep()

```
void* u_sleep (
            void * arg )
```

Sleep for a specified duration.

Blocks the calling process for the specified number of seconds.

Parameters

arg	Pointer to NULL-terminated argv array.

Returns

Always NULL.

4.53 src/shell/u_shell.h File Reference

PennOS shell entry point declaration.

This graph shows which files directly or indirectly include this file:

Functions

- void * u_shell (void *arg)

  Main shell loop for PennOS.

4.53.1 Detailed Description

PennOS shell entry point declaration.

This header declares the main shell loop function that provides the interactive command-line interface for PennOS.

4.53.2 Function Documentation

4.53.2.1 u_shell()

```
void* u_shell (
            void * arg )
```

Main shell loop for PennOS.

Runs the interactive shell or executes a script file.

Parameters

arg	NULL for interactive mode, or argv array with script filename.

Returns

Always NULL.

4.54 src/util/log.c File Reference

Implementation of the PennOS scheduler event logger.

```
#include "log.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Include dependency graph for log.c:

Functions

- int log_init (const char *log_file)

  Initialize the logging system.

- void log_close (void)
  Close the logging system.
- unsigned int log_get_ticks (void)
  Get the current clock tick count.
- void log_tick (void)
  Increment the clock tick counter.
- void log_schedule (pid_t pid, int queue, const char ∗process_name, bool interrupt)
  Log a process scheduling event.
- void log_create (pid_t pid, int nice_value, const char ∗process_name)
  Log process creation.
- void log_signaled (pid_t pid, int nice_value, const char ∗process_name)
  Log process termination by signal.
- void log_exited (pid_t pid, int nice_value, const char ∗process_name)
  Log normal process exit.
- void log_zombie (pid_t pid, int nice_value, const char ∗process_name)
  Log process entering zombie state.
- void log_orphan (pid_t pid, int nice_value, const char ∗process_name)
  Log process being orphaned.
- void log_waited (pid_t pid, int nice_value, const char ∗process_name, bool is_init)
  Log process being reaped by wait.
- void log_nice (pid_t pid, int old_nice, int new_nice, const char ∗process_name)
  Log priority change.
- void log_blocked (pid_t pid, int nice_value, const char ∗process_name)
  Log process blocking (sleep/wait).
- void log_unblocked (pid_t pid, int nice_value, const char ∗process_name)
  Log process unblocking (wakeup).
- void log_stopped (pid_t pid, int nice_value, const char ∗process_name)
  Log process being stopped (SIGSTOP).
- void log_continued (pid_t pid, int nice_value, const char ∗process_name)
  Log process being continued (SIGCONT).

4.54.1 Detailed Description

Implementation of the PennOS scheduler event logger.

This file implements logging of scheduler events to a file for debugging and grading purposes. Each log entry includes a timestamp (clock ticks), event type, process ID, priority, and process name.

Log format: [ticks]\tEVENT\tpid\tpriority\tprocess_name

4.54.2 Function Documentation

4.54.2.1 log_blocked()

```
void log_blocked (
            pid_t pid,
            int nice_value,
            const char * process_name )
```

Log process blocking (sleep/wait).

Log process blocking.

Parameters

pid	Process ID.
nice_value	Process priority.
process_name	Name of the process.

4.54.2.2 log_close()

```
void log_close (
            void  )
```

Close the logging system.

Closes the log file if open and sets handle to NULL.

4.54.2.3 log_continued()

```
void log_continued (
            pid_t pid,
            int nice_value,
            const char * process_name )
```

Log process being continued (SIGCONT).

Log process continuation.

Parameters

pid	Process ID.
nice_value	Process priority.
process_name	Name of the process.

4.54.2.4 log_create()

```
void log_create (
            pid_t pid,
            int nice_value,
            const char * process_name )
```

Log process creation.

Parameters

pid	New process ID.
nice_value	Process priority.
process_name	Name of the process.

4.54.2.5 log_exited()

```
void log_exited (
            pid_t pid,
            int nice_value,
            const char * process_name )
```

Log normal process exit.

Log process normal exit.

Parameters

pid	Process ID.
nice_value	Process priority.
process_name	Name of the process.

4.54.2.6 log_get_ticks()

```
unsigned int log_get_ticks (
            void )
```

Get the current clock tick count.

Get current clock ticks since boot.

Returns

Number of clock ticks since log_init.

4.54.2.7 log_init()

```
int log_init (
            const char * log_file )
```

Initialize the logging system.

Opens the log file for writing, closes any previously open file, and resets the clock tick counter to 0.

Parameters

log_file	Path to the log file.

Returns

0 on success, -1 on error.

4.54.2.8 log_nice()

```
void log_nice (
            pid_t pid,
            int old_nice,
            int new_nice,
            const char * process_name )
```

Log priority change.

Log nice value change.

Parameters

pid	Process ID.
old_nice	Previous priority.
new_nice	New priority.
process_name	Name of the process.

4.54.2.9 log_orphan()

```
void log_orphan (
            pid_t pid,
            int nice_value,
            const char * process_name )
```

Log process being orphaned.

Log process orphaning.

Parameters

pid	Process ID.
nice_value	Process priority.
process_name	Name of the process.

4.54.2.10 log_schedule()

```
void log_schedule (
            pid_t pid,
```

```
            int queue,
            const char * process_name,
            bool interrupt )
```

Log a process scheduling event.

Log a schedule event.

Logs SCHEDULE or RESCHEDULE (if interrupts were disabled).

Parameters

pid	Process ID being scheduled.
queue	Priority queue (0, 1, or 2).
process_name	Name of the process.
interrupt	True if process had interrupts disabled.

4.54.2.11 log_signaled()

```
void log_signaled (
            pid_t pid,
            int nice_value,
            const char * process_name )
```

Log process termination by signal.

Log process signaled termination.

Parameters

pid	Process ID.
nice_value	Process priority.
process_name	Name of the process.

4.54.2.12 log_stopped()

```
void log_stopped (
            pid_t pid,
            int nice_value,
            const char * process_name )
```

Log process being stopped (SIGSTOP).

Log process stop.

Parameters

pid	Process ID.
nice_value	Process priority.
process_name	Name of the process.

4.54.2.13 log_tick()

```
void log_tick (
            void  )
```

Increment the clock tick counter.

Called by the scheduler on each SIGALRM (100ms interval).

4.54.2.14 log_unblocked()

```
void log_unblocked (
            pid_t pid,
            int nice_value,
            const char * process_name )
```

Log process unblocking (wakeup).

Log process unblocking.

Parameters

pid	Process ID.
nice_value	Process priority.
process_name	Name of the process.

4.54.2.15 log_waited()

```
void log_waited (
            pid_t pid,
            int nice_value,
            const char * process_name,
            bool is_init )
```

Log process being reaped by wait.

Log process wait/reap.

Parameters

pid	Process ID.
nice_value	Process priority.
process_name	Name of the process.
is_init	True if reaped by init process.

4.54.2.16 log_zombie()

```
void log_zombie (
            pid_t pid,
            int nice_value,
            const char * process_name )
```

Log process entering zombie state.

Log process zombification.

Parameters

pid	Process ID.
nice_value	Process priority.
process_name	Name of the process.

4.55 src/util/log.h File Reference

Scheduler event logging interface for PennOS.

```
#include <stdio.h>
#include "process/pcb/pcb_t.h"
```

Include dependency graph for log.h: This graph shows which files directly or indirectly include this file:

Functions

- int log_init (const char ∗log_file)

  Initialize the logging system.
- void log_close (void)

  Close the logging system.
- unsigned int log_get_ticks (void)

  Get current clock ticks since boot.
- void log_tick (void)

  Increment the clock tick counter.
- void log_schedule (pid_t pid, int queue, const char ∗process_name, bool interrupt)

  Log a schedule event.
- void log_create (pid_t pid, int nice_value, const char ∗process_name)

Log process creation.

- void log_signaled (pid_t pid, int nice_value, const char ∗process_name)

  Log process signaled termination.
- void log_exited (pid_t pid, int nice_value, const char ∗process_name)

  Log process normal exit.
- void log_zombie (pid_t pid, int nice_value, const char ∗process_name)

  Log process zombification.
- void log_orphan (pid_t pid, int nice_value, const char ∗process_name)

  Log process orphaning.
- void log_waited (pid_t pid, int nice_value, const char ∗process_name, bool is_init)

  Log process wait/reap.
- void log_nice (pid_t pid, int old_nice, int new_nice, const char ∗process_name)

  Log nice value change.
- void log_blocked (pid_t pid, int nice_value, const char ∗process_name)

  Log process blocking.
- void log_unblocked (pid_t pid, int nice_value, const char ∗process_name)

  Log process unblocking.
- void log_stopped (pid_t pid, int nice_value, const char ∗process_name)

  Log process stop.
- void log_continued (pid_t pid, int nice_value, const char ∗process_name)

  Log process continuation.

4.55.1 Detailed Description

Scheduler event logging interface for PennOS.

This header provides functions for logging scheduler events to a file, including process creation, scheduling, state changes, and termination. The log format is: [ticks] EVENT pid priority process_name

4.55.2 Function Documentation

4.55.2.1 log_blocked()

```
void log_blocked (
            pid_t pid,
            int nice_value,
            const char * process_name )
```

Log process blocking.

Parameters

pid	Process ID
nice_value	Priority value
process_name	Name of the process

Log process blocking.

Parameters

pid	Process ID.
nice_value	Process priority.
process_name	Name of the process.

4.55.2.2 log_close()

```
void log_close (
            void  )
```

Close the logging system.

Closes the log file if open and sets handle to NULL.

4.55.2.3 log_continued()

```
void log_continued (
            pid_t pid,
            int nice_value,
            const char * process_name )
```

Log process continuation.

Parameters

pid	Process ID
nice_value	Priority value
process_name	Name of the process

Log process continuation.

Parameters

pid	Process ID.
nice_value	Process priority.
process_name	Name of the process.

4.55.2.4 log_create()

```
void log_create (
            pid_t pid,
```

```
            int nice_value,
            const char * process_name )
```

Log process creation.

Parameters

pid	Process ID
nice_value	Priority value
process_name	Name of the process
pid	New process ID.
nice_value	Process priority.
process_name	Name of the process.

4.55.2.5 log_exited()

```
void log_exited (
            pid_t pid,
            int nice_value,
            const char * process_name )
```

Log process normal exit.

Parameters

pid	Process ID
nice_value	Priority value
process_name	Name of the process

Log process normal exit.

Parameters

pid	Process ID.
nice_value	Process priority.
process_name	Name of the process.

4.55.2.6 log_get_ticks()

```
unsigned int log_get_ticks (
            void )
```

Get current clock ticks since boot.

Returns

Number of clock ticks

Get current clock ticks since boot.

Returns

Number of clock ticks since log_init.

4.55.2.7 log_init()

```
int log_init (
            const char * log_file )
```

Initialize the logging system.

Parameters

log_file	File path for logging output

Returns

0 on success, -1 on error

Opens the log file for writing, closes any previously open file, and resets the clock tick counter to 0.

Parameters

log_file	Path to the log file.

Returns

0 on success, -1 on error.

4.55.2.8 log_nice()

```
void log_nice (
            pid_t pid,
            int old_nice,
            int new_nice,
            const char * process_name )
```

Log nice value change.

Parameters

pid	Process ID
old_nice	Old priority value
new_nice	New priority value
process_name	Name of the process

Log nice value change.

Parameters

pid	Process ID.
old_nice	Previous priority.
new_nice	New priority.
process_name	Name of the process.

4.55.2.9 log_orphan()

```
void log_orphan (
            pid_t pid,
            int nice_value,
            const char * process_name )
```

Log process orphaning.

Parameters

pid	Process ID
nice_value	Priority value
process_name	Name of the process

Log process orphaning.

Parameters

pid	Process ID.
nice_value	Process priority.
process_name	Name of the process.

4.55.2.10 log_schedule()

```
void log_schedule (
            pid_t pid,
```

```
            int queue,
            const char * process_name,
            bool interrupt )
```

Log a schedule event.

Parameters

pid	Process ID
queue	Priority queue (0, 1, or 2)
process_name	Name of the process

Log a schedule event.

Logs SCHEDULE or RESCHEDULE (if interrupts were disabled).

Parameters

pid	Process ID being scheduled.
queue	Priority queue (0, 1, or 2).
process_name	Name of the process.
interrupt	True if process had interrupts disabled.

4.55.2.11 log_signaled()

```
void log_signaled (
            pid_t pid,
            int nice_value,
            const char * process_name )
```

Log process signaled termination.

Parameters

pid	Process ID
nice_value	Priority value
process_name	Name of the process

Log process signaled termination.

Parameters

pid	Process ID.
nice_value	Process priority.
process_name	Name of the process.

4.55.2.12 log_stopped()

```
void log_stopped (
            pid_t pid,
            int nice_value,
            const char * process_name )
```

Log process stop.

Parameters

pid	Process ID
nice_value	Priority value
process_name	Name of the process

Log process stop.

Parameters

pid	Process ID.
nice_value	Process priority.
process_name	Name of the process.

4.55.2.13 log_tick()

```
void log_tick (
            void )
```

Increment the clock tick counter.

Called by the scheduler on each SIGALRM (100ms interval).

4.55.2.14 log_unblocked()

```
void log_unblocked (
            pid_t pid,
            int nice_value,
            const char * process_name )
```

Log process unblocking.

Parameters

pid	Process ID
nice_value	Priority value
process_name	Name of the process

Log process unblocking.

Parameters

pid	Process ID.
nice_value	Process priority.
process_name	Name of the process.

4.55.2.15 log_waited()

```
void log_waited (
            pid_t pid,
            int nice_value,
            const char * process_name,
            bool is_init )
```

Log process wait/reap.

Parameters

pid	Process ID
nice_value	Priority value
process_name	Name of the process

Log process wait/reap.

Parameters

pid	Process ID.
nice_value	Process priority.
process_name	Name of the process.
is_init	True if reaped by init process.

4.55.2.16 log_zombie()

```
void log_zombie (
            pid_t pid,
            int nice_value,
            const char * process_name )
```

Log process zombification.

Parameters

pid	Process ID
nice_value	Priority value
process_name	Name of the process

Log process zombification.

Parameters

pid	Process ID.
nice_value	Process priority.
process_name	Name of the process.

4.56 src/util/panic.c File Reference

Implementation of kernel panic handling.

```
#include "./panic.h"
#include <errno.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include "io/term/s_term.h"
```

Include dependency graph for panic.c:

Functions

- void print_and_abort (const char *error_message)

  Print an error message to stderr and abort.

4.56.1 Detailed Description

Implementation of kernel panic handling.

This file implements the print_and_abort function used by the panic macro to handle unrecoverable errors in Penn↩ OS.

4.56.2 Function Documentation

4.56.2.1 print_and_abort()

```
void print_and_abort (
            const char * error_message )
```

Print an error message to stderr and abort.

Print error message and abort the program.

Writes the error message to STDERR_FILENO using s_write, handling partial writes and retrying on EINTR/↩ EAGAIN. After the message is written (or write fails), calls abort() to terminate the program.

Parameters

error_message	The message to print before aborting.

4.57 src/util/panic.h File Reference

Kernel panic handling for PennOS.

This graph shows which files directly or indirectly include this file:

Macros

- #define panic(error_message) print_and_abort(error_message)

  Panic macro (enabled).

Functions

- void print_and_abort (const char ∗error_message)

  Print error message and abort the program.

4.57.1 Detailed Description

Kernel panic handling for PennOS.

This header provides the panic macro for handling unrecoverable errors. When enabled (default), panic prints an error message and aborts. Can be disabled by defining DISABLE_PANIC before including this header.

4.57.2 Macro Definition Documentation

4.57.2.1 panic

```
#define panic(
            error_message ) print_and_abort(error_message)
```

Panic macro (enabled).

Calls print_and_abort with the given error message, which prints to stderr and aborts the program.

Parameters

error_message	String describing the panic reason.

4.57.3 Function Documentation

4.57.3.1 print_and_abort()

```
void print_and_abort (
            const char * error_message )
```

Print error message and abort the program.

Writes the error message to stderr using s_write, then calls abort(). Handles partial writes and retries on EINTR/↩ EAGAIN.

Parameters

error_message	The error message to display.

Print error message and abort the program.

Writes the error message to STDERR_FILENO using s_write, handling partial writes and retrying on EINTR/↩ EAGAIN. After the message is written (or write fails), calls abort() to terminate the program.

Parameters

error_message	The message to print before aborting.

4.58 src/util/parser.c File Reference

Implementation of the shell command line parser.

```
#include "parser.h"
#include <ctype.h>
#include <errno.h>
#include <stdlib.h>
#include <string.h>
#include "io/term/s_term.h"
#include <stdio.h>
```

Include dependency graph for parser.c:

Macros

- #define **JUMP_OUT**(code)

Functions

- int parse_command (const char ∗const cmd_line, struct parsed_command ∗∗const result)

  Parse a command line into a parsed_command structure.
- void print_parsed_command (int output, const struct parsed_command ∗const cmd)

  Print a parsed command to a file descriptor.
- void print_parser_errcode (int output, int err_code)

  Print a descriptive error message for a parser error code.
- bool parseId (char ∗str, uint64_t ∗out)

  Parse a string as a uint64_t.

4.58.1 Detailed Description

Implementation of the shell command line parser.

This file implements a two-pass parser for shell commands:

- Pass 1: Validates token sequences and counts arguments
- Pass 2: Populates the parsed_command structure

The parser handles comments (#), whitespace, special tokens (< > >> | &), and regular arguments.

4.58.2 Macro Definition Documentation

4.58.2.1 JUMP_OUT

```
#define JUMP_OUT(
                code )
```

Value:

```
do {                    \
  ret_code = code;      \
  goto PROCESS_ERROR;   \
} while (0)
```

4.58.3 Function Documentation

4.58.3.1 parse_command()

```
int parse_command (
            const char *const cmd_line,
            struct parsed_command **const result )
```

Parse a command line into a parsed_command structure.

Two-pass parsing:

1. First pass validates tokens and counts arguments
2. Second pass allocates memory and populates the structure

Memory layout of parsed_command:

- Fixed fields (is_background, etc.)
- commands[] array (num_commands pointers)
- arguments[] array (total_strings + num_commands pointers)
- String data (null-terminated copies of tokens)

Parameters

cmd_line	NULL-terminated command string.
result	Output pointer for allocated structure.

Returns

0 on success, -1 on system error, 1-7 on parse error.

layout of memory for `struct parsed_command` bool is_background; bool is_file_append;

const char *stdin_file; const char *stdout_file;

size_t num_commands;

commands are pointers to `arguments` char **commands[num_commands];

below are hidden in memory **

arguments are pointers to `original_string + num_commands` because all argv are null-terminated char *arguments[total_strings + num_commands];

original_string is a copy of the cmdline but with each token null-terminated char *original_string;

4.58.3.2 parseId()

```
bool parseId (
            char * str,
            uint64_t * out )
```

Parse a string as a uint64_t.

Uses strtol with base 10. Fails if the entire string is not consumed or if overflow occurs.

Parameters

str	String to parse.
out	Output pointer for parsed value.

Returns

true on success, false on error.

4.58.3.3 print_parsed_command()

```
void print_parsed_command (
            int output,
            const struct parsed_command *const cmd )
```

Print a parsed command to a file descriptor.

Outputs commands with arguments, redirection operators, and pipe symbols in proper order.

Parameters

output	File descriptor to write to.
cmd	Parsed command to print.

4.58.3.4 print_parser_errcode()

```
void print_parser_errcode (
            int output,
            int err_code )
```

Print a descriptive error message for a parser error code.

Maps error codes to human-readable messages and prints them.

Parameters

output	File descriptor to write to.
err_code	Parser error code.

4.59 src/util/parser.h File Reference

Shell command line parser for PennOS.

```
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
```

Include dependency graph for parser.h: This graph shows which files directly or indirectly include this file:

Classes

- struct parsed_command

Macros

- #define **PARSER_SUCCESS** 0
- #define **UNEXPECTED_FILE_INPUT** 1
- #define **UNEXPECTED_FILE_OUTPUT** 2
- #define **UNEXPECTED_PIPELINE** 3
- #define **UNEXPECTED_AMPERSAND** 4
- #define **EXPECT_INPUT_FILENAME** 5
- #define **EXPECT_OUTPUT_FILENAME** 6
- #define **EXPECT_COMMANDS** 7

Functions

- int parse_command (const char ∗cmd_line, struct parsed_command ∗∗result)

  Parse a command line into a parsed_command structure.
- void print_parsed_command (int fd, const struct parsed_command ∗cmd)

  Print a parsed command to a file descriptor.
- void print_parser_errcode (int fd, int err_code)

  Print a descriptive error message for a parser error code.
- bool parseId (char ∗str, uint64_t ∗out)

  Parse a string as a uint64_t.

4.59.1 Detailed Description

Shell command line parser for PennOS.

This header provides the command line parsing interface for the PennOS shell. It handles tokenization of shell commands including:

- Input/output redirection (< > >>)
- Pipelines (|)
- Background execution (&)
- Comments (#)

Author

hanbangw, 21fa

4.59.2 Function Documentation

4.59.2.1 parse_command()

```
int parse_command (
            const char *const cmd_line,
            struct parsed_command **const result )
```

Parse a command line into a parsed_command structure.

Arguments: cmd_line: a null-terminated string that is the command line result: a non-null pointer to a `struct parsed_command *`

Return value (int): an error code which can be, 0: parser finished succesfully -1: parser encountered a system call error 1-7: parser specific error, see error type above

This function will parse the given `cmd_line` and store the parsed information into a `struct parsed_command`. The memory needed for the struct will be allocated by this function, and the pointer to the memory will be stored into the given `*result`.

You can directly use the result in system calls. See demo for more information.

If the function returns a successful value (0), a `struct parsed_command` is guareenteed to be allocated and stored in the given `*result`. It is the caller's responsibility to free the given pointer using `free(3)`.

Otherwise, no `struct parsed_command` is allocated and `*result` is unchanged. If a system call error (-1) is returned, the caller can use `errno(3)` or `perror(3)` to gain more information about the error.

Two-pass parsing:

1. First pass validates tokens and counts arguments
2. Second pass allocates memory and populates the structure

Memory layout of parsed_command:

- Fixed fields (is_background, etc.)
- commands[] array (num_commands pointers)
- arguments[] array (total_strings + num_commands pointers)
- String data (null-terminated copies of tokens)

Parameters

cmd_line	NULL-terminated command string.
result	Output pointer for allocated structure.

Returns

0 on success, -1 on system error, 1-7 on parse error.

layout of memory for `struct parsed_command` bool is_background; bool is_file_append;

const char *stdin_file; const char *stdout_file;

size_t num_commands;

commands are pointers to `arguments` char **commands[num_commands];

below are hidden in memory **

arguments are pointers to `original_string + num_commands` because all argv are null-terminated char *arguments[total_strings + num_commands];

original_string is a copy of the cmdline but with each token null-terminated char *original_string;

4.59.2.2 parseId()

```
bool parseId (
            char * str,
            uint64_t * out )
```

Parse a string as a uint64_t.

Arguments: str: the string to parse out: a pointer to where the result should be assigned if successful.

Attempts to parse the string for an id, returning true if successful and false otherwise. If successful, sets *out to the parsed value.

Uses strtol with base 10. Fails if the entire string is not consumed or if overflow occurs.

Parameters

str	String to parse.
out	Output pointer for parsed value.

Returns

true on success, false on error.

4.59.2.3 print_parsed_command()

```
void print_parsed_command (
            int output,
            const struct parsed_command *const cmd )
```

Print a parsed command to a file descriptor.

Arguments: output: where to print parsed command to cmd: a pointer to a struct parsed_command * with a valid command

Print the given command to the provided output location under a standard format. The ampersand (&) character indicating background processes are not included.

Outputs commands with arguments, redirection operators, and pipe symbols in proper order.

Parameters

output	File descriptor to write to.
cmd	Parsed command to print.

4.59.2.4 print_parser_errcode()

```
void print_parser_errcode (
            int output,
            int err_code )
```

Print a descriptive error message for a parser error code.

Arguments: output: where to print the error to err_code: the error code to print

Prints the error description of err_code to the file stream

Maps error codes to human-readable messages and prints them.

Parameters

output	File descriptor to write to.
err_code	Parser error code.

Index

__reserved
    file_st, 9

age
    job_st, 11
alive_child
    pcb_t, 16
apply_offset
    k_filetable.c, 64
    k_filetable.h, 74
avail_process
    pcb_table_t, 19

block_size
    fat_metadata_st, 7

cmd_info
    job_st, 11
command
    job_st, 11
commands.h
    run_command, 158
    u_echo, 159
    u_man, 159
curr_age
    job_table_st, 12
curr_block
    k_fd_entry_t, 13

fat_metadata_st, 7
    block_size, 7
    tot_blocks, 7
fat_ut, 8
    metadata, 8
    next, 8
fd_alloc
    k_filetable.c, 64
file
    k_open_file_entry_t, 14
file_alloc
    k_filetable.c, 65
file_st, 9
    __reserved, 9
    first_block, 9
    mtime, 9
    name, 9
    perm, 10
    size, 10
    type, 10
first_block
    file_st, 9
free_job_table
    job_table.c, 164
    job_table_t.h, 167

h_close
    h_file.c, 42
h_file.c
    h_close, 42
    h_lseek, 43
    h_open, 43
    h_read, 43
    h_write, 44
h_lseek
    h_file.c, 43
h_open
    h_file.c, 43
h_read
    h_file.c, 43
h_write
    h_file.c, 44
hfd
    k_fd_entry_t, 13

ign_int
    pcb_t, 17
ign_stop
    pcb_t, 17
init_job_table
    job_table.c, 164
    job_table_t.h, 167

job_add
    job_table.c, 164
    job_table_t.h, 167
job_remove
    job_table.c, 165
    job_table_t.h, 168
job_st, 10
    age, 11
    cmd_info, 11
    command, 11
    pid, 11
job_table.c
    free_job_table, 164
    init_job_table, 164
    job_add, 164
    job_remove, 165
    job_touch, 165
    print_job_status, 165

job_table_st, 11
curr_age, 12
latest_job, 12
vector, 12
job_table_t.h
free_job_table, 167
init_job_table, 167
job_add, 167
job_remove, 168
job_touch, 168
print_job_status, 168
job_touch
job_table.c, 165
job_table_t.h, 168
JUMP_OUT
parser.c, 193

k_can_interrupt
k_schedule.c, 136
k_schedule.h, 141
k_claim_terminal
k_term.c, 94
k_term.h, 96
k_close
k_file.c, 45
k_dir_add_entry
k_path.c, 84
k_path.h, 87
k_dir_rem_entry
k_path.c, 85
k_path.h, 88
k_dir_search
k_path.c, 85
k_path.h, 88
k_dir_upd_entry
k_path.c, 86
k_path.h, 89
k_disable_interrupts
k_schedule.c, 136
k_schedule.h, 141
k_enable_interrupts
k_schedule.c, 136
k_schedule.h, 141
k_ext_open
k_file.c, 46
k_ext_read
k_file.c, 46
k_fat.c
k_get_blk_size, 26
k_get_block_size, 26
k_get_data_pos, 27
k_get_fat, 27
k_get_fat_fd, 27
k_has_mount, 28
k_mkfs, 28
k_mount, 28
k_release_block, 29
k_release_fat_chain, 29
k_request_block, 30
k_sizeof_fat, 30
k_start_block, 30
k_unmount, 31
k_fat.h
k_get_blk_size, 32
k_get_block_size, 32
k_get_data_pos, 33
k_get_fat, 33
k_get_fat_fd, 34
k_has_mount, 34
k_mkfs, 34
k_mount, 35
k_release_block, 35
k_release_fat_chain, 35
k_request_block, 36
k_sizeof_fat, 36
k_start_block, 36
k_unmount, 37
k_fd_alloc
k_filetable.c, 65
k_filetable.h, 75
k_fd_alloc_h
k_filetable.c, 66
k_filetable.h, 75
k_fd_close
k_filetable.c, 66
k_filetable.h, 76
k_fd_dup
k_filetable.c, 67
k_filetable.h, 76
k_fd_entry_t, 12
curr_block, 13
hfd, 13
mode, 13
open_file_idx, 13
refcount, 13
target_offset, 13
virtual_offset, 13
k_fd_file_ref
k_filetable.c, 67
k_filetable.h, 77
k_fd_get_hfd
k_filetable.c, 68
k_filetable.h, 77
k_fd_get_offset
k_filetable.c, 68
k_filetable.h, 77
k_fd_get_size
k_filetable.c, 68
k_filetable.h, 78
k_fd_has_entry
k_filetable.c, 69
k_filetable.h, 78
k_fd_init
k_filetable.h, 79
k_fd_is_open
k_filetable.c, 69
k_filetable.h, 79

k_fd_is_write_locked
k_filetable.c, 70
k_filetable.h, 79
k_fd_mode
k_filetable.c, 70
k_filetable.h, 80
k_fd_set_offset
k_filetable.c, 70
k_filetable.h, 80
k_fd_walk
k_filetable.c, 71
k_filetable.h, 81
k_file.c
k_close, 45
k_ext_open, 46
k_ext_read, 46
k_lseek, 47
k_mv, 47
k_open, 48
k_read, 49
k_unlink, 49
k_write, 50
k_filetable.c
apply_offset, 64
fd_alloc, 64
file_alloc, 65
k_fd_alloc, 65
k_fd_alloc_h, 66
k_fd_close, 66
k_fd_dup, 67
k_fd_file_ref, 67
k_fd_get_hfd, 68
k_fd_get_offset, 68
k_fd_get_size, 68
k_fd_has_entry, 69
k_fd_is_open, 69
k_fd_is_write_locked, 70
k_fd_mode, 70
k_fd_set_offset, 70
k_fd_walk, 71
k_lfd_get, 71
k_lfd_put, 72
k_lfd_release, 72
k_filetable.h
apply_offset, 74
k_fd_alloc, 75
k_fd_alloc_h, 75
k_fd_close, 76
k_fd_dup, 76
k_fd_file_ref, 77
k_fd_get_hfd, 77
k_fd_get_offset, 77
k_fd_get_size, 78
k_fd_has_entry, 78
k_fd_init, 79
k_fd_is_open, 79
k_fd_is_write_locked, 79
k_fd_mode, 80
k_fd_set_offset, 80
k_fd_walk, 81
k_lfd_get, 81
k_lfd_put, 82
k_lfd_release, 82
k_get_blk_size
k_fat.c, 26
k_fat.h, 32
k_get_block_size
k_fat.c, 26
k_fat.h, 32
k_get_data_pos
k_fat.c, 27
k_fat.h, 33
k_get_fat
k_fat.c, 27
k_fat.h, 33
k_get_fat_fd
k_fat.c, 27
k_fat.h, 34
k_get_root_dir
k_path.c, 86
k_path.h, 90
k_get_terminal_owner
k_term.c, 94
k_term.h, 96
k_get_terminal_pass_parent
k_term.c, 94
k_term.h, 96
k_has_mount
k_fat.c, 28
k_fat.h, 34
k_is_term
k_term.c, 94
k_term.h, 97
k_lfd_get
k_filetable.c, 71
k_filetable.h, 81
k_lfd_put
k_filetable.c, 72
k_filetable.h, 82
k_lfd_release
k_filetable.c, 72
k_filetable.h, 82
k_lifecycle.c
k_proc_cleanup, 110
k_proc_create, 110
k_proc_zombify, 111
transfer_parent, 111
k_lifecycle.h
k_pcb_data, 112
k_proc_cleanup, 113
k_proc_create, 113
k_proc_zombify, 114
k_spthread_create, 114
k_ls
k_path.c, 86
k_path.h, 90

k_lseek
k_file.c, 47
k_mkfs
k_fat.c, 28
k_fat.h, 34
k_mount
k_fat.c, 28
k_fat.h, 35
k_mv
k_file.c, 47
k_next_process
k_pcb.c, 121
k_pcb.h, 124
k_notify_parent
k_signal.c, 148
k_signal.h, 150
k_open
k_file.c, 48
k_open_file_entry_t, 14
file, 14
refcount, 14
k_pass_terminal_to_next_child
k_term.c, 95
k_term.h, 97
k_path.c
k_dir_add_entry, 84
k_dir_rem_entry, 85
k_dir_search, 85
k_dir_upd_entry, 86
k_get_root_dir, 86
k_ls, 86
k_path.h
k_dir_add_entry, 87
k_dir_rem_entry, 88
k_dir_search, 88
k_dir_upd_entry, 89
k_get_root_dir, 90
k_ls, 90
k_pcb.c
k_next_process, 121
k_pcb_claim, 121
k_pcb_data, 121
k_pcb_init, 122
k_pcb_list_add, 122
k_pcb_list_insert, 122
k_pcb_list_remove, 123
k_pcb_table_size, 123
k_proc_release, 123
k_pcb.h
k_next_process, 124
k_pcb_claim, 125
k_pcb_data, 125
k_pcb_init, 126
k_pcb_list_add, 126
k_pcb_list_insert, 126
k_pcb_list_remove, 127
k_pcb_table_size, 127
k_proc_release, 127
k_pcb_claim
k_pcb.c, 121
k_pcb.h, 125
k_pcb_data
k_lifecycle.h, 112
k_pcb.c, 121
k_pcb.h, 125
k_pcb_init
k_pcb.c, 122
k_pcb.h, 126
k_pcb_list_add
k_pcb.c, 122
k_pcb.h, 126
k_pcb_list_insert
k_pcb.c, 122
k_pcb.h, 126
k_pcb_list_remove
k_pcb.c, 123
k_pcb.h, 127
k_pcb_table_size
k_pcb.c, 123
k_pcb.h, 127
k_pollpid
k_signal.c, 148
k_signal.h, 150
k_proc_cleanup
k_lifecycle.c, 110
k_lifecycle.h, 113
k_proc_create
k_lifecycle.c, 110
k_lifecycle.h, 113
k_proc_release
k_pcb.c, 123
k_pcb.h, 127
k_proc_zombify
k_lifecycle.c, 111
k_lifecycle.h, 114
k_read
k_file.c, 49
k_release_block
k_fat.c, 29
k_fat.h, 35
k_release_fat_chain
k_fat.c, 29
k_fat.h, 35
k_remove_int
k_schedule.c, 136
k_schedule.h, 141
k_remove_stop
k_schedule.c, 136
k_schedule.h, 142
k_request_block
k_fat.c, 30
k_fat.h, 36
k_sched_add
k_schedule.c, 137
k_schedule.h, 142
k_sched_block

k_schedule.c, 137
k_schedule.h, 142
k_sched_clean
k_schedule.c, 138
k_schedule.h, 143
k_sched_current_pid
k_schedule.c, 138
k_schedule.h, 143
k_sched_init
k_schedule.c, 138
k_schedule.h, 143
k_sched_remove
k_schedule.c, 138
k_schedule.h, 144
k_sched_run
k_schedule.c, 139
k_schedule.h, 144
k_sched_stop
k_schedule.c, 139
k_schedule.h, 145
k_sched_unblock
k_schedule.c, 139
k_schedule.h, 145
k_schedule.c
k_can_interrupt, 136
k_disable_interrupts, 136
k_enable_interrupts, 136
k_remove_int, 136
k_remove_stop, 136
k_sched_add, 137
k_sched_block, 137
k_sched_clean, 138
k_sched_current_pid, 138
k_sched_init, 138
k_sched_remove, 138
k_sched_run, 139
k_sched_stop, 139
k_sched_unblock, 139
k_schedule.h
k_can_interrupt, 141
k_disable_interrupts, 141
k_enable_interrupts, 141
k_remove_int, 141
k_remove_stop, 142
k_sched_add, 142
k_sched_block, 142
k_sched_clean, 143
k_sched_current_pid, 143
k_sched_init, 143
k_sched_remove, 144
k_sched_run, 144
k_sched_stop, 145
k_sched_unblock, 145
k_signal.c
k_notify_parent, 148
k_pollpid, 148
k_signal.h
k_notify_parent, 150
k_pollpid, 150
k_sizeof_fat
k_fat.c, 30
k_fat.h, 36
k_spthread_create
k_lifecycle.h, 114
k_start_block
k_fat.c, 30
k_fat.h, 36
k_term.c
k_claim_terminal, 94
k_get_terminal_owner, 94
k_get_terminal_pass_parent, 94
k_is_term, 94
k_pass_terminal_to_next_child, 95
k_term.h
k_claim_terminal, 96
k_get_terminal_owner, 96
k_get_terminal_pass_parent, 96
k_is_term, 97
k_pass_terminal_to_next_child, 97
k_unlink
k_file.c, 49
k_unmount
k_fat.c, 31
k_fat.h, 37
k_write
k_file.c, 50

latest_job
job_table_st, 12
log.c
log_blocked, 175
log_close, 176
log_continued, 176
log_create, 176
log_exited, 177
log_get_ticks, 177
log_init, 177
log_nice, 178
log_orphan, 178
log_schedule, 178
log_signaled, 179
log_stopped, 179
log_tick, 180
log_unblocked, 180
log_waited, 180
log_zombie, 181
log.h
log_blocked, 182
log_close, 183
log_continued, 183
log_create, 183
log_exited, 184
log_get_ticks, 184
log_init, 185
log_nice, 185
log_orphan, 186
log_schedule, 186

log_signaled, 187
log_stopped, 187
log_tick, 188
log_unblocked, 188
log_waited, 189
log_zombie, 189
log_blocked
log.c, 175
log.h, 182
log_close
log.c, 176
log.h, 183
log_continued
log.c, 176
log.h, 183
log_create
log.c, 176
log.h, 183
log_exited
log.c, 177
log.h, 184
log_get_ticks
log.c, 177
log.h, 184
log_init
log.c, 177
log.h, 185
log_nice
log.c, 178
log.h, 185
log_orphan
log.c, 178
log.h, 186
log_schedule
log.c, 178
log.h, 186
log_signaled
log.c, 179
log.h, 187
log_stopped
log.c, 179
log.h, 187
log_tick
log.c, 180
log.h, 188
log_unblocked
log.c, 180
log.h, 188
log_waited
log.c, 180
log.h, 189
log_zombie
log.c, 181
log.h, 189

main
pennfat.c, 104
pennos.c, 106
metadata
fat_ut, 8
mode
k_fd_entry_t, 13
mtime
file_st, 9

name
file_st, 9
process_info_t, 20
nex_sibling
pcb_t, 17
next
fat_ut, 8

open_file_idx
k_fd_entry_t, 13

panic
panic.h, 191
panic.c
print_and_abort, 190
panic.h
panic, 191
print_and_abort, 192
par_pid
process_info_t, 20
parent
pcb_t, 17
parse_command
parser.c, 193
parser.h, 197
parsed_command, 15
parseId
parser.c, 194
parser.h, 198
parser.c
JUMP_OUT, 193
parse_command, 193
parseId, 194
print_parsed_command, 195
print_parser_errcode, 195
parser.h
parse_command, 197
parseId, 198
print_parsed_command, 198
print_parser_errcode, 199
pcb_state_t
pcb_state_t.h, 128
pcb_state_t.h
pcb_state_t, 128
pcb_t, 15
alive_child, 16
ign_int, 17
ign_stop, 17
nex_sibling, 17
parent, 17
prev_sibling, 17
process_name, 17
sched_priority, 17

state, 17
terminated_by_signal, 18
thread, 18
vector, 16
waitpid_target, 18
wakeup_time, 18
wstatus, 18
zombie_child, 18
pcb_table_t, 19
avail_process, 19
vector, 19
pennfat.c
main, 104
pennos.c
main, 106
u_init_main, 107
pennos.h
u_init_main, 108
perm
file_st, 10
pid
job_st, 11
process_info_t, 20
prev_sibling
pcb_t, 17
print_and_abort
panic.c, 190
panic.h, 192
print_file_entry
s_path.c, 91
print_job_status
job_table.c, 165
job_table_t.h, 168
print_parsed_command
parser.c, 195
parser.h, 198
print_parser_errcode
parser.c, 195
parser.h, 199
priority
process_info_t, 20
process_info_t, 20
name, 20
par_pid, 20
pid, 20
priority, 20
state, 21
process_name
pcb_t, 17
process_spawner
s_lifecycle.c, 115

refcount
k_fd_entry_t, 13
k_open_file_entry_t, 14
run_command
commands.h, 158

s_can_sig_int
s_signal.c, 152
s_signal.h, 154
s_can_sig_stop
s_signal.c, 152
s_signal.h, 155
s_chmod
s_file.c, 51
s_file.h, 57
s_claim_terminal
s_term.c, 98
s_term.h, 102
s_close
s_file.c, 52
s_file.h, 57
s_exit
s_lifecycle.c, 116
s_lifecycle.h, 117
s_ext_open
s_file.c, 52
s_file.h, 58
s_fat.c
s_has_mount, 38
s_mkfs, 38
s_mount, 39
s_unmount, 39
s_fat.h
s_has_mount, 40
s_mkfs, 40
s_mount, 41
s_unmount, 41
s_file.c
s_chmod, 51
s_close, 52
s_ext_open, 52
s_get_open_mode, 53
s_lseek, 53
s_mv, 53
s_open, 54
s_read, 54
s_same_source, 55
s_unlink, 55
s_write, 56
s_file.h
s_chmod, 57
s_close, 57
s_ext_open, 58
s_get_open_mode, 58
s_lseek, 59
s_mv, 59
s_open, 59
s_read, 60
s_same_source, 60
s_unlink, 61
s_write, 61
s_fprintf
s_term.h, 100
s_get_open_mode
s_file.c, 53

s_file.h, 58
s_get_terminal_owner
s_term.c, 99
s_term.h, 102
s_get_terminal_pass_parent
s_term.c, 99
s_term.h, 103
s_getpid
s_schedule.c, 146
s_schedule.h, 147
s_has_mount
s_fat.c, 38
s_fat.h, 40
s_ignore_int
s_signal.c, 152
s_signal.h, 156
s_ignore_stop
s_signal.c, 153
s_signal.h, 156
s_kill
s_signal.c, 153
s_signal.h, 157
s_lifecycle.c
process_spawner, 115
s_exit, 116
s_spawn, 116
s_waitpid, 116
s_lifecycle.h
s_exit, 117
s_spawn, 118
s_waitpid, 118
s_logout
s_signal.c, 154
s_signal.h, 157
s_ls
s_path.c, 92
s_path.h, 92
s_lseek
s_file.c, 53
s_file.h, 59
s_mkfs
s_fat.c, 38
s_fat.h, 40
s_mount
s_fat.c, 39
s_fat.h, 41
s_mv
s_file.c, 53
s_file.h, 59
s_nice
s_pcb.c, 130
s_pcb.h, 132
s_open
s_file.c, 54
s_file.h, 59
s_pass_terminal_to_next_child
s_term.c, 99
s_term.h, 103
s_path.c
print_file_entry, 91
s_ls, 92
s_path.h
s_ls, 92
s_pcb.c
s_nice, 130
s_proc_state, 131
s_ps, 131
s_pcb.h
s_nice, 132
s_proc_state, 133
s_ps, 133
s_perror
s_term.h, 101
s_printf
s_term.h, 101
s_proc_state
s_pcb.c, 131
s_pcb.h, 133
s_ps
s_pcb.c, 131
s_pcb.h, 133
s_read
s_file.c, 54
s_file.h, 60
s_same_source
s_file.c, 55
s_file.h, 60
s_schedule.c
s_getpid, 146
s_sleep, 146
s_schedule.h
s_getpid, 147
s_sleep, 147
s_signal.c
s_can_sig_int, 152
s_can_sig_stop, 152
s_ignore_int, 152
s_ignore_stop, 153
s_kill, 153
s_logout, 154
s_signal.h
s_can_sig_int, 154
s_can_sig_stop, 155
s_ignore_int, 156
s_ignore_stop, 156
s_kill, 157
s_logout, 157
s_sleep
s_schedule.c, 146
s_schedule.h, 147
s_spawn
s_lifecycle.c, 116
s_lifecycle.h, 118
s_term.c
s_claim_terminal, 98
s_get_terminal_owner, 99

s_get_terminal_pass_parent, 99
s_pass_terminal_to_next_child, 99
s_term.h
s_claim_terminal, 102
s_fprintf, 100
s_get_terminal_owner, 102
s_get_terminal_pass_parent, 103
s_pass_terminal_to_next_child, 103
s_perror, 101
s_printf, 101
s_unlink
s_file.c, 55
s_file.h, 61
s_unmount
s_fat.c, 39
s_fat.h, 41
s_waitpid
s_lifecycle.c, 116
s_lifecycle.h, 118
s_write
s_file.c, 56
s_file.h, 61
sched_priority
pcb_t, 17
size
file_st, 10
spthread_fwd_args_st, 21
spthread_meta_st, 21
spthread_signal_args_st, 22
spthread_st, 22
src/io/fat/k_fat.c, 25
src/io/fat/k_fat.h, 31
src/io/fat/s_fat.c, 37
src/io/fat/s_fat.h, 39
src/io/file/h_file.c, 42
src/io/file/k_file.c, 44
src/io/file/s_file.c, 50
src/io/file/s_file.h, 56
src/io/filetable/filetable.h, 62
src/io/filetable/k_filetable.c, 62
src/io/filetable/k_filetable.h, 73
src/io/fstd.h, 83
src/io/path/k_path.c, 83
src/io/path/k_path.h, 87
src/io/path/s_path.c, 91
src/io/path/s_path.h, 92
src/io/term/k_term.c, 93
src/io/term/k_term.h, 95
src/io/term/s_term.c, 98
src/io/term/s_term.h, 99
src/pennfat.c, 103
src/pennfat.h, 105
src/pennos.c, 105
src/pennos.h, 107
src/process/lifecycle/k_lifecycle.c, 109
src/process/lifecycle/k_lifecycle.h, 112
src/process/lifecycle/s_lifecycle.c, 115
src/process/lifecycle/s_lifecycle.h, 117
src/process/lifecycle/thread_info_t.h, 119
src/process/pcb/k_pcb.c, 120
src/process/pcb/k_pcb.h, 124
src/process/pcb/pcb_state_t.h, 128
src/process/pcb/pcb_t.h, 128
src/process/pcb/pcb_table_t.h, 129
src/process/pcb/s_pcb.c, 130
src/process/pcb/s_pcb.h, 131
src/process/pstd.h, 133
src/process/schedule/k_schedule.c, 134
src/process/schedule/k_schedule.h, 139
src/process/schedule/s_schedule.c, 145
src/process/schedule/s_schedule.h, 146
src/process/signal/k_signal.c, 148
src/process/signal/k_signal.h, 149
src/process/signal/s_signal.c, 151
src/process/signal/s_signal.h, 154
src/shell/commands.c, 158
src/shell/commands.h, 158
src/shell/file/u_file.h, 160
src/shell/job/job_table.c, 163
src/shell/job/job_table_t.h, 166
src/shell/job/u_job.h, 169
src/shell/process/u_process.h, 171
src/shell/u_shell.h, 173
src/util/log.c, 174
src/util/log.h, 181
src/util/panic.c, 190
src/util/panic.h, 191
src/util/parser.c, 192
src/util/parser.h, 195
state
pcb_t, 17
process_info_t, 21

target_offset
k_fd_entry_t, 13
terminated_by_signal
pcb_t, 18
thread
pcb_t, 18
thread_info_st, 22
thread_info_t
thread_info_t.h, 120
thread_info_t.h
thread_info_t, 120
tot_blocks
fat_metadata_st, 7
transfer_parent
k_lifecycle.c, 111
type
file_st, 10

u_bg
u_job.h, 169
u_cat
u_file.h, 160
u_chmod
u_file.h, 161

u_cp
u_file.h, 161
u_echo
commands.h, 159
u_fg
u_job.h, 170
u_file.h
u_cat, 160
u_chmod, 161
u_cp, 161
u_ls, 161
u_mv, 162
u_rm, 162
u_touch, 162
u_init_main
pennos.c, 107
pennos.h, 108
u_job.h
u_bg, 169
u_fg, 170
u_jobs, 170
u_jobs
u_job.h, 170
u_kill
u_process.h, 171
u_logout
u_process.h, 172
u_ls
u_file.h, 161
u_man
commands.h, 159
u_mv
u_file.h, 162
u_nice
u_process.h, 172
u_nice_pid
u_process.h, 172
u_process.h
u_kill, 171
u_logout, 172
u_nice, 172
u_nice_pid, 172
u_ps, 173
u_sleep, 173
u_ps
u_process.h, 173
u_rm
u_file.h, 162
u_shell
u_shell.h, 174
u_shell.h
u_shell, 174
u_sleep
u_process.h, 173
u_touch
u_file.h, 162

vector
job_table_st, 12
pcb_t, 16
pcb_table_t, 19
vector_info_st, 23
virtual_offset
k_fd_entry_t, 13

waitpid_target
pcb_t, 18
wakeup_time
pcb_t, 18
wstatus
pcb_t, 18

zombie_child
pcb_t, 18